agrupamento de escolas de rio tinto n.º3

aert3

Criação de páginas Web

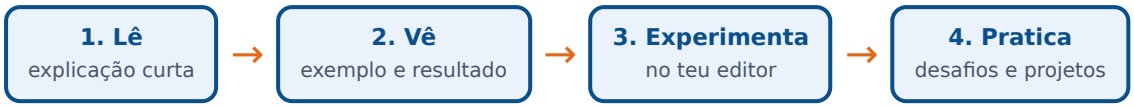
HTML5 • CSS • JavaScript

Sebenta de Estudo



Como usar esta sebenta

Esta sebenta ensina a criar websites **aprendendo a fazer**. Cada tema segue sempre o mesmo caminho: **explicação curta → exemplo → resultado → exercício**.



As caixas que vais encontrar

💡 DICA
Um truque ou um atalho útil.

⚠️ ATENÇÃO
Um cuidado a ter, para evitar problemas.

❌ ERRO FREQUENTE
Um erro que acontece muitas vezes e como o evitar.

✅ BOA PRÁTICA
A forma profissional de fazer as coisas.

🧪 EXPERIMENTA TU
Uma pequena atividade para fazeres já, no teu computador.

💻 CÓDIGO
Código para escreveres, copiares ou estudares.

🚀 DESAFIO
Um desafio maior, ou uma extensão de um projeto.

🧩 DESAFIOS DE CÓDIGO
Exercícios curtos (5 a 15 minutos): completar, encontrar erros, prever resultados, corrigir e modificar. As soluções estão no **Anexo A**.

O que precisas

- Um **navegador** (Chrome, Edge ou Firefox).
- Um **editor de código**: o **Visual Studio Code**, com a extensão **Live Server**, é a escolha recomendada.
- Uma **pasta de projeto** para cada site (o capítulo 1 mostra como a organizar).

O projeto progressivo

Ao longo da sebenta, vais construir o **website do Clube de Fotografia da Escola** em **14 etapas**. Os cartões laranja «🏗️ Projeto progressivo» mostram o que acrescentar em cada etapa e o resultado esperado. No fim, tens o site completo no **Anexo B**. Todos os resultados mostrados foram gerados a partir do próprio código apresentado.

Índice

Parte A · HTML5

1 Introdução ao desenvolvimento Web 5

2 A página HTML5 base 8

3 Principais tags HTML 12

4 Tabelas 29

5 Formulários 36

6 HTML semântico 43

7 Multimédia 49

8 iframe: conteúdo incorporado 51

Parte B · CSS3

9 Introdução ao CSS 54

10 Seletores CSS 59

11 Propriedades CSS mais usadas 65

12 O modelo de caixa (Box Model) 77

13 Flexbox 82

14 CSS Grid 90

15 Design responsivo 97

Parte C · JavaScript

16 Introdução ao JavaScript 104

17 JavaScript e DOM 110

Parte D · Prática

18 O projeto progressivo: visão geral 118

19 Erros frequentes e como os resolver 121

20 Boas práticas 125

21 Mini-projetos 127

22 Projeto final: o teu website 133

Anexos

A Soluções dos desafios de código 135

B Website de referência: código completo e imagens 141

agrupamento de escolas de rio tinto n.º3

aert3

Linguagem HTML5

Sebenta de Estudo



PARTE A · CAPÍTULO 1

Introdução ao desenvolvimento Web

Antes de escrever código, vamos perceber como funciona a Web, o que é uma página, um website e qual o papel de cada tecnologia.

Neste capítulo vais aprender a:

- distinguir página Web de website;
- explicar o que faz o navegador;
- distinguir HTML, CSS e JavaScript;
- criar a pasta do teu primeiro projeto Web.

1.1 Página Web e website

	Página Web	Website
O que é	Um **documento único**, normalmente um ficheiro HTML, com um endereço próprio.	Um **conjunto de páginas ligadas entre si**, com o mesmo endereço de base (domínio).
Exemplo	`https://escola.pt/contactos.html`	`https://escola.pt` (com todas as suas páginas)
Analogia	Uma **folha** de um livro.	O **livro** completo.



Três páginas ligadas entre si formam um pequeno website. É isso que vais construir ao longo desta sebenta.

1.2 O que faz o navegador?

O **navegador** (Chrome, Edge, Firefox, Safari…) é o programa que **pede** páginas a um servidor, **lê** o código recebido (HTML, CSS e JavaScript) e **desenha** a página no ecrã.



> 💡 **DICA**
> Durante o curso, o servidor é o teu próprio computador: abres os ficheiros diretamente no navegador (ou com a extensão **Live Server** do VS Code).

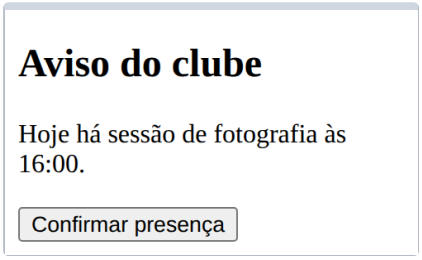
1.3 HTML, CSS e JavaScript

Uma página Web é construída com **três tecnologias** que trabalham em conjunto. Imagina uma **casa**:

Tecnologia	Função	Analogia da casa
HTML	**Estrutura e conteúdo**: títulos, textos, imagens, listas, tabelas…	As paredes e as divisões
CSS	**Aparência e apresentação**: cores, letras, espaços, disposição.	A pintura e a decoração
JavaScript	**Comportamento e interatividade**: reagir a cliques, alterar a página.	A eletricidade e os interruptores

Vê o **mesmo conteúdo** em três fases. Repara que o HTML nunca muda: o CSS acrescenta aparência e o JavaScript acrescenta reação ao clique.

① Só HTML




```
<h2>Aviso do clube</h2>
<p id="msg">Hoje há sessão de
fotografia às 16:00.</p>
<button id="btn">Confirmar
presença</button>
```

② HTML + CSS




```
body { font-family: Arial, sans-
serif; text-align: center;
       background: #e8f1fb;
padding: 20px; }
h2 { color: #0b4f8a; }
button { background: #e06a1b;
color: #fff; border: 0;
         padding: 10px 18px;
border-radius: 6px; }
```

③ + JavaScript (após clicar)




```
document.querySelector("#btn").add
EventListener("click", function ()
{

document.querySelector("#msg").tex
tContent =
        "Presença confirmada. Até
já!";
});
```

> ⚠️ **ATENÇÃO**
> Não precisas de perceber já este código! Serve só para veres o papel de cada tecnologia. Nos próximos capítulos vamos aprender cada uma delas, **por esta ordem: HTML → CSS → JavaScript**.

1.4 As ferramentas de que precisas

Ferramenta	Para quê	Exemplo
Editor de código	Escrever e guardar os ficheiros.	Visual Studio Code (gratuito)
Navegador	Ver e testar as páginas.	Chrome, Edge ou Firefox
Live Server (extensão)	Atualizar a página automaticamente ao guardar.	Botão **Go Live** no VS Code
Ferramentas de programador	Ver o código, medir espaços, testar ecrãs pequenos, ver erros.	Tecla `F12` no navegador

1.5 A pasta do projeto

Cada website vive numa **pasta** própria. Vamos usar sempre esta organização:

```
meu_site/
│
├── index.html
├── css/
│   └── estilo.css
├── js/
│   └── script.js
└── imagens/
```

Item	O que contém
`index.html`	A **página inicial**. É o nome que os servidores procuram por omissão.
`css/estilo.css`	Os **estilos** (aparência) de todo o website.
`js/script.js`	O **código JavaScript** (comportamento).
`imagens/`	As **fotografias e ilustrações** usadas nas páginas.

1. Cria uma pasta chamada `meu_site` num local fácil de encontrar (por exemplo, nos Documentos).
2. Abre o VS Code e escolhe **Ficheiro › Abrir Pasta...**, selecionando essa pasta.
3. No Explorador do VS Code, cria o ficheiro `index.html` e as pastas `css`, `js` e `imagens`.
4. Dentro de `css` cria `estilo.css` e dentro de `js` cria `script.js`.

> ✅ **BOA PRÁTICA**
> Usa nomes de ficheiros e pastas em **letras minúsculas**, **sem espaços nem acentos** (usa `_` ou `-`).
> Exemplo: `sobre-nos.html` em vez de `Sobre Nós.html`.

1.6 O ciclo de trabalho



> ❌ **ERRO FREQUENTE**
> Esquecer de **guardar o ficheiro** (`Ctrl`+`S`) antes de atualizar o navegador. Se o separador do ficheiro tiver um pontinho branco, ainda há alterações por guardar!

> 🧪 **EXPERIMENTA TU 1 — A TUA PASTA DE PROJETO**
> Cria a estrutura de pastas `meu_site` apresentada nesta secção:
> - a pasta `meu_site` com o ficheiro `index.html` (por agora vazio);
> - as pastas `css`, `js` e `imagens`;
> - os ficheiros `estilo.css` e `script.js` nas pastas certas.

> 🎯 **QUAL SERÁ O RESULTADO? — D1 (5 MIN)**
> O Mário criou este ficheiro chamado `Página Inicial.html` e colocou-o dentro de uma pasta `Meu Site`.
> Que **duas más práticas** encontras nos nomes?

📋 Ficha rápida — Introdução

Termo	Significado	Exemplo
`Navegador`	Pede, lê e mostra as páginas	`Chrome, Edge, Firefox`
`HTML`	Estrutura e conteúdo	`<h1>Título</h1>`
`CSS`	Aparência	`h1 { color: blue; }`
`JavaScript`	Comportamento	`alert('Olá!');`
`index.html`	Página inicial do website	`meu_site/index.html`

PARTE A · CAPÍTULO 2

HTML5: a estrutura da página

Todas as páginas HTML5 partem da mesma estrutura mínima. Vais aprendê-la linha a linha e criar a tua primeira página.

Neste capítulo vais aprender a:

- escrever a estrutura mínima de uma página HTML5;
- explicar cada linha dessa estrutura;
- distinguir tag, elemento e atributo;
- usar indentação e comentários.

2.1 A estrutura mínima

💻 **CÓDIGO — INDEX.HTML**

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>A minha primeira página</title>
</head>

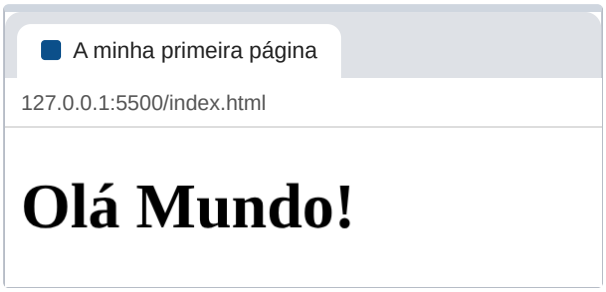
<body>

    <h1>Olá Mundo!</h1>

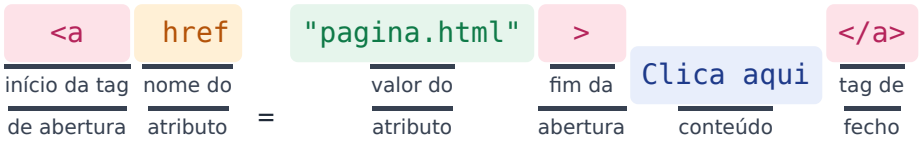
</body>
</html>
```

Linha	Explicação simples
`<!DOCTYPE html>`	Diz ao navegador que o documento é **HTML5**. Fica sempre na primeira linha.
`<html lang="pt">`	Abre o documento. `lang="pt"` indica que o idioma é o português.
`<head>`	Cabeçalho **técnico**: informação sobre a página. **Não aparece** no ecrã.
`<meta charset="UTF-8">`	Permite que os **acentos** e o "ç" apareçam corretamente.
`<meta name="viewport" …>`	Faz a página **ajustar-se ao ecrã** do telemóvel (essencial para páginas responsivas).
`<title>`	Título da página, mostrado no **separador do navegador**.
`<body>`	Corpo da página: tudo o que aqui está **aparece no ecrã**.
`<h1>Olá Mundo!</h1>`	Um título principal. É o nosso primeiro **conteúdo visível**.
`</body>` e `</html>`	Fecham o corpo e o documento.

O resultado no navegador — o título no separador e o texto na página:



2.2 Tag, elemento e atributo



ELEMENTO COMPLETO = tag de abertura + conteúdo + tag de fecho

Conceito	O que é	Exemplo
Tag	A etiqueta entre `< >`.	`<p>` e `</p>`
Elemento	Tag de abertura + conteúdo + tag de fecho.	`<p>Olá!</p>`
Atributo	Informação extra na tag de abertura: `nome="valor"`.	`<a href="pagina.html">`
Tag de abertura	Começa o elemento.	`<p>`
Tag de fecho	Termina o elemento; tem uma barra `/`.	`</p>`
Elemento vazio	Não tem conteúdo nem tag de fecho.	` ` `<img>` `<hr>`

Os elementos podem estar **dentro de outros** (aninhamento). Regra de ouro: **o último a abrir é o primeiro a fechar**.

❌ **TAGS CRUZADAS**

```
<p>Texto <strong>importante</p></strong>
```

✅ **FAZER**

```
<p>Texto <strong>importante</strong></p>
```

❌ **ERRO FREQUENTE**

Esquecer a tag de fecho, como em `<p>Texto` sem `</p>`. O navegador tenta adivinhar, mas a página pode ficar estragada. Confirma sempre que **cada tag aberta foi fechada**.

2.3 Indentação e comentários

Indentar é afastar as linhas para a direita (com a tecla `Tab` ou 4 espaços) conforme o nível de aninhamento. O navegador ignora a indentação, mas ela torna o código muito mais **legível**.

❌ **SEM INDENTAÇÃO**

```
<body>
<h1>Animais</h1>
<ul>
<li>Cão</li>
<li>Gato</li>
</ul>
</body>
```

✅ **COM INDENTAÇÃO**

```
<body>
    <h1>Animais</h1>
    <ul>
        <li>Cão</li>
        <li>Gato</li>
    </ul>
</body>
```

`<!-- comentário -->` — COMENTÁRIOS

Função	Nota escrita no código que **não aparece** na página. Serve para explicar ou para desativar código.
Sintaxe	`<!-- texto do comentário -->`
Quando usar	Para explicar zonas do código ou "desligar" temporariamente uma parte.

EXEMPLO

```
<!-- Título principal da página -->
<h1>Olá Mundo!</h1>

<!-- <p>Este parágrafo está desativado.</p> -->
<p>Este parágrafo aparece.</p>
```

RESULTADO ESPERADO

Olá Mundo!

Este parágrafo aparece.

> 💡 **DICA**
> No VS Code, seleciona as linhas e prime `Ctrl`+`/` para comentar ou descomentar.

> ❌ **ERRO FREQUENTE**
> Pensar que os comentários são secretos: qualquer pessoa pode ver o código-fonte da página (`Ctrl`+`U`).

Um **ficheiro HTML** é apenas texto. Para o veres no navegador, guarda-o com a extensão `.html` e abre-o (duplo clique, ou botão **Go Live** do Live Server).

🏗️ Projeto progressivo · Etapa 1 de 14 — A PRIMEIRA PÁGINA

O QUE ACRESCENTAS

```
<h1>A minha primeira página</h1>
<p>Olá Mundo!</p>
```

RESULTADO ESPERADO

A minha primeira página

Olá Mundo!

*A partir daqui vamos construir, capítulo a capítulo, o website do **Clube de Fotografia da Escola**.*

🧪 EXPERIMENTA TU 2 — A TUA PRIMEIRA PÁGINA

Escreve, no ficheiro `index.html` da tua pasta `meu_site`, a estrutura mínima do início deste capítulo e:

- altera o `<title>` para o nome do teu website;
- escreve um `<h1>` com o teu nome e um `<p>` com uma frase sobre ti;
- acrescenta um comentário a explicar o que é o `<head>`;
- guarda e abre no navegador; confirma que o título aparece no **separador**.

🎯 QUAL SERÁ O RESULTADO? — D2 (5 MIN)

Sem testar, diz o que aparece na página:

```
<h1>Bom dia</h1>
<!-- <p>Primeira linha</p> -->
<p>Segunda linha</p>
```

🔍 ENCONTRA O ERRO — D3 (5 MIN)

Este código tem **dois erros**. Encontra-os.

```
<h1>Bem-vindo</h1>
<p>Esta página é <strong>muito</p></strong>
<p>Fim
```

🧩 COMPLETA O CÓDIGO — D4 (10 MIN)

Completa os espaços `___` para criar uma página válida.

```
<!DOCTYPE html>
<___ lang="pt">
<___>
    <meta charset="UTF-8">
    <___>Teste</title>
</head>
<___>
    <h1>Olá!</h1>
</body>
___
```

📋 Ficha rápida — HTML5: estrutura

Elemento	Função	Exemplo
`<!DOCTYPE html>`	Declara HTML5	`<!DOCTYPE html>`
`<html>`	Elemento raiz	`<html lang="pt">`
`<head>`	Informação técnica	`<head>…</head>`
`<meta>`	Metadados	`<meta charset="UTF-8">`
`<title>`	Título no separador	`<title>Olá</title>`
`<body>`	Conteúdo visível	`<body>…</body>`
`<!-- -->`	Comentário	`<!-- nota -->`

PARTE A · CAPÍTULO 3

Principais tags HTML

As tags mais úteis para um principiante, organizadas por categorias. Cada uma vem com função, sintaxe, exemplo e resultado.

Neste capítulo vais aprender a:

- escolher a tag certa para cada tipo de conteúdo;
- escrever títulos, parágrafos e formatação de texto;
- criar listas e hiperligações;
- inserir imagens com caminhos relativos corretos.

3.1 Estrutura

<html>

ESTRUTURA

Função	Elemento raiz: contém todo o documento (`<head>` e `<body>`).
Sintaxe	`<html lang="pt"> … </html>`
Quando usar	Sempre, logo a seguir ao `<!DOCTYPE html>`.

EXEMPLO

```
<html lang="pt">
    <head> ... </head>
    <body> ... </body>
</html>
```

RESULTADO ESPERADO

Sem resultado visível: é o "contentor" de toda a página.

> 💡 **DICA**
> O atributo `lang` ajuda leitores de ecrã e tradutores automáticos.

<head>

ESTRUTURA

Função	Guarda informação **sobre** a página (metadados, título, ligação ao CSS).
Sintaxe	`<head> … </head>`
Quando usar	Sempre, antes do `<body>`.

EXEMPLO

```
<head>
    <meta charset="UTF-8">
    <title>O meu website</title>
    <link rel="stylesheet" href="css/estilo.css">
</head>
```

RESULTADO ESPERADO

Nada do que está no `<head>` aparece na página (exceto o título, no separador).

> ❌ **ERRO FREQUENTE**
> Escrever texto da página dentro do `<head>`: não aparece no ecrã!

<meta>

ESTRUTURA

Função	Define **metadados**: codificação, comportamento em telemóveis, descrição...
Sintaxe	`<meta charset="UTF-8">`
Quando usar	Dentro do `<head>`. O `charset` e o `viewport` devem estar em todas as páginas.

EXEMPLO

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<meta name="description" content="Clube de
Fotografia da Escola">
```

RESULTADO ESPERADO

Sem resultado visível. Sem `charset="UTF-8"`, "Aplicação" poderia aparecer como "AplicaÃ§Ã£o".

> 💡 **DICA**
> É um elemento **vazio**: não tem tag de fecho.

> ❌ **ERRO FREQUENTE**
> Esquecer o `charset`: os acentos podem aparecer estragados ("AplicaÃ§Ã£o").

<title>

ESTRUTURA

Função	Define o título da página: aparece no **separador** do navegador e nos resultados de pesquisa.
Sintaxe	`<title>Texto</title>`
Quando usar	Uma vez em cada página, dentro do `<head>`.

EXEMPLO

```
<title>Clube de Fotografia da Escola</title>
```

RESULTADO ESPERADO



> 💡 **DICA**
> Escolhe um título curto e descritivo. Não confundas com o `<h1>`, que aparece *dentro* da página.

<body>

ESTRUTURA

Função	Contém **todo o conteúdo visível** da página.
Sintaxe	`<body> … </body>`
Quando usar	Sempre, a seguir ao `</head>`. Só existe um por página.

EXEMPLO

```
<body>
    <h1>Bem-vindo!</h1>
    <p>Este texto aparece na página.</p>
</body>
```

RESULTADO ESPERADO

Bem-vindo!

Este texto aparece na página.

> 💡 **DICA**
> Tudo o que vais aprender a seguir vai dentro do `<body>`.

3.2 Texto

<h1> … <h6>

TEXTO

Função	Criam **títulos e subtítulos**. O `h1` é o mais importante; o `h6` o menos importante.
Sintaxe	`<h1>Título</h1> … <h6>Título</h6>`
Quando usar	Para organizar o conteúdo em níveis, como os capítulos de um livro.

EXEMPLO

```
<h1>Título principal (h1)</h1>
<h2>Subtítulo (h2)</h2>
<h3>Secção (h3)</h3>
<h4>Nível 4</h4>
<h5>Nível 5</h5>
<h6>Nível 6</h6>
```

RESULTADO ESPERADO

Título principal (h1)

Subtítulo (h2)

Secção (h3)

Nível 4

Nível 5

Nível 6

✅ **BOA PRÁTICA**

Usa **um só** `<h1>` por página e não saltes níveis (de `h2` passa para `h3`).

❌ **ERRO FREQUENTE**

Usar um título só para "aumentar a letra". O tamanho é tarefa do CSS.

<p>

TEXTO

Função	Define um **parágrafo**.
Sintaxe	`<p>Texto</p>`
Quando usar	Sempre que tenhas um bloco de texto corrido.

EXEMPLO

```
<p>O HTML serve para estruturar páginas Web.</p>
<p>Este é outro parágrafo, separado do anterior.
</p>
```

RESULTADO ESPERADO

O HTML serve para estruturar páginas Web.

Este é outro parágrafo, separado do anterior.

❌ **ERRO FREQUENTE**

Carregar em `Enter` no editor **não** cria nova linha na página: o navegador junta tudo. Usa novos `<p>`.

 TEXTO

Função Insere uma **quebra de linha** dentro do mesmo parágrafo. Elemento vazio.

Sintaxe `<br>`

Quando usar Em moradas ou versos, quando queres mudar de linha **sem** criar novo parágrafo.

EXEMPLO

```
<p>
    Escola Secundária<br>
    Rua das Flores, 25<br>
    4000-000 Porto
</p>
```

RESULTADO ESPERADO

Escola Secundária
Rua das Flores, 25
4000-000 Porto

> 💡 **DICA**
> Não uses vários `<br>` seguidos para "criar espaço": isso é função do CSS.

<hr> TEXTO

Função Desenha uma **linha horizontal** que separa temas. Elemento vazio.

Sintaxe `<hr>`

Quando usar Para marcar uma mudança de assunto.

EXEMPLO

```
<p>Fim do primeiro tema.</p>
<hr>
<p>Início do segundo tema.</p>
```

RESULTADO ESPERADO

Fim do primeiro tema.

Início do segundo tema.

TEXTO

Função Marca texto de **grande importância** (negrito).

Sintaxe `<strong>texto</strong>`

Quando usar Para avisos e palavras-chave essenciais.

EXEMPLO

```
<p>Não te esqueças de <strong>guardar o
ficheiro</strong>.</p>
```

RESULTADO ESPERADO

Não te esqueças de **guardar o ficheiro**.

> 💡 **DICA**
> Prefere `<strong>` a `<b>`: tem significado, não é só visual.

TEXTO

Função Dá **ênfase** a uma palavra (itálico).

Sintaxe `<em>texto</em>`

Quando usar Para realçar a entoação ou palavras estrangeiras.

EXEMPLO

```
<p>Esta página é <em>muito</em> simples.</p>
```

RESULTADO ESPERADO

Esta página é *muito* simples.

<mark>

TEXTO

Função Destaca texto, como um marcador fluorescente.

Sintaxe `<mark>texto</mark>`

Quando usar Para chamar a atenção para um termo relevante.

EXEMPLO

```
<p>O elemento <mark>mark</mark> serve para
destacar.</p>
```

RESULTADO ESPERADO

O elemento mark serve para destacar.

<small>

TEXTO

Função Mostra texto em tamanho reduzido.

Sintaxe `<small>texto</small>`

Quando usar Para notas legais e informação secundária.

EXEMPLO

```
<p>Preço: 10 €<br><small>IVA incluído.</small></p>
```

RESULTADO ESPERADO

Preço: 10 €
IVA incluído.

<sub>

TEXTO

Função Escreve em **subscrito**.

Sintaxe `<sub>texto</sub>`

Quando usar Em fórmulas químicas e matemáticas.

EXEMPLO

```
<p>A fórmula da água é H<sub>2</sub>O.</p>
```

RESULTADO ESPERADO

A fórmula da água é H_2O.

<sup> TEXTO

Função	Escreve em **sobrescrito**.
Sintaxe	`<sup>texto</sup>`
Quando usar	Em potências e unidades (m², cm³).

EXEMPLO

```
<p>A área é de 25 m<sup>2</sup>.</p>
```

RESULTADO ESPERADO

A área é de 25 m^{2}.

🏗️ Projeto progressivo · Etapa 2 de 14 TÍTULOS E PARÁGRAFOS

O QUE ACRESCENTAS

```
<h1>Clube de Fotografia da Escola</h1>
<p>Olá Mundo! Bem-vindo ao <strong>Clube de
Fotografia</strong>.</p>

<h2>Quem somos</h2>
<p>O Clube de Fotografia reúne alunos e professores
que gostam de captar o mundo através de uma
objetiva.</p>
<p>Nas sessões aprendemos a compor imagens, a usar
a luz e a contar histórias com fotografias.</p>

<h2>O que fazemos</h2>
<p>Todas as semanas realizamos atividades para
todos os níveis:</p>
```

RESULTADO ESPERADO

Clube de Fotografia da Escola

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.

O que fazemos

Todas as semanas realizamos atividades para todos os níveis:

Passámos a ter título principal, subtítulos, parágrafos e formatação de texto.

🧪 EXPERIMENTA TU 3 — PÁGINA SOBRE MIM

Cria `sobre.html` com **título** (`h1`), **dois subtítulos** (`h2`), **três parágrafos** e, pelo menos, uma palavra em `strong` , uma em `em` e uma em `mark` . As imagens, a lista e a ligação ficam para mais adiante!

🎯 QUAL SERÁ O RESULTADO? — D5 (5 MIN)

O que aparece no navegador?

```
<p>Linha um<br>linha <strong>dois</strong></p>
<hr>
<p>H<sub>2</sub>O</p>
```

🔍 ENCONTRA O ERRO — D6 (5 MIN)

A Ana queria um título, mas a página não aparece com o aspeto certo. Onde está o erro?

```
<h1>Bem-vindo<h1>
<p>Este é o meu site.</p>
```

3.3 Organização de conteúdos

— ORGANIZAÇÃO

Função	Cria um **bloco genérico** para agrupar elementos. Não tem significado próprio.
Sintaxe	`<div> … </div>`
Quando usar	Para agrupar elementos quando nenhuma tag semântica (capítulo 6) se adequa, sobretudo para aplicar CSS.

EXEMPLO

```
<div>
    <h3>Aviso</h3>
    <p>Este bloco agrupa um título e um parágrafo.
</p>
</div>
<div>
    <h3>Outro grupo</h3>
    <p>Cada div começa numa nova linha.</p>
</div>
```

RESULTADO ESPERADO

Aviso

Este bloco agrupa um título e um parágrafo.

Outro grupo

Cada div começa numa nova linha.

Sem CSS, a `div` não altera o aspeto da página. A sua utilidade vem com a classe (`class`) e o CSS.

> 💡 **DICA**
> A `div` é um elemento de **bloco**: ocupa a linha toda.

— ORGANIZAÇÃO

Função	Delimita uma **parte de texto** dentro de uma linha, sem a quebrar.
Sintaxe	`<span> … </span>`
Quando usar	Para isolar uma palavra que queiras estilizar com CSS.

EXEMPLO

```
<p>O meu animal preferido é o <span>cão</span>.</p>
```

RESULTADO ESPERADO

O meu animal preferido é o cão.

Também não altera o aspeto, até lhe aplicares CSS.

> 💡 **DICA**
> O `span` é um elemento **de linha**: fica no meio do texto.

	Elementos de bloco	Elementos de linha
Comportamento	Começam numa nova linha e ocupam toda a largura.	Ficam dentro da linha, só ocupam o espaço necessário.
Exemplos	`<h1>` `<p>` `<div>` `<ul>` `<table>`	`<strong>` `<em>` `<span>` `<a>` `<img>`

3.4 Listas

 — LISTAS

Função	Cria uma **lista não ordenada**, com marcadores.
Sintaxe	`<ul> <li>…</li> </ul>`
Quando usar	Quando a **ordem não importa**: compras, características, menus.

EXEMPLO

```
<ul>
    <li>Cão</li>
    <li>Gato</li>
    <li>Coelho</li>
</ul>
```

RESULTADO ESPERADO

- Cão
- Gato
- Coelho

> 💡 **DICA**
> **ul** = *unordered list*.

 — LISTAS

Função	Cria uma **lista ordenada**, numerada automaticamente.
Sintaxe	`<ol> <li>…</li> </ol>`
Quando usar	Quando a **ordem importa**: passos, instruções, classificações.

EXEMPLO

```
<ol>
    <li>Abrir o editor</li>
    <li>Escrever o código</li>
    <li>Guardar o ficheiro</li>
</ol>
```

RESULTADO ESPERADO

1. Abrir o editor
2. Escrever o código
3. Guardar o ficheiro

> 💡 **DICA**
> **ol** = *ordered list*. Com `<ol start="5">` a contagem começa no 5.

 — LISTAS

Função	Define **cada item** de uma lista.
Sintaxe	`<li>Item</li>`
Quando usar	Sempre **dentro** de `<ul>` ou `<ol>`. Pode conter outra lista.

EXEMPLO

```
<ul>
    <li>Primeiro item</li>
    <li>Segundo item
        <ul>
            <li>Sub-item A</li>
            <li>Sub-item B</li>
        </ul>
    </li>
</ul>
```

RESULTADO ESPERADO

- Primeiro item
- Segundo item
  - Sub-item A
  - Sub-item B

> ❌ **ERRO FREQUENTE**
> Colocar `<li>` fora de uma lista, ou texto solto dentro de `<ul>` sem o envolver em `<li>`.

🏗️ Projeto progressivo · Etapa 3 de 14

O QUE ACRESCENTAS

```
<h2>O que fazemos</h2>
<p>Todas as semanas realizamos atividades para
todos os níveis:</p>
<ul>
    <li>Saídas fotográficas pela cidade</li>
    <li>Workshops de composição e luz</li>
    <li>Exposições de fotografias na escola</li>
</ul>

<h2>Como participar</h2>
<ol>
    <li>Preenche o formulário de contacto</li>
    <li>Traz a tua câmara ou telemóvel</li>
    <li>Vem à primeira sessão!</li>
</ol>
```

RESULTADO ESPERADO

Clube de Fotografia da Escola

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.

O que fazemos

Todas as semanas realizamos atividades para todos os níveis:

- Saídas fotográficas pela cidade
- Workshops de composição e luz
- Exposições de fotografias na escola

Como participar

1. Preenche o formulário de contacto
2. Traz a tua câmara ou telemóvel
3. Vem à primeira sessão!

🧪 EXPERIMENTA TU 4 — UMA RECEITA

Cria `receita.html` com o nome de uma receita (`h1`), uma lista **não ordenada** de quatro ingredientes e uma lista **ordenada** com quatro passos. Num dos passos, cria uma lista aninhada com dois detalhes.

🧩 COMPLETA O CÓDIGO — D7 (5 MIN)

Completa os espaços para criar uma lista numerada com três países.

```
___
    <li>Portugal</li>
    ___
    <li>França</li>
___
```

🛠️ CORRIGE O CÓDIGO — D8 (5 MIN)

A lista não está bem construída. Corrige-a.

```
<ul>
    <li>Maçã</li>
    Banana
    <li>Laranja</li>
```

3.5 Hiperligações

<a>

HIPERLIGAÇÕES

Função	Cria uma **hiperligação** (*anchor*) para outra página, ficheiro ou endereço de email.
Sintaxe	`<a href="destino">Texto</a>`
Quando usar	Sempre que quiseres que o utilizador clique para ir a outro sítio.

EXEMPLO

```
<!-- 1. Outra página do mesmo website -->
<p><a href="galeria.html">Ver a galeria</a></p>

<!-- 2. Site externo -->
<p><a href="https://pt.wikipedia.org/wiki/Fotografia">Fotografia na Wikipédia</a></p>

<!-- 3. Endereço de email -->
<p><a href="mailto:clube@escola.pt">Enviar email ao clube</a></p>

<!-- 4. Abrir num novo separador -->
<p><a href="https://pt.wikipedia.org" target="_blank" rel="noopener noreferrer">Wikipédia (novo
separador)</a></p>
```

RESULTADO ESPERADO

Ver a galeria

Fotografia na Wikipédia

Enviar email ao clube

Wikipédia (novo separador)

✅ **BOA PRÁTICA**
O texto da ligação deve ser **descritivo**: prefere “ver o horário das sessões” a “clica aqui”.

❌ **ERRO FREQUENTE**
Esquecer `https://` nos endereços externos: o navegador procura um ficheiro local em vez de ir à Internet.

href=""

ATRIBUTO DE

Função	Indica o **destino** da ligação. É o atributo obrigatório.
Sintaxe	`href="destino"`
Quando usar	Em todas as ligações. O valor depende do tipo de destino (tabela abaixo).

EXEMPLO

```
<p><a href="index.html">Página do site</a></p>
<p><a href="paginas/sobre.html">Página numa
subpasta</a></p>
<p><a href="#contactos">Zona da mesma página</a>
</p>
```

RESULTADO ESPERADO

Página do site

Página numa subpasta

Zona da mesma página

Destino	Valor de href	Exemplo
Outra página do site	Nome do ficheiro	`href="galeria.html"`
Página numa subpasta	Pasta/ficheiro	`href="paginas/sobre.html"`
Site externo	Endereço completo com `https://`	`href="https://www.wikipedia.org"`
Parte da mesma página	`#` + `id`	`href="#contactos"`
Email	`mailto:` + endereço	`href="mailto:clube@escola.pt"`

`target=""` ATRIBUTO DE

Função Diz **onde abrir** a ligação. Com `_blank` abre num novo separador.

Sintaxe `target="_blank"`

Quando usar Em ligações **externas**, para o utilizador não perder a tua página.

EXEMPLO

```
<a href="https://pt.wikipedia.org" target="_blank"
rel="noopener noreferrer">
    Abrir noutro separador
</a>
```

RESULTADO ESPERADO

Abrir noutro separador

> 💡 **DICA**
> Junta `rel="noopener noreferrer"` às ligações com `_blank`: é uma boa prática de segurança.

> ❌ **ERRO FREQUENTE**
> Usar `target="_blank"` em todas as ligações. Nas ligações entre páginas do teu site, é melhor abrir no mesmo separador.

`title=""` ATRIBUTO DE

Função Acrescenta um **texto de ajuda** que aparece quando se pousa o rato sobre a ligação.

Sintaxe `title="texto de ajuda"`

Quando usar Quando o texto da ligação, sozinho, não explica tudo.

EXEMPLO

```
<a href="galeria.html" title="Fotografias dos
nossos alunos">Galeria</a>
```

RESULTADO ESPERADO

Galeria Fotografias dos nossos alunos

Ao pousar o rato sobre a ligação aparece a "dica" (simulação).

🏗️ Projeto progressivo · Etapa 4 de 14

HIPERLIGAÇÕES

O QUE ACRESCENTAS

```
<p>
    <a href="index.html">Início</a> |
    <a href="galeria.html">Galeria</a> |
    <a href="contactos.html">Contactos</a>
</p>

<p>Saber mais: <a
href="https://pt.wikipedia.org/wiki/Fotografia"
target="_blank" rel="noopener
noreferrer">fotografia na Wikipédia</a>.</p>
<p>Contacto: <a
href="mailto:clube@escola.pt">clube@escola.pt</a>
</p>
```

RESULTADO ESPERADO

Clube de Fotografia da Escola

Início | Galeria | Contactos

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.

O que fazemos

Todas as semanas realizamos atividades para todos os níveis:

- Saídas fotográficas pela cidade
- Workshops de composição e luz
- Exposições de fotografias na escola

Como participar

1. Preenche o formulário de contacto

Ainda só existe o `index.html`: as ligações internas ficam prontas para as páginas que vamos criar.

🧪 EXPERIMENTA TU 5 — FAVORITOS

Cria `favoritos.html` com uma lista de **três ligações externas** (uma a abrir em novo separador), uma ligação para `sobre.html`, uma ligação `mailto:` e, no fim, uma ligação com `href="#topo"` que volte ao título (dá `id="topo"` ao `h1`).

🛠️ CORRIGE O CÓDIGO — D9 (5 MIN)

A ligação não funciona como esperado. Corrige-a para abrir num novo separador.

```
<a href=https://www.wikipedia.org>Wikipédia</a>
```

3.6 Imagens

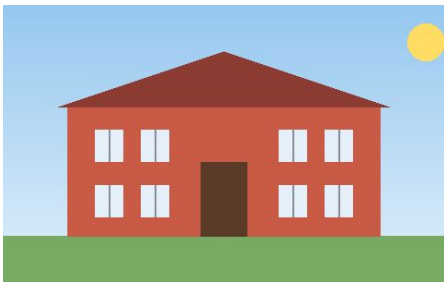
IMAGENS

Função	Insere uma **imagem** na página. Elemento vazio.
Sintaxe	`<img src="ficheiro.jpg" alt="descrição">`
Quando usar	Sempre que quiseres mostrar uma fotografia, ilustração ou logótipo.

EXEMPLO

```
<img src="imagens/escola.jpg" alt="Imagem da
escola" width="220">
```

RESULTADO ESPERADO



> 💡 **DICA**
> Formatos mais usados: `.jpg` (fotografias), `.png` (gráficos, transparências), `.webp` e `.svg`.

> ❌ **ERRO FREQUENTE**
> Nomes de ficheiro com espaços, acentos ou maiúsculas diferentes das do código.

src=""

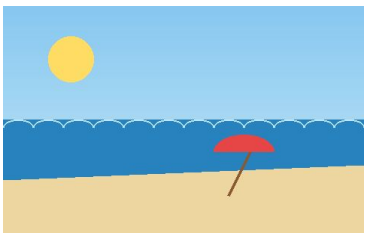
ATRIBUTO DE

Função	Indica o **caminho** do ficheiro da imagem (*source*). Obrigatório.
Sintaxe	`src="caminho/ficheiro.jpg"`
Quando usar	Sempre. Pode ser um caminho **relativo** (ficheiro do teu projeto) ou um endereço da Internet.

EXEMPLO

```
<img src="imagens/praia.jpg" alt="Praia com mar e
areia" width="180">
```

RESULTADO ESPERADO



`alt=""` — ATRIBUTO DE

Função	Texto **alternativo** que descreve a imagem: lido por leitores de ecrã e mostrado se a imagem falhar.
Sintaxe	`alt="descrição"`
Quando usar	**Sempre**. Em imagens só decorativas, usa `alt=""`.

EXEMPLO

```
<img src="imagens/escolaa.jpg" alt="Imagem da
escola" width="200">
```

RESULTADO ESPERADO

Imagem da escola

O nome do ficheiro está errado de propósito (`escolaa.jpg`): vês o texto alternativo em vez da imagem.

> ✅ **BOA PRÁTICA**
> Descreve o que a imagem *mostra*, de forma curta. Não escrevas "imagem de...".

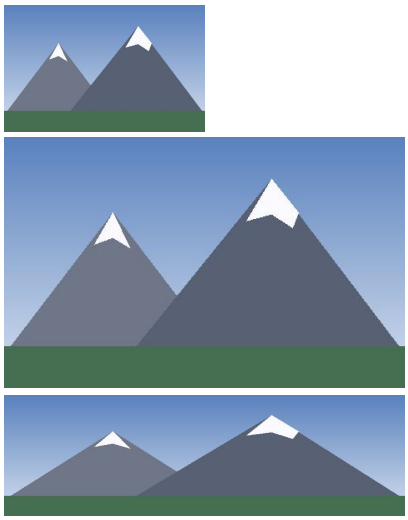
`width="" e height=""` — ATRIBUTOS DE

Função	Definem a **largura** e a **altura** da imagem, em píxeis.
Sintaxe	`width="300" height="200"`
Quando usar	Para controlar o tamanho. Se indicares só um, o outro ajusta-se e a proporção mantém-se.

EXEMPLO

```
<img src="imagens/montanha.jpg" alt="Montanhas"
width="100"><br>
<img src="imagens/montanha.jpg" alt="Montanhas"
width="200"><br>
<img src="imagens/montanha.jpg" alt="Montanhas"
width="200" height="60">
```

RESULTADO ESPERADO



> ❌ **ERRO FREQUENTE**
> Indicar `width` e `height` com proporção diferente da original: a imagem fica **deformada** (terceira imagem).

Caminhos relativos: onde está a imagem?

O `src` diz ao navegador **onde está o ficheiro**, **partindo da pasta onde está a página HTML**. Imagina esta organização:

```
meu_site/
├── index.html              ← usa  imagens/escola.jpg
├── imagens/
│   └── escola.jpg
└── paginas/
    └── sobre.html          ← usa  ../imagens/escola.jpg
```

PÁGINA NA RAIZ (INDEX.HTML)

```
<img src="imagens/escola.jpg" alt="Imagem da escola">
```

*A imagem está numa subpasta **ao lado** de index.html.*

PÁGINA NUMA SUBPASTA (PAGINAS/SOBRE.HTML)

```
<img src="../imagens/escola.jpg" alt="Imagem da escola">
```

`../` *sobe uma pasta e só depois entra em* `imagens`.

A página está em…	A imagem está em…	O src será…
`index.html`	mesma pasta	`src="escola.jpg"`
`index.html`	subpasta `imagens`	`src="imagens/escola.jpg"`
`paginas/sobre.html`	pasta `imagens`, **um nível acima**	`src="../imagens/escola.jpg"`
qualquer página	outro site	`src="https://exemplo.pt/escola.jpg"`

⚠️ **ATENÇÃO**

`../` significa "**sobe uma pasta**". Usa sempre a barra normal `/` (nunca `\`) e mantém as maiúsculas/minúsculas exatamente como no nome do ficheiro.

❌ **ERRO FREQUENTE**

Usar **caminhos absolutos do computador**, como `C:\Users\Ana\Imagens\escola.jpg`. Só funcionam no teu computador! Usa sempre caminhos relativos à pasta do projeto.

🏗️ Projeto progressivo · Etapa 5 de 14

IMAGENS

O QUE ACRESCENTAS

```
<h2>Quem somos</h2>
<p>O Clube de Fotografia reúne alunos e
professores…</p>
<p>Nas sessões aprendemos a compor imagens…</p>
<img src="imagens/escola.jpg" alt="Edifício da
escola" width="360">
```

RESULTADO ESPERADO

Clube de Fotografia da Escola

Início | Galeria | Contactos

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



O ficheiro `escola.jpg` está na pasta `imagens`. No `galeria.html` vamos usar as fotografias dos alunos.

🧪 EXPERIMENTA TU 6 — ANIMAIS.HTML

Cria `animais.html` com um título principal, dois parágrafos, uma lista com três animais, **uma imagem** (na pasta `imagens`, com caminho relativo e `alt`) e **uma ligação para um site** sobre animais.

🧪 EXPERIMENTA TU 7 — PÁGINA COMPLETA SOBRE.HTML

Cria `sobre.html` contendo:

- um título principal;
- dois subtítulos;
- três parágrafos;
- uma imagem;
- uma lista;
- uma hiperligação para um site externo.

🔍 ENCONTRA O ERRO — D10 (10 MIN)

A imagem não aparece. A página está em `meu_site/paginas/sobre.html` e a imagem em `meu_site/imagens/escola.jpg`. Qual é o erro?

```
<img src="imagens/escola.jpg">
```

✏️ MODIFICA O EXEMPLO — D11 (5 MIN)

Altera o código para a imagem ter **250 píxeis de largura**, e um texto alternativo “Fotografia da escola”.

```
<img src="imagens/escola.jpg" alt="Escola">
```

🎯 QUAL SERÁ O RESULTADO? — D12 (10 MIN)

A página `index.html` está na raiz do site. Que imagens aparecem?

```
<img src="imagens/praia.jpg" alt="Praia" width="80">
<img src="../imagens/montanha.jpg" alt="Montanha" width="80">
```

📋 Ficha rápida — Estrutura, texto e conteúdo

Elemento	Função	Exemplo
`<h1>…<h6>`	Títulos	`<h1>Título</h1>`
`<p>`	Parágrafo	`<p>Texto</p>`
` `	Quebra de linha	`Linha 1 Linha 2`
`<hr>`	Linha horizontal	`<hr>`
`<strong>`	Importante (negrito)	`<strong>aviso</strong>`
`<em>`	Ênfase (itálico)	`<em>muito</em>`
`<mark>`	Destaque	`<mark>nota</mark>`
`<small>`	Texto pequeno	`<small>nota</small>`
`<sub> <sup>`	Subscrito e sobrescrito	`H<sub>2</sub>O m<sup>2</sup>`
`<div>`	Bloco genérico	`<div>…</div>`
`<span>`	Texto genérico em linha	`<span>x</span>`
`<ul> <ol> <li>`	Listas	`<ul><li>Item</li></ul>`
`<a href>`	Hiperligação	`<a href="pag.html">Ir</a>`
`target`	Abrir noutro separador	`target="_blank"`
`title`	Texto de ajuda	`title="ajuda"`
`<img>`	Imagem	`<img src="imagens/f.jpg" alt="Foto">`
`width / height`	Tamanho da imagem	`width="300"`

PARTE A · CAPÍTULO 4

Tabelas

As tabelas apresentam **dados organizados em linhas e colunas**, como horários, notas ou preços.

Neste capítulo vais aprender a:

- construir uma tabela completa com cabeçalho, corpo e rodapé;
- usar `th` e `td` corretamente;
- juntar células com `colspan` e `rowspan`.

> ⚠️ **ATENÇÃO**
> As tabelas servem para **dados**, nunca para definir a disposição da página (isso faz-se com CSS). Nos exemplos usamos `border="1"` só para **veres as linhas**: é um atributo antigo que vamos substituir por CSS no capítulo 11. Sem ele, a tabela aparece sem linhas.

<table>

TABELAS

Função Cria uma **tabela**: contém todos os outros elementos de tabela.

Sintaxe `<table> … </table>`

Quando usar Para dados em linhas e colunas.

EXEMPLO

```
<table border="1">
    <tr>
        <td>Célula 1</td>
        <td>Célula 2</td>
    </tr>
</table>
```

RESULTADO ESPERADO

Célula 1	Célula 2

<tr>

TABELAS

Função Define uma **linha** (*table row*).

Sintaxe `<tr> … </tr>`

Quando usar Uma por cada linha. Só contém `<th>` ou `<td>`.

EXEMPLO

```
<table border="1">
    <tr>
        <td>Linha 1, A</td>
        <td>Linha 1, B</td>
    </tr>
    <tr>
        <td>Linha 2, A</td>
        <td>Linha 2, B</td>
    </tr>
</table>
```

RESULTADO ESPERADO

Linha 1, A	Linha 1, B
Linha 2, A	Linha 2, B

<th>

TABELAS

Função	Define uma **célula de cabeçalho** (*table header*): a negrito e centrada.
Sintaxe	`<th>Título</th>`
Quando usar	Nos cabeçalhos das colunas (ou das linhas).

EXEMPLO

```
<table border="1">
    <tr>
        <th>Nome</th>
        <th>Idade</th>
    </tr>
    <tr>
        <td>Ana</td>
        <td>16</td>
    </tr>
</table>
```

RESULTADO ESPERADO

Nome	Idade
Ana	16

✅ **BOA PRÁTICA**

Usa `<th>` em vez de negrito à mão: ajuda os leitores de ecrã.

<td>

TABELAS

Função	Define uma **célula de dados** (*table data*).
Sintaxe	`<td>Dados</td>`
Quando usar	Para cada valor da tabela.

EXEMPLO

```
<table border="1">
    <tr>
        <th>Aluno</th>
        <th>Nota</th>
    </tr>
    <tr>
        <td>Rui</td>
        <td>15</td>
    </tr>
</table>
```

RESULTADO ESPERADO

Aluno	Nota
Rui	15

❌ **ERRO FREQUENTE**

Cada linha deve ter o mesmo número de células. Se faltar uma, a tabela fica desalinhada.

<caption>

TABELAS

Função Dá um **título** à tabela.

Sintaxe `<caption>Título</caption>`

Quando usar Logo a seguir a `<table>`. Aparece acima da tabela.

EXEMPLO

```
<table border="1">
    <caption>Notas do 1.º período</caption>
    <tr>
        <th>Disciplina</th>
        <th>Nota</th>
    </tr>
    <tr>
        <td>Português</td>
        <td>15</td>
    </tr>
</table>
```

RESULTADO ESPERADO

Notas do 1.º período

Disciplina	Nota
Português	15

<thead>

TABELAS

Função Agrupa a(s) linha(s) do **cabeçalho**.

Sintaxe `<thead> <tr>…</tr> </thead>`

Quando usar Em tabelas maiores. Visualmente não altera nada; dá **estrutura** e permite estilos com CSS.

EXEMPLO

```
<table border="1">
    <thead>
        <tr>
            <th>Produto</th>
            <th>Preço</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Caderno</td>
            <td>2 €</td>
        </tr>
    </tbody>
</table>
```

RESULTADO ESPERADO

Produto	Preço
Caderno	2 €

<tbody>

TABELAS

Função Agrupa as linhas do **corpo** da tabela.

Sintaxe `<tbody> <tr>…</tr> </tbody>`

Quando usar Depois do `<thead>`: contém os dados principais.

EXEMPLO

```
<table border="1">
    <tbody>
        <tr><td>Caderno</td><td>2 €</td></tr>
        <tr><td>Caneta</td><td>1 €</td></tr>
    </tbody>
</table>
```

RESULTADO ESPERADO

Caderno	2 €
Caneta	1 €

<tfoot>

TABELAS

Função Agrupa as linhas de **rodapé** (totais, médias...).

Sintaxe `<tfoot> <tr>…</tr> </tfoot>`

Quando usar Para linhas de resumo.

EXEMPLO

```
<table border="1">
    <tbody>
        <tr><td>Caderno</td><td>2 €</td></tr>
        <tr><td>Caneta</td><td>1 €</td></tr>
    </tbody>
    <tfoot>
        <tr><th>Total</th><td>3 €</td></tr>
    </tfoot>
</table>
```

RESULTADO ESPERADO

Caderno	2 €
Caneta	1 €
Total	3 €

> 💡 **DICA**
> Ordem recomendada: `caption`, `thead`, `tbody`, `tfoot`.

colspan e rowspan

ATRIBUTOS DE /

Função Juntam células: `colspan` ocupa várias **colunas**; `rowspan` ocupa várias **linhas**.

Sintaxe `colspan="2" rowspan="2"`

Quando usar Só quando for realmente necessário (por exemplo, um título que abrange várias colunas).

EXEMPLO

```
<table border="1">
    <tr>
        <th>Dia</th>
        <th colspan="2">Sessões</th>
    </tr>
    <tr>
        <td rowspan="2">Terça</td>
        <td>15:30</td>
        <td>Sala 12</td>
    </tr>
    <tr>
        <td>17:00</td>
        <td>Sala 14</td>
    </tr>
</table>
```

RESULTADO ESPERADO

Dia	Sessões	
Terça	15:30	Sala 12
	17:00	Sala 14

> ❌ **ERRO FREQUENTE**
> Contar mal as células: com `colspan="2"` a linha fica com uma célula “a menos”.

Exemplo de uma tabela completa

Tabela completa EXEMPLO

CÓDIGO

```html
<table border="1">
    <caption>Notas do 1.º período</caption>
    <thead>
        <tr>
            <th>Disciplina</th>
            <th>Teste 1</th>
            <th>Teste 2</th>
            <th>Média</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Português</td>
            <td>14</td>
            <td>16</td>
            <td>15</td>
        </tr>
        <tr>
            <td>Matemática</td>
            <td>12</td>
            <td>15</td>
            <td>13,5</td>
        </tr>
        <tr>
            <td>Inglês</td>
            <td>17</td>
            <td>18</td>
            <td>17,5</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th colspan="3">Média geral</th>
            <td>15,3</td>
        </tr>
    </tfoot>
</table>
```

RESULTADO ESPERADO

Notas do 1.º período

Disciplina	Teste 1	Teste 2	Média
Português	14	16	15
Matemática	12	15	13,5
Inglês	17	18	17,5
Média geral			15,3

O `colspan="3"` faz a célula “Média geral” ocupar as três primeiras colunas.

🏗️ Projeto progressivo · Etapa 6 de 14 — TABELA DE HORÁRIOS

O QUE ACRESCENTAS

```html
<h2>Horário das sessões</h2>
<table border="1">
    <caption>Horário das sessões</caption>
    <thead>
        <tr>
            <th>Dia</th>
            <th>Hora</th>
            <th>Sala</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Terça-feira</td>
            <td>15:30</td>
            <td>Sala 12</td>
        </tr>
        <tr>
            <td>Quinta-feira</td>
            <td>16:00</td>
            <td>Laboratório de Imagem</td>
        </tr>
    </tbody>
</table>
```

RESULTADO ESPERADO

Clube de Fotografia da Escola

Início | Galeria | Contactos

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



*O `border="1"` é **temporário**: no capítulo de CSS vamos substituí-lo por estilos.*

🧪 EXPERIMENTA TU 8 — HORARIO.HTML

Cria `horario.html` com o teu horário semanal:

- um `caption` com o título;
- um `thead` com os dias da semana;
- um `tbody` com três linhas de aulas;
- uma célula com `colspan` (por exemplo, o almoço) ou `rowspan`;
- um `tfoot` com uma nota final.

🎯 QUAL SERÁ O RESULTADO? — D13 (5 MIN)

Quantas linhas e colunas tem a tabela? O que aparece na primeira linha?

```html
<table border="1">
    <tr><th>Nome</th><th>Idade</th></tr>
    <tr><td>Rui</td><td>17</td></tr>
</table>
```

🧩 COMPLETA O CÓDIGO — D14 (10 MIN)

Completa a tabela para que “Total” ocupe duas colunas.

```html
<table border="1">
    <tr><td>Caneta</td><td>1 €</td><td>3 €</td></tr>
    <tr>
        <th ___>Total</th>
        <___>9 €</td>
    </tr>
</table>
```

🔍 ENCONTRA O ERRO — D15 (5 MIN)

Esta tabela fica desalinhada. Porquê?

```
<table border="1">
    <tr><td>Maçã</td><td>1 €</td><td>Kg</td></tr>
    <tr><td>Pera</td><td>2 €</td></tr>
</table>
```

📋 Ficha rápida — Tabelas

Elemento	Função	Exemplo
`<table>`	Tabela	`<table>…</table>`
`<caption>`	Título	`<caption>Notas</caption>`
`<thead>`	Cabeçalho	`<thead>…</thead>`
`<tbody>`	Corpo	`<tbody>…</tbody>`
`<tfoot>`	Rodapé	`<tfoot>…</tfoot>`
`<tr>`	Linha	`<tr>…</tr>`
`<th>`	Célula de cabeçalho	`<th>Nome</th>`
`<td>`	Célula de dados	`<td>Ana</td>`
`colspan`	Junta colunas	`<td colspan="2">`
`rowspan`	Junta linhas	`<td rowspan="2">`

PARTE A · CAPÍTULO 5

Formulários

Os formulários permitem ao utilizador **introduzir informação**: contactos, inscrições, pesquisas... Nesta fase aprendes a construí-los.

Neste capítulo vais aprender a:

- criar formulários com `form`, `label` e `input`;
- escolher o tipo de campo certo;
- usar `name`, `id`, `value`, `placeholder` e `required`;
- construir um formulário de contacto completo.

<form>

FORMULÁRIOS

Função	Cria um **formulário**: os campos ficam **dentro** dele.
Sintaxe	`<form action="destino" method="post"> … </form>`
Quando usar	Sempre que queiras recolher dados.

EXEMPLO

```
<form action="#" method="post">
    <label for="nome">Nome:</label>
    <input type="text" id="nome" name="nome">
    <button type="submit">Enviar</button>
</form>
```

RESULTADO ESPERADO

Nome: Enviar

> 💡 **DICA**
> `action` é o destino dos dados e `method` a forma de os enviar (`get` ou `post`). Por agora, usa `action="#"`: só estamos a construir o aspeto do formulário.

<label>

FORMULÁRIOS

Função	Cria a **etiqueta** de um campo. Liga-se ao campo com `for` = `id`.
Sintaxe	`<label for="id-do-campo">Texto</label>`
Quando usar	Em **todos** os campos.

EXEMPLO

```
<label for="email">O teu email:</label>
<input type="email" id="email" name="email">
```

RESULTADO ESPERADO

O teu email:

> ✅ **BOA PRÁTICA**
> Ao clicar no texto da `<label>`, o cursor vai para o campo. Nas caixas de seleção aumenta a área clicável!

> ❌ **ERRO FREQUENTE**
> O valor de `for` não coincidir com o `id` do campo.

<input>

FORMULÁRIOS

Função	Cria um **campo de entrada de dados**. O aspeto depende do atributo `type`. Elemento vazio.
Sintaxe	`<input type="tipo" name="nome" id="id">`
Quando usar	Para respostas curtas e escolhas (tipos na tabela abaixo).

EXEMPLO

```
<label for="curso">Curso:</label>
<input type="text" id="curso" name="curso"
       placeholder="Ex.: Informática" required>
```

RESULTADO ESPERADO

Curso: Ex.: Informática

> 💡 **DICA**
> Sem `type`, o navegador assume `type="text"`.

<textarea>

FORMULÁRIOS

Função	Cria uma **caixa de texto com várias linhas**.
Sintaxe	`<textarea name="nome" rows="4" cols="30"></textarea>`
Quando usar	Quando a resposta pode ser longa.

EXEMPLO

```
<label for="msg">Mensagem:</label><br>
<textarea id="msg" name="msg" rows="4" cols="30"
placeholder="Escreve aqui..."></textarea>
```

RESULTADO ESPERADO

Mensagem:
Escreve aqui...

> 💡 **DICA**
> Tem tag de fecho. `rows` e `cols` definem o tamanho visível.

<select>

FORMULÁRIOS

Função	Cria uma **lista pendente** onde se escolhe uma opção.
Sintaxe	`<select name="nome"> <option>…</option> </select>`
Quando usar	Quando há uma lista fixa de opções.

EXEMPLO

```
<label for="curso2">Curso:</label>
<select id="curso2" name="curso2">
    <option value="">-- Escolhe --</option>
    <option value="info">Informática</option>
    <option value="mult">Multimédia</option>
    <option value="elet"
selected>Eletrónica</option>
</select>
```

RESULTADO ESPERADO

Curso: Eletrónica

> 💡 **DICA**
> `selected` deixa uma opção pré-selecionada.

<option>

FORMULÁRIOS

Função	Define cada **opção** de um `<select>`.
Sintaxe	`<option value="valor">Texto visível</option>`
Quando usar	Sempre dentro de um `<select>`.

EXEMPLO

```
<select name="ano">
    <option value="10">10.º ano</option>
    <option value="11">11.º ano</option>
    <option value="12">12.º ano</option>
</select>
```

RESULTADO ESPERADO



> 💡 **DICA**
> O `value` é o que é enviado; o texto é o que o utilizador vê.

<button>

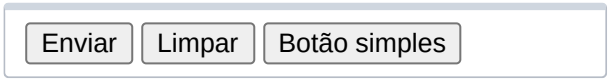
FORMULÁRIOS

Função	Cria um **botão** clicável.
Sintaxe	`<button type="submit">Texto</button>`
Quando usar	Para enviar (`submit`), limpar (`reset`) ou para ações com JavaScript (`button`).

EXEMPLO

```
<button type="submit">Enviar</button>
<button type="reset">Limpar</button>
<button type="button">Botão simples</button>
```

RESULTADO ESPERADO



Tipos de input mais utilizados

Tipo e função	Código	Resultado
`type="text"` Texto curto.	`<input type="text" name="nome" placeholder="O teu nome">`	O teu nome
`type="password"` Palavra-passe (escondida).	`<input type="password" name="pass" value="segredo">`	•••••••
`type="email"` Email (formato validado).	`<input type="email" name="email" placeholder="nome@exemplo.pt">`	nome@exemplo.pt
`type="number"` Números (`min`, `max`).	`<input type="number" name="idade" min="10" max="99" value="16">`	16
`type="date"` Data, com calendário.	`<input type="date" name="nascimento">`	dd/mm/aaaa
`type="radio"` **Uma só** opção (mesmo `name`).	`<input type="radio" id="t1" name="turno" value="manha"> <label for="t1">Manhã</label> <input type="radio" id="t2" name="turno" value="tarde" checked> <label for="t2">Tarde</label>`	○ Manhã ◉ Tarde
`type="checkbox"` **Várias** opções.	`<input type="checkbox" id="c1" name="hobby" value="ler" checked> <label for="c1">Ler</label> <input type="checkbox" id="c2" name="hobby" value="jogar"> <label for="c2">Jogar</label>`	☑ Ler ☐ Jogar
`type="file"` Escolher um ficheiro.	`<input type="file" name="foto">`	Escolher ficheiro Nenhum ficheiro selecionado
`type="submit"` Botão de envio.	`<input type="submit" value="Enviar">`	Enviar
`type="reset"` Botão que limpa o formulário.	`<input type="reset" value="Limpar">`	Limpar

> ❌ **ERRO FREQUENTE**
> Dar **nomes diferentes** aos botões de opção (`radio`) do mesmo grupo: deixam de funcionar em grupo e o utilizador consegue escolher todos. Os botões do mesmo grupo têm o **mesmo** `name`.

Atributos mais utilizados

Atributo	Função	Exemplo
`name`	Nome do campo: identifica os dados **quando o formulário é enviado**.	`name="email"`
`id`	Identificador **único** na página; liga a `label` ao campo.	`id="email"`
`value`	Valor inicial do campo, ou valor enviado por radio/checkbox.	`value="Portugal"`
`placeholder`	Texto de **ajuda** a cinzento no campo vazio.	`placeholder="Ex.: Ana"`
`required`	Campo **obrigatório** (atributo sem valor).	`<input type="text" required>`
`checked`	Deixa um radio/checkbox **já marcado**.	`<input type="checkbox" checked>`
`min / max`	Valor mínimo e máximo de um campo numérico ou de data.	`<input type="number" min="1" max="12">`

> 💡 **DICA**
> `name` e `id` costumam ter o mesmo valor, mas têm funções diferentes: o `name` serve para **enviar** os dados; o `id` serve para **identificar** o elemento na página.

> ❌ **ERRO FREQUENTE**
> Usar o **mesmo** `id` em vários elementos da página. O `id` tem de ser **único**! Para vários elementos com o mesmo estilo usa `class` (capítulo 10).

Formulário de contacto

Formulário de contacto EXEMPLO

CÓDIGO

```
<form action="#" method="post">
    <label for="nome">Nome</label>
    <input type="text" id="nome" name="nome"
placeholder="O teu nome" required>

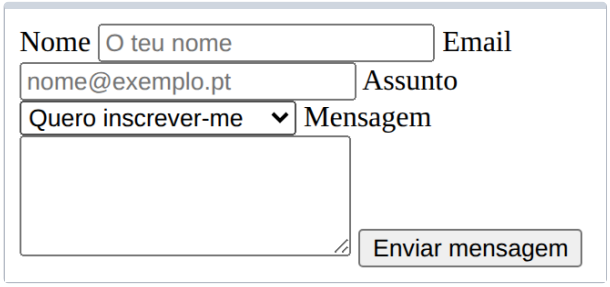
    <label for="email">Email</label>
    <input type="email" id="email" name="email"
placeholder="nome@exemplo.pt" required>

    <label for="assunto">Assunto</label>
    <select id="assunto" name="assunto">
        <option value="inscricao">Quero inscrever-
me</option>
        <option value="info">Pedido de
informação</option>
        <option value="outro">Outro
assunto</option>
    </select>

    <label for="mensagem">Mensagem</label>
    <textarea id="mensagem" name="mensagem"
rows="4"></textarea>

    <button type="submit">Enviar mensagem</button>
</form>
```

RESULTADO ESPERADO



Ainda sem CSS: o aspeto vai melhorar no capítulo 11. Repara que cada `label` está ligada ao seu campo.

🏗️ Projeto progressivo · Etapa 7 de 14

PÁGINA DE CONTACTOS

O QUE ACRESCENTAS

```
<form action="#" method="post">
    <label for="nome">Nome</label>
    <input type="text" id="nome" name="nome"
placeholder="O teu nome" required>

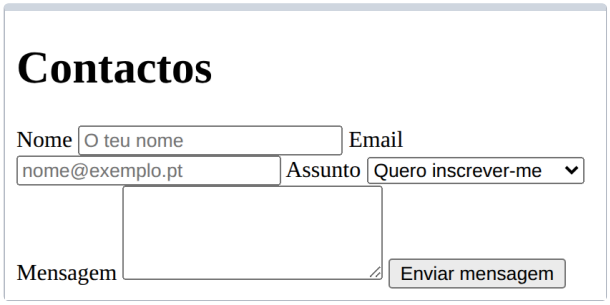
    <label for="email">Email</label>
    <input type="email" id="email" name="email"
placeholder="nome@exemplo.pt" required>

    <label for="assunto">Assunto</label>
    <select id="assunto" name="assunto">
        <option value="inscricao">Quero inscrever-
me</option>
        <option value="info">Pedido de
informação</option>
        <option value="outro">Outro
assunto</option>
    </select>

    <label for="mensagem">Mensagem</label>
    <textarea id="mensagem" name="mensagem"
rows="4"></textarea>

    <button type="submit">Enviar mensagem</button>
</form>
```

RESULTADO ESPERADO (CONTACTOS.HTML)



Nova página do projeto: `contactos.html` *(só o corpo é mostrado).*

🧪 EXPERIMENTA TU 9 — INSCRICAO.HTML

Cria `inscricao.html`, uma inscrição num clube da escola, com:

- nome (obrigatório), email (obrigatório) e data de nascimento;
- um escalão em `radio` e vários interesses em `checkbox`;
- uma lista pendente `select` com o ano de escolaridade;
- uma `textarea` (“Porque queres participar?”) e um campo para carregar uma fotografia;
- botões **Enviar** e **Limpar**, com todas as `label` ligadas por `for` / `id`.

🧩 COMPLETA O CÓDIGO — D16 (5 MIN)

Completa o campo de email para ser **obrigatório** e mostrar a ajuda “nome@exemplo.pt”.

```
<label for="email">Email</label>
<input type="___" id="email" name="email"
       ___="nome@exemplo.pt" ___>
```

🔍 ENCONTRA O ERRO — D17 (5 MIN)

Clicar no texto “Nome” não coloca o cursor no campo. Porquê?

```
<label for="nome">Nome</label>
<input type="text" id="name">
```

🎯 QUAL SERÁ O RESULTADO? — D18 (5 MIN)

Quantas opções se podem escolher em cada grupo?

```
<input type="radio" name="turno" value="m"> Manhã
<input type="radio" name="turno" value="t"> Tarde

<input type="checkbox" name="hobby" value="a"> Ler
<input type="checkbox" name="hobby" value="b"> Jogar
```

📋 Ficha rápida — Formulários

Elemento	Função	Exemplo
`<form>`	Formulário	`<form action="#">…</form>`
`<label>`	Legenda de um campo	`<label for="n">Nome</label>`
`<input>`	Campo de entrada	`<input type="text" id="n" name="n">`
`<textarea>`	Texto com várias linhas	`<textarea name="m"></textarea>`
`<select>`	Lista pendente	`<select>…</select>`
`<option>`	Opção da lista	`<option value="a">A</option>`
`<button>`	Botão	`<button type="submit">Enviar</button>`
`type`	Tipo do campo	`type="email"`
`name`	Nome dos dados	`name="email"`
`id`	Identificador único	`id="email"`
`value`	Valor	`value="Portugal"`
`placeholder`	Texto de ajuda	`placeholder="Ex.: Ana"`
`required`	Obrigatório	`required`
`checked`	Já marcado	`checked`
`min / max`	Limites	`min="1" max="10"`

PARTE A · CAPÍTULO 6

HTML semântico

Elementos “com significado”: em vez de usar `<div>` para tudo, o HTML5 tem tags que dizem **o que cada zona da página é**.

Neste capítulo vais aprender a:

- explicar porque é importante o HTML semântico;
- escolher entre `header`, `nav`, `main`, `section`, `article`, `aside` e `footer`;
- estruturar uma página completa.

6.1 Porquê usar elementos semânticos?

- **Legibilidade:** ao ler `<nav>` percebes que é o menu; uma `<div>` não diz nada.
- **Acessibilidade:** os leitores de ecrã permitem saltar diretamente para o menu ou para o conteúdo principal.
- **Motores de pesquisa:** percebem melhor a importância de cada parte da página.
- **Manutenção:** o código fica organizado e mais fácil de alterar.

Só com div ou **com elementos semânticos**? Ambas as páginas aparecem **exatamente** iguais no navegador; a diferença está no **significado**:

❌ **SÓ COM DIV**

```
<div id="topo">
    <h1>Clube de Xadrez</h1>
</div>
<div id="menu">
    <a href="#regras">Regras</a>
</div>
<div id="conteudo">
    <div id="regras">
        <h2>Regras</h2>
        <p>Cada jogador move uma peça.</p>
    </div>
</div>
<div id="rodape">
    <p>Escola Secundária</p>
</div>
```

✅ **COM ELEMENTOS SEMÂNTICOS**

```
<header>
    <h1>Clube de Xadrez</h1>
</header>
<nav>
    <a href="#regras">Regras</a>
</nav>
<main>
    <section id="regras">
        <h2>Regras</h2>
        <p>Cada jogador move uma peça.</p>
    </section>
</main>
<footer>
    <p>Escola Secundária</p>
</footer>
```

6.2 A estrutura típica de uma página

💻 **CÓDIGO — ESTRUTURA SEMÂNTICA**

```
<header>
    ...
</header>

<nav>
    ...
</nav>

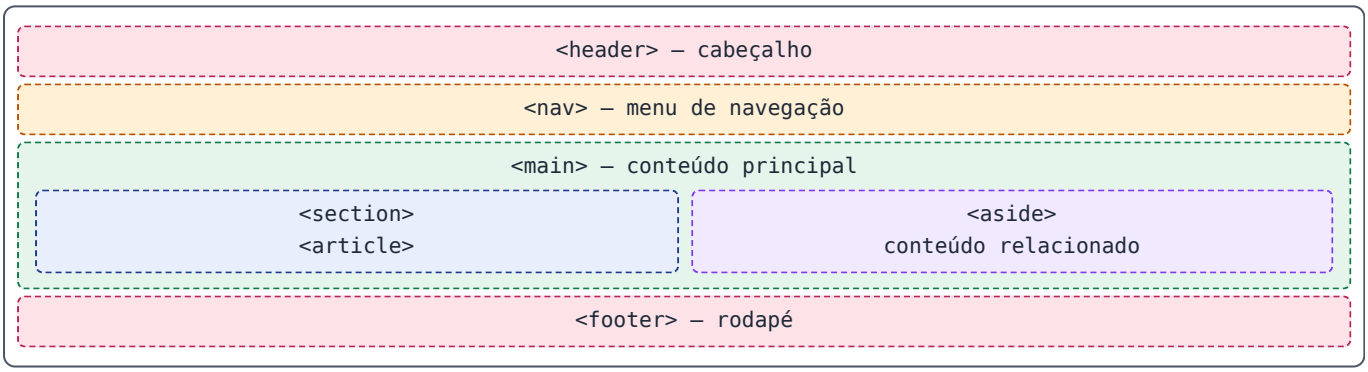
<main>

    <section>
        ...
    </section>

    <article>
        ...
    </article>

</main>

<footer>
    ...
</footer>
```



<header> SEMÂNTICOS

Função	Zona de **cabeçalho** da página (ou de uma secção): título, logótipo, apresentação.
Sintaxe	`<header> … </header>`
Quando usar	No topo da página.

EXEMPLO

```
<header>
    <h1>Clube de Ciências</h1>
    <p>Aprender a experimentar</p>
</header>
```

RESULTADO ESPERADO

Clube de Ciências

Aprender a experimentar

*Visualmente não altera o aspeto: o valor está no **significado**.*

💡 **DICA**

Não confundas `<header>` (visível) com `<head>` (metadados, invisível).

<nav> SEMÂNTICOS

Função	Agrupa as ligações principais de **navegação** (o menu).
Sintaxe	`<nav> … </nav>`
Quando usar	Para o menu principal.

EXEMPLO

```
<nav>
    <ul>
        <li><a href="index.html">Início</a></li>
        <li><a href="galeria.html">Galeria</a></li>
        <li><a href="contactos.html">Contactos</a>
</li>
    </ul>
</nav>
```

RESULTADO ESPERADO

- Início
- Galeria
- Contactos

> ✅ **BOA PRÁTICA**
> Constrói o menu com uma **lista** dentro do `<nav>`.

<main> SEMÂNTICOS

Função	Contém o **conteúdo principal e único** da página.
Sintaxe	`<main> … </main>`
Quando usar	Uma única vez por página, fora de `header`, `nav` e `footer`.

EXEMPLO

```
<main>
    <h2>Bem-vindos</h2>
    <p>Aqui fica o conteúdo principal desta página.
</p>
</main>
```

RESULTADO ESPERADO

Bem-vindos

Aqui fica o conteúdo principal desta página.

<section> SEMÂNTICOS

Função	Agrupa conteúdo **do mesmo tema**, normalmente com um título.
Sintaxe	`<section> <h2>…</h2> … </section>`
Quando usar	Para dividir a página em partes temáticas.

EXEMPLO

```
<section>
    <h2>Atividades</h2>
    <p>Todas as quartas há experiências no
laboratório.</p>
</section>
```

RESULTADO ESPERADO

Atividades

Todas as quartas há experiências no laboratório.

> 💡 **DICA**
> Se a parte não tem título ou é só para efeitos de estilo, usa `<div>`.

<article>

SEMÂNTICOS

Função	Conteúdo **independente** que faria sentido sozinho: uma notícia, uma publicação, um cartão.
Sintaxe	`<article> … </article>`
Quando usar	Para conteúdos que podem ser reutilizados noutro sítio.

EXEMPLO

```
<article>
    <h2>Feira de Ciência</h2>
    <p>A feira realiza-se no próximo mês.</p>
</article>
```

RESULTADO ESPERADO

<article>
<h2>Feira de Ciência</h2>
A feira realiza-se no próximo mês.
</article>

<aside>

SEMÂNTICOS

Função	Conteúdo **relacionado**, mas à parte do principal: curiosidades, dicas, publicidade.
Sintaxe	`<aside> … </aside>`
Quando usar	Para conteúdo lateral ou complementar.

EXEMPLO

```
<aside>
    <h3>Sabias que…?</h3>
    <p>A primeira fotografia foi tirada em 1826.
</p>
</aside>
```

RESULTADO ESPERADO

<aside>
<h3>Sabias que…?</h3>
A primeira fotografia foi tirada em 1826.
</aside>

<footer>

SEMÂNTICOS

Função	Zona de **rodapé** da página (ou de uma secção): autoria, direitos, contactos.
Sintaxe	`<footer> … </footer>`
Quando usar	No fim da página.

EXEMPLO

```
<footer>
    <p>&copy; 2026 Clube de Ciências</p>
</footer>
```

RESULTADO ESPERADO

<footer>
© 2026 Clube de Ciências
</footer>

> 💡 **DICA**
> O símbolo © escreve-se `©`.

🏗️ Projeto progressivo · Etapa 8 de 14

ESTRUTURA SEMÂNTICA

O QUE ACRESCENTAS

```
<header>
    <h1>Clube de Fotografia da Escola</h1>
    <p>Captar o mundo, uma fotografia de cada vez.
</p>
    <nav>
        <ul>
            <li><a href="index.html">Início</a>
</li>
            <li><a href="galeria.html">Galeria</a>
</li>
            <li><a
href="contactos.html">Contactos</a></li>
        </ul>
    </nav>
</header>

<main>
    <section id="quem-somos"> … </section>
    <section id="atividades"> … </section>
    <section id="participar"> … </section>
    <section id="horario"> … </section>
</main>

<footer> … </footer>
```

RESULTADO ESPERADO

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

- Início
- Galeria
- Contactos

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



*Visualmente a página **não muda**, mas agora o código tem estrutura e significado. O aspeto é o próximo passo (CSS).*

🧪 EXPERIMENTA TU 10 — DE DIV PARA SEMÂNTICO

Copia o código de “Só com div” (acima) para `semantico.html` e substitui as `div` pelos elementos semânticos certos. Confirma que a página continua a funcionar.

🧩 COMPLETA O CÓDIGO — D19 (10 MIN)

Completa com o elemento semântico adequado (`header` , `nav` , `main` , `footer`).

```
<___>
    <h1>O meu site</h1>
</___>
<___>
    <a href="index.html">Início</a>
</___>
<___>
    <p>Conteúdo principal.</p>
</___>
<___>
    <p>© 2026</p>
</___>
```

🎯 QUAL SERÁ O RESULTADO? — D20 (5 MIN)

Escolhe o elemento semântico mais adequado para cada situação: (a) menu principal; (b) uma notícia independente; (c) uma nota lateral com curiosidades; (d) direitos de autor.

📋 Ficha rápida — HTML semântico

Elemento	Função	Exemplo
`<header>`	Cabeçalho	`<header>…</header>`
`<nav>`	Menu de navegação	`<nav>…</nav>`
`<main>`	Conteúdo principal	`<main>…</main>`
`<section>`	Secção temática	`<section>…</section>`
`<article>`	Conteúdo independente	`<article>…</article>`
`<aside>`	Conteúdo relacionado	`<aside>…</aside>`
`<footer>`	Rodapé	`<footer>…</footer>`

PARTE A · CAPÍTULO 7

Multimédia

O HTML5 reproduz **áudio** e **vídeo** sem programas adicionais.

Neste capítulo vais aprender a:

- inserir áudio e vídeo com controlos;
- oferecer formatos alternativos com `<source>`.

<audio>

MULTIMÉDIA

Função	Insere um **leitor de áudio**.
Sintaxe	`<audio controls> … </audio>`
Quando usar	Para música, podcasts ou sons. Guarda os ficheiros numa pasta, por exemplo `audio`.

EXEMPLO

```
<audio controls>
    <source src="audio/musica.mp3"
type="audio/mpeg">
    O teu navegador não suporta áudio.
</audio>
```

RESULTADO ESPERADO



Aspeto aproximado do leitor de áudio (varia consoante o navegador).

> 💡 **DICA**
> `controls` mostra os botões; `loop` repete; `muted` silencia. Os navegadores **bloqueiam o `autoplay`** com som.

<video>

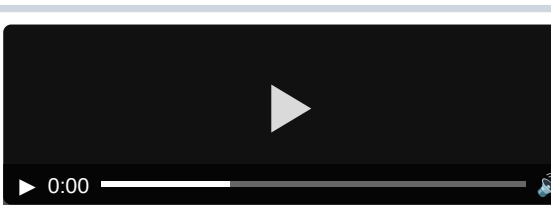
MULTIMÉDIA

Função	Insere um **leitor de vídeo**.
Sintaxe	`<video controls width="…"> … </video>`
Quando usar	Para vídeos guardados no teu projeto (para vídeos do YouTube usa o `<iframe>`).

EXEMPLO

```
<video controls width="320"
poster="imagens/capa.jpg">
    <source src="video/filme.mp4" type="video/mp4">
    O teu navegador não suporta vídeo.
</video>
```

RESULTADO ESPERADO



Aspeto aproximado do leitor de vídeo.

> 💡 **DICA**
> `poster` define a imagem mostrada antes de começar. O formato mais compatível é o **MP4 (H.264)**.

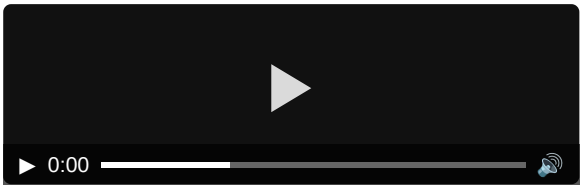
<source> MULTIMÉDIA

Função	Indica **um ficheiro** de áudio ou vídeo e o seu formato; permite oferecer **alternativas**.
Sintaxe	`<source src="ficheiro" type="tipo/formato">`
Quando usar	Dentro de `<audio>` ou `<video>`. O navegador usa a **primeira** fonte que consegue reproduzir.

EXEMPLO

```
<video controls width="320">
    <source src="video/filme.webm"
type="video/webm">
    <source src="video/filme.mp4" type="video/mp4">
    O teu navegador não suporta vídeo.
</video>
```

RESULTADO ESPERADO



O resultado visível é o mesmo leitor.

> 💡 **DICA**
> O texto escrito dentro de `<video>` só aparece se o navegador não suportar o elemento.

🧪 EXPERIMENTA TU 11 — MULTIMEDIA.HTML

Cria `multimedia.html` com um `video` (uma fonte MP4, com `controls` e `width`), um `audio` com `controls` e um título `h2` por cima de cada um.

🔍 ENCONTRA O ERRO — D21 (5 MIN)

O vídeo não mostra botões de reprodução. O que falta?

```
<video width="320">
    <source src="video/filme.mp4" type="video/mp4">
</video>
```

📋 Ficha rápida — Multimédia

Elemento	Função	Exemplo
`<audio>`	Leitor de áudio	`<audio controls>…</audio>`
`<video>`	Leitor de vídeo	`<video controls width="320">…</video>`
`<source>`	Fonte do ficheiro	`<source src="f.mp4" type="video/mp4">`
`controls`	Mostra os botões	`controls`

PARTE A · CAPÍTULO 8

iframe: conteúdo incorporado

Um `<iframe>` é uma **janela dentro da página** que mostra outra página. É muito usado para incorporar vídeos e mapas.

Neste capítulo vais aprender a:

- incorporar uma página ou um conteúdo externo;
- conhecer as limitações e os cuidados de segurança.

<iframe>

INCORPORAÇÃO

Função	Mostra **outra página** dentro da tua (*inline frame*).
Sintaxe	`<iframe src="pagina.html"></iframe>`
Quando usar	Para mostrar mapas, vídeos do YouTube ou outra página do teu site.

EXEMPLO

```
<iframe src="pagina.html"
        width="300" height="110"
        title="Página incorporada">
</iframe>
```

RESULTADO ESPERADO

Página incorporada

Este conteúdo vem de outro ficheiro HTML.

A moldura mostra o conteúdo de `pagina.html` dentro da tua página.

✅ **BOA PRÁTICA**

Usa sempre o atributo `title` (acessibilidade) e define `width` e `height`.

Incorporar conteúdo externo

Muitos serviços (YouTube, Google Maps, OpenStreetMap…) oferecem a opção **Partilhar › Incorporar**, que fornece o código `<iframe>` já pronto para copiares:

```
<iframe width="320" height="180"
        src="https://www.youtube.com/embed/ID_DO_VIDEO"
        title="Vídeo do YouTube" allowfullscreen>
</iframe>
```

Substitui `ID_DO_VIDEO` pelo código do vídeo que queres incorporar.

Segurança e limitações

- **Só incorpores conteúdo de origens em que confias:** uma página externa dentro da tua pode ser insegura.
- **Alguns websites impedem** ser apresentados dentro de um `iframe` (por razões de segurança). Nesse caso, a moldura aparece vazia ou com uma mensagem de erro.
- Uma página incorporada **pode ser lenta** a carregar e alterar-se sem que tu controles.
- Existe o atributo `sandbox` para limitar o que a página incorporada pode fazer (uso avançado).



O que aparece quando um website bloqueia a incorporação (mensagem aproximada).

❌ ERRO FREQUENTE

Tentar incorporar um website qualquer com `<iframe src="https://www.site.pt">` e ficar com uma moldura em branco: muitos sites não o permitem. Testa primeiro!

🧪 EXPERIMENTA TU 12 — IFRAME

Cria `incorporar.html` com um `iframe` que mostre outra página tua (por exemplo `sobre.html`) com `width`, `height` e `title`.

🛠️ CORRIGE O CÓDIGO — D22 (5 MIN)

O `iframe` não tem tamanho nem descrição. Melhora-o para 400×250 píxeis e com o título “Mapa da escola”.

```
<iframe src="mapa.html"></iframe>
```

📋 Ficha rápida — iframe

Elemento	Função	Exemplo
`<iframe>`	Página dentro da página	`<iframe src="p.html" title="…"></iframe>`
`src`	Endereço da página	`src="pagina.html"`
`width / height`	Tamanho da moldura	`width="400" height="250"`
`title`	Descrição (acessibilidade)	`title="Mapa"`

agrupamento de escolas de rio tinto n.º3

aert3

Linguagem CSS

Sebenta de Estudo

```
seletores
{
    propriedade:
    valor;
}
```

cores
tipografia
disposição
layouts

Cores

```
color: #2563eb;
background: #f8fafc;
border: 1px solid #e5e7eb;
```

Tipografia

Aa

```
font-family:
'Inter', sans-serif;
font-size: 1.25rem;
font-weight: 600;
line-height: 1.6;
```

Disposição

```
display: flex;
justify-content:
 center;
align-items: center;
gap: 1rem;
```




```
/* Estilos principais */
body {
  font-family: 'Inter', sans-serif;
  background: #f8fafc;
  color: #1f2937;
  margin: 0;
}
.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem;
}
.btn {
  background: #2563eb;
  color: #ffffff;
  padding: 0.75rem 1.5rem;
  border-radius: 8px;
  transition: background 0.3s;
}
```

FOLHAS DE ESTILO CSS

SELETORES E ESPECIFICIDADE

CORES E TIPOGRAFIA

LAYOUTS RESPONSIVOS

ANIMAÇÕES E TRANSIÇÕES

PROJETOS WEB

</>

- header
- container
- card
- btn
- footer

PARTE B · CAPÍTULO 9

Introdução ao CSS

O CSS (*Cascading Style Sheets*) dá **aspeto** às páginas: cores, tipos de letra, espaços e disposição dos elementos.

Neste capítulo vais aprender a:

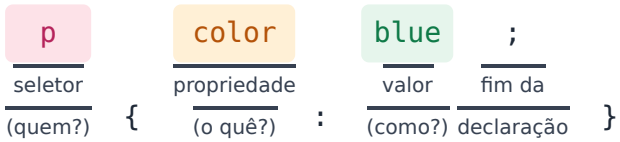
- distinguir conteúdo (HTML) de apresentação (CSS);
- escrever uma regra CSS: seletor, propriedade e valor;
- ligar um ficheiro CSS externo a uma página;
- aplicar o primeiro estilo ao projeto.

9.1 Conteúdo e apresentação

	HTML	CSS
Responde a	O **quê**? (o que é cada coisa)	**Como** se vê? (que aspeto tem)
Exemplo	«Isto é um título»	«Os títulos são azuis e grandes»
Se faltar	A página fica sem conteúdo	A página continua a funcionar, mas fica sem estilo

✅ **BOA PRÁTICA**
Regra de ouro: o **HTML** diz o que cada coisa **é**; o **CSS** diz como cada coisa **se vê**. Nunca uses tags HTML só para «mudar o aspeto».

9.2 Anatomia de uma regra CSS



Lê-se assim: «**todos os parágrafos** (seletor) **têm a cor** (propriedade) **azul** (valor)». Uma regra pode ter **várias declarações**, cada uma terminada em `;`:

Uma regra com três declarações — SINTAXE

CÓDIGO

HTML

```html
<p>Olá, CSS!</p>
```

CSS

```css
p {
    color: white;
    background-color: #0b4f8a;
    font-size: 20px;
}
```

RESULTADO ESPERADO

Olá, CSS!

❌ **ERRO FREQUENTE**
Esquecer o `;` entre declarações, ou escrever declarações **fora** das chavetas `{ }`: o navegador ignora o que está errado, sem avisar.

9.3 Três maneiras de usar CSS

Forma	Onde se escreve	Quando usar
Inline	No atributo `style` do elemento	Quase nunca (mistura estrutura e aspeto)
Interno	Dentro de `<style>`, no `<head>`	Testes rápidos ou páginas isoladas
Externo ✅	Num ficheiro `.css` ligado com `<link>`	**Sempre que possível** (recomendado)

`style=""` — CSS INLINE

Função Aplica estilo **a um único elemento**, escrito na própria tag.

Sintaxe `style="propriedade: valor;"`

Quando usar Só em casos pontuais; não reutilizável.

EXEMPLO

```
<p style="color: red; font-size: 22px;">Texto
vermelho.</p>
<p>Texto normal.</p>
```

RESULTADO ESPERADO

Texto vermelho.

Texto normal.

> ❌ **ERRO FREQUENTE**
> Com muitos `style` espalhados, alterar o aspeto do site obriga a mexer em **todas** as páginas.

`<style>` — CSS INTERNO

Função Guarda as regras CSS **dentro da própria página**.

Sintaxe `<style> regras </style>`

Quando usar Quando a página é única ou para experimentar.

EXEMPLO

```
<head>
    <style>
        p { color: green; }
    </style>
</head>
<body>
    <p>Texto verde.</p>
    <p>Outro parágrafo verde.</p>
</body>
```

RESULTADO ESPERADO

Texto verde.

Outro parágrafo verde.

`<link rel="stylesheet">` — CSS EXTERNO

Função	Liga a página a um **ficheiro CSS separado**: um único ficheiro serve todas as páginas.
Sintaxe	`<link rel="stylesheet" href="css/estilo.css">`
Quando usar	Sempre, em projetos com mais de uma página.

EXEMPLO

index.html

```html
<head>
    <link rel="stylesheet" href="css/estilo.css">
</head>
<body>
    <p>Texto azul.</p>
</body>
```

css/estilo.css

```css
p { color: #0b4f8a; }
```

RESULTADO ESPERADO

Texto azul.

✅ **BOA PRÁTICA**

Usa `<link>` dentro do `<head>` e um caminho **relativo** (como nas imagens).

❌ **ERRO FREQUENTE**

Caminho errado em `href` (ex.: `estilo.css` em vez de `css/estilo.css`): o estilo não aparece, sem qualquer erro.

💡 **DICA**

Comentários em CSS escrevem-se assim: `/* isto é um comentário */`. Servem para organizar o ficheiro em secções.

🏗️ Projeto progressivo · Etapa 9 de 14

CSS BASE DO SITE

O QUE ACRESCENTAS

index.html — dentro do <head>

```
<head>
    ...
    <link rel="stylesheet" href="css/estilo.css">
</head>
```

css/estilo.css

```
/* ===== Base ===== */
* { box-sizing: border-box; }

body {
    font-family: Arial, Helvetica, sans-serif;
    line-height: 1.6;
    color: #222;
    margin: 0;
    background: #f4f6f9;
}

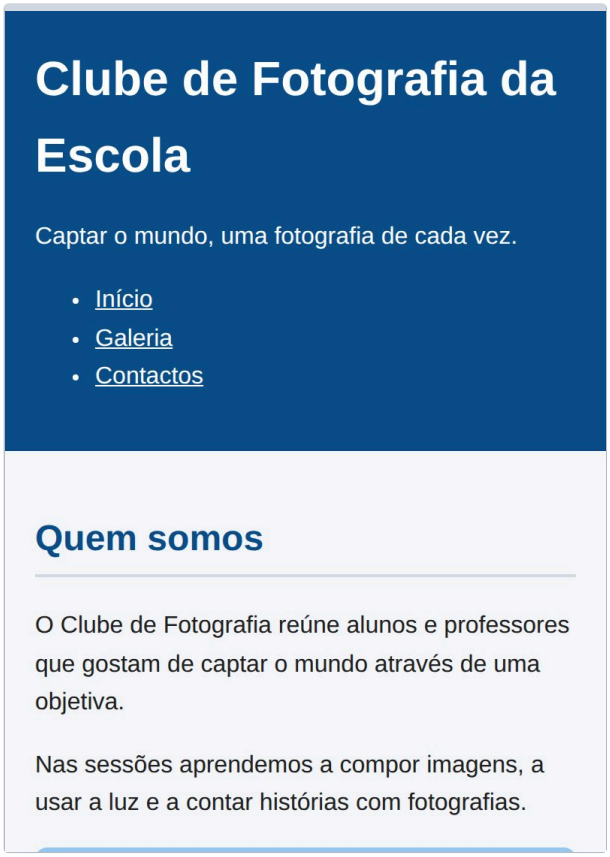
header {
    background: #0b4f8a;
    color: #fff;
    padding: 20px;
}

header a { color: #fff; }

header h1 { margin: 0 0 8px; }

/* … continua (ver Anexo B) … */
```

RESULTADO ESPERADO



*Cria a pasta `css` e o ficheiro `estilo.css`; liga-o em **todas** as páginas. O HTML não mudou: só o aspeto.*

🧪 EXPERIMENTA TU 13 — MUDA A COR

No teu `estilo.css`, altera a cor do `header` para outra cor (por exemplo `#1d4ed8` ou `darkgreen`). Grava, atualiza o navegador e observa.

🧩 COMPLETA O CÓDIGO — D23 (5 MIN)

Completa a regra para que os títulos `h2` fiquem vermelhos.

```
___ {
    ___: red;
___
```

🔍 ENCONTRA O ERRO — D24 (5 MIN)

O texto não fica verde. Quantos erros existem?

```
p {
    color: green
}
font-size: 20px;
```

🛠️ CORRIGE O CÓDIGO — D25 (5 MIN)

A página está em `meu_site/index.html` e o CSS em `meu_site/css/estilo.css`, mas não há estilo. Corrige.

```
<link rel="stylesheet" href="estilo.css">
```

📋 Ficha rápida — CSS: introdução

Elemento	Função	Exemplo
`seletor { }`	Regra CSS	`p { color: red; }`
`propriedade: valor;`	Declaração	`color: blue;`
`style=""`	CSS inline	`<p style="color:red">`
`<style>`	CSS interno	`<style>p{}</style>`
`<link>`	CSS externo	`<link rel="stylesheet" href="…">`
`/* */`	Comentário	`/* nota */`

PARTE B · CAPÍTULO 10

Seletores CSS

Um **seletor** indica **a que elementos** se aplica o estilo. Escolher bem o seletor é metade do trabalho em CSS.

Neste capítulo vais aprender a:

- selecionar por elemento, classe e id;
- agrupar seletores e escolher descendentes;
- distinguir classe de id;
- usar `:hover` para reagir ao rato.

Seletor	Escreve-se	Seleciona	Exemplo
Elemento	`p`	todos os elementos dessa tag	`p { … }`
Classe	`.nome`	elementos com `class="nome"`	`.destaque { … }`
Id	`#nome`	o elemento com `id="nome"`	`#topo { … }`
Múltiplo	`a, b`	vários seletores ao mesmo tempo	`h1, h2 { … }`
Descendente	`a b`	**b** dentro de **a**	`nav a { … }`

`p { }` — SELETOR DE ELEMENTO

Função Estiliza **todos** os elementos de um tipo.

Sintaxe `elemento { propriedade: valor; }`

Quando usar Estilo geral para todas as tags iguais.

EXEMPLO

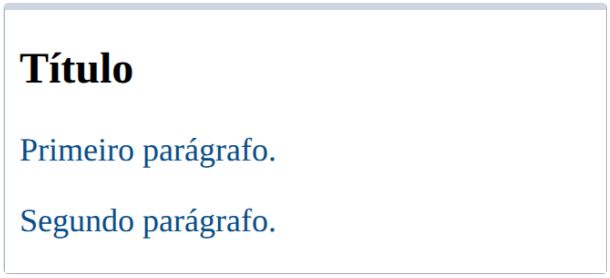
HTML

```html
<h2>Título</h2>
<p>Primeiro parágrafo.</p>
<p>Segundo parágrafo.</p>
```

CSS

```css
p {
    color: #0b4f8a;
    font-size: 18px;
}
```

RESULTADO ESPERADO



> 💡 **DICA**
> Ótimo para definir o aspeto «por omissão» do site (`body` , `p` , `h2` , `img` …).

.classe { }

SELETOR DE CLASSE

Função	Estiliza os elementos que tenham essa **classe**. Pode ser usada em **vários** elementos.
Sintaxe	`.nome { propriedade: valor; }`
Quando usar	Quando queres estilizar **alguns** elementos, não todos.

EXEMPLO

HTML

```html
<p>Texto normal.</p>
<p class="destaque">Texto em destaque.</p>
<p class="destaque">Outro destaque.</p>
```

CSS

```css
.destaque {
    background: yellow;
    font-weight: bold;
}
```

RESULTADO ESPERADO

Texto normal.

Texto em destaque.

Outro destaque.

✅ **BOA PRÁTICA**

Um elemento pode ter **várias classes**, separadas por espaços: `class="cartao destaque"`.

❌ **ERRO FREQUENTE**

Esquecer o **ponto** no CSS (`destaque { }`): passa a procurar uma tag chamada «destaque».

#id { }

SELETOR DE ID

Função	Estiliza **um único** elemento, identificado por `id`.
Sintaxe	`#nome { propriedade: valor; }`
Quando usar	Para um elemento **único** na página (ou como âncora de ligação).

EXEMPLO

HTML

```html
<h2 id="titulo">Galeria</h2>
<p>Fotografias da escola.</p>
```

CSS

```css
#titulo {
    color: white;
    background: #e06a1b;
    padding: 6px;
}
```

RESULTADO ESPERADO

Galeria

Fotografias da escola.

❌ **ERRO FREQUENTE**

Repetir o mesmo `id` em vários elementos. O `id` tem de ser **único**; para vários usa `class`.

h1, h2 { }

SELETORES MÚLTIPLOS

Função	Aplica **o mesmo estilo** a vários seletores, separados por vírgula.
Sintaxe	`seletor1, seletor2 { … }`
Quando usar	Evita repetir a mesma regra.

EXEMPLO

HTML

```html
<h1>Título 1</h1>
<h2>Título 2</h2>
<p>Parágrafo.</p>
```

CSS

```css
h1, h2 {
    color: #b5245b;
    font-family: Georgia, serif;
}
```

RESULTADO ESPERADO

Título 1

Título 2

Parágrafo.

nav a { }

SELETOR DESCENDENTE

Função	Seleciona elementos **dentro de** outros (com um espaço entre os seletores).
Sintaxe	`pai filho { … }`
Quando usar	Estilizar só as ligações do menu, sem afetar as restantes.

EXEMPLO

HTML

```html
<nav>
    <a href="#">Início</a>
    <a href="#">Galeria</a>
</nav>
<p><a href="#">Outra ligação</a></p>
```

CSS

```css
nav a {
    color: white;
    background: #0b4f8a;
    padding: 4px 8px;
    text-decoration: none;
}
```

RESULTADO ESPERADO

Início Galeria

Outra ligação

Só as ligações dentro de `<nav>` mudaram; a da última linha ficou igual.

:hover — PSEUDO-CLASSE

Função	Estilo aplicado quando o **rato passa por cima** do elemento.
Sintaxe	`seletor:hover { … }`
Quando usar	Menus, botões e ligações.

EXEMPLO

HTML

```html
<a class="botao" href="#">Passa o rato</a>
```

CSS

```css
.botao {
    background: #0b4f8a;
    color: white;
    padding: 6px 12px;
    text-decoration: none;
}

.botao:hover {
    background: #e06a1b;
}
```

RESULTADO ESPERADO

RATO FORA



RATO POR CIMA



> 💡 **DICA**
> Foi assim que criámos o menu do projeto: `nav a:hover { … }`.

Classe ou id?

	Classe (`.`)	Id (`#`)
Quantos elementos?	Muitos	Só **um** por página
Em HTML	`class="cartao"`	`id="topo"`
Em CSS	`.cartao { }`	`#topo { }`
Prefere	**Na maioria dos casos**, para estilos	Para ligações internas e para o JavaScript

> ⚠️ **ATENÇÃO**
> Quando duas regras **entram em conflito**, ganha a mais **específica**: `#id` vence `.classe`, que vence `elemento`. Se forem igualmente específicas, ganha a **última** escrita.

Conflito de regras

ESPECIFICIDADE

CÓDIGO

HTML

```html
<p id="a" class="b">Que cor tenho?</p>
```

CSS

```css
p { color: red; }
.b { color: blue; }
#a { color: green; }
```

RESULTADO ESPERADO

Que cor tenho?

*O texto fica **verde**: o id é mais específico do que a classe e do que o elemento.*

🏗️ Projeto progressivo · Etapa 10 de 14

MENU DE NAVEGAÇÃO

O QUE ACRESCENTAS

css/estilo.css — acrescenta

```css
/* ===== Menu de navegação ===== */
nav ul {
    list-style: none;
    margin: 0;
    padding: 0;
}

nav li {
    display: inline-block;
    margin-right: 8px;
}

nav a {
    display: inline-block;
    color: #fff;
    text-decoration: none;
    padding: 8px 14px;
    background: rgba(255, 255, 255, 0.15);
    border-radius: 6px;
}

nav a:hover { background: #e06a1b; }
```

RESULTADO ESPERADO



*Usámos seletores **descendentes** (`nav ul`, `nav a`) e a pseudo-classe `:hover`. No capítulo 13 vamos melhorar o menu com Flexbox.*

> 🧪 **EXPERIMENTA TU 14 — CLASSES**
> Cria três parágrafos, dá a classe `aviso` ao segundo e ao terceiro, e cria uma regra `.aviso` com fundo amarelo. Depois acrescenta `#importante` a um deles e uma cor diferente.

🧩 COMPLETA O CÓDIGO — D26 (5 MIN)

Completa para que só o parágrafo com a classe `nota` fique itálico.

HTML

```html
<p>Normal.</p>
<p class="nota">Nota.</p>
```

CSS

```css
___ {
    font-style: italic;
}
```

🎯 QUAL SERÁ O RESULTADO? — D27 (5 MIN)

De que cor fica o texto do parágrafo?

HTML

```html
<p id="x" class="y">Texto</p>
```

CSS

```css
#x { color: green; }
.y { color: orange; }
```

🔍 ENCONTRA O ERRO — D28 (5 MIN)

O estilo `.cartao` não é aplicado. Porquê?

HTML

```html
<div id="cartao">Olá</div>
```

CSS

```css
.cartao { background: yellow; }
```

📋 Ficha rápida — Seletores

Elemento	Função	Exemplo
`p`	Elemento	`p { … }`
`.x`	Classe	`.x { … }`
`#x`	Id	`#x { … }`
`a, b`	Vários	`h1, h2 { … }`
`a b`	Descendente	`nav a { … }`
`:hover`	Rato por cima	`a:hover { … }`

PARTE B · CAPÍTULO 11

Propriedades CSS mais usadas

Há centenas de propriedades, mas com cerca de **vinte** constrói-se a maioria dos sites. Aqui estão as mais úteis, por categoria.

Neste capítulo vais aprender a:

- formatar texto: cor, tipo de letra, tamanho e alinhamento;
- definir fundos e imagens de fundo;
- controlar dimensões, margens e espaçamentos;
- criar contornos e cantos arredondados.

Valores: cores e unidades

Cores	Exemplo	Unidades	Significado
Nome	`red`	`px`	pixéis (tamanho fixo)
Hexadecimal	`#0b4f8a`	`%`	percentagem do elemento-pai
RGB	`rgb(11, 79, 138)`	`em / rem`	relativo ao tamanho da letra
RGBA (transparência)	`rgba(0,0,0,0.5)`	`vw / vh`	percentagem da largura / altura do ecrã

11.1 Texto

`color` — TEXTO

Função Define a **cor do texto**.

Sintaxe `color: cor;`

Quando usar Sempre que precisares de mudar a cor da letra.

EXEMPLO

HTML

```html
<p>Texto a preto.</p>
<p class="a">Texto a azul.</p>
```

CSS

```css
p { color: black; }
.a { color: #0b4f8a; }
```

RESULTADO ESPERADO

Texto a preto.

Texto a azul.

> 💡 **DICA**
> Escolhe cores com **bom contraste** em relação ao fundo: texto claro sobre fundo claro não se lê.

font-family

TEXTO

Função Define o **tipo de letra**.

Sintaxe `font-family: tipo1, tipo2, família;`

Quando usar Para escolher a letra do site (normalmente no `body`).

EXEMPLO

HTML

```
<p class="a">Arial, sem serifas.</p>
<p class="b">Georgia, com serifas.</p>
<p class="c">Courier, monoespaçada.</p>
```

CSS

```
.a { font-family: Arial, sans-serif; }
.b { font-family: Georgia, serif; }
.c { font-family: "Courier New", monospace; }
```

RESULTADO ESPERADO

Arial, sem serifas.

Georgia, com serifas.

Courier, monoespaçada.

✅ **BOA PRÁTICA**

Indica **alternativas** e termina com uma família genérica (`sans-serif`, `serif`, `monospace`).

font-size

TEXTO

Função Define o **tamanho da letra**.

Sintaxe `font-size: 16px;`

Quando usar Títulos, destaques e texto corrido.

EXEMPLO

HTML

```
<p class="p">Pequeno (14px)</p>
<p class="m">Médio (18px)</p>
<p class="g">Grande (26px)</p>
```

CSS

```
.p { font-size: 14px; }
.m { font-size: 18px; }
.g { font-size: 26px; }
```

RESULTADO ESPERADO

Pequeno (14px)

Médio (18px)

Grande (26px)

font-weight

TEXTO

Função Define a **espessura** (negrito).

Sintaxe `font-weight: normal | bold | 100…900;`

Quando usar Destacar texto sem usar `<strong>`, ou retirar o negrito a um título.

EXEMPLO

HTML

```
<p class="n">Normal</p>
<p class="b">Negrito</p>
```

CSS

```
.n { font-weight: normal; }
.b { font-weight: bold; }
```

RESULTADO ESPERADO

Normal

Negrito

font-style

TEXTO

Função	Define o **estilo** da letra (itálico).
Sintaxe	`font-style: normal | italic;`
Quando usar	Citações e notas.

EXEMPLO

HTML

```html
<p class="i">Texto em itálico.</p>
<p>Texto normal.</p>
```

CSS

```css
.i { font-style: italic; }
```

RESULTADO ESPERADO

Texto em itálico.

Texto normal.

text-align

TEXTO

Função	Define o **alinhamento horizontal** do texto.
Sintaxe	`text-align: left | center | right | justify;`
Quando usar	Títulos centrados, rodapés, textos de tabela.

EXEMPLO

HTML

```html
<p class="e">Esquerda</p>
<p class="c">Centro</p>
<p class="d">Direita</p>
```

CSS

```css
.e { text-align: left; }
.c { text-align: center; }
.d { text-align: right; }
```

RESULTADO ESPERADO

Esquerda

Centro

Direita

*Aplica-se ao **conteúdo** de um bloco, não ao próprio bloco.*

text-decoration

TEXTO

Função	Adiciona ou remove **sublinhado** e traços.
Sintaxe	`text-decoration: none | underline | line-through;`
Quando usar	Menus (retirar o sublinhado às ligações).

EXEMPLO

HTML

```html
<a href="#">Ligação normal</a><br>
<a class="s" href="#">Ligação sem sublinhado</a>
<br>
<span class="r">Preço antigo</span>
```

CSS

```css
.s { text-decoration: none; }
.r { text-decoration: line-through; }
```

RESULTADO ESPERADO

Ligação normal
Ligação sem sublinhado
~~Preço antigo~~

> 💡 **DICA**
> Não removas o sublinhado das ligações em texto corrido: deixa de se perceber que são clicáveis.

line-height — TEXTO

Função	Define a **altura da linha** (espaço entre linhas).
Sintaxe	`line-height: 1.6;`
Quando usar	Melhorar a leitura de textos longos.

EXEMPLO

HTML

```
<p class="a">Linhas muito juntas. Linhas muito
juntas. Linhas muito juntas. Linhas muito juntas.
</p>
<p class="b">Linhas mais espaçadas. Linhas mais
espaçadas. Linhas mais espaçadas. Linhas mais
espaçadas.</p>
```

CSS

```
.a { line-height: 1; }
.b { line-height: 1.6; }
```

RESULTADO ESPERADO

Linhas muito juntas. Linhas muito juntas. Linhas muito juntas. Linhas muito juntas.

Linhas mais espaçadas. Linhas mais espaçadas. Linhas mais espaçadas. Linhas mais espaçadas.

> ✅ **BOA PRÁTICA**
> Usa valores **sem unidade** entre `1.4` e `1.7` para texto corrido.

11.2 Fundo

background-color — FUNDO

Função	Define a **cor de fundo**.
Sintaxe	`background-color: cor;`
Quando usar	Cabeçalhos, botões, cartões.

EXEMPLO

HTML

```
<p class="a">Fundo amarelo claro</p>
<p class="b">Fundo azul, letra branca</p>
```

CSS

```
.a { background-color: #fff3b0; }
.b { background-color: #0b4f8a; color: white; }
```

RESULTADO ESPERADO

Fundo amarelo claro

Fundo azul, letra branca

background-image

FUNDO

Função	Coloca uma **imagem de fundo** num elemento.
Sintaxe	`background-image: url("imagens/x.jpg");`
Quando usar	Faixas e cabeçalhos decorativos.

EXEMPLO

HTML

```html
<div class="banner">Clube de Fotografia</div>
```

CSS

```css
.banner {
    background-image: url("imagens/paisagem.jpg");
    color: white;
    font-size: 22px;
    font-weight: bold;
    padding: 40px 10px;
}
```

RESULTADO ESPERADO



Por omissão a imagem mantém o tamanho original e repete-se: aqui só se vê o canto superior. Vê `background-size` a seguir.

> ⚠️ **ATENÇÃO**
> No CSS o caminho é relativo ao **ficheiro CSS**, não à página HTML. Se o CSS está em `css/`, usa `../imagens/x.jpg`.

background-size

FUNDO

Função	Controla como a imagem de fundo **se ajusta**.
Sintaxe	`background-size: cover | contain | 100px;`
Quando usar	`cover`: preenche todo o elemento (pode cortar); `contain`: mostra a imagem inteira.

EXEMPLO

HTML

```html
<div class="banner">Cobre tudo</div>
```

CSS

```css
.banner {
    background-image: url("imagens/praia.jpg");
    background-size: cover;
    color: white;
    font-weight: bold;
    padding: 40px 10px;
}
```

RESULTADO ESPERADO



background-position — FUNDO

Função	Define a **posição** da imagem de fundo.
Sintaxe	`background-position: center | top | left bottom;`
Quando usar	Escolher a parte da imagem que fica visível.

EXEMPLO

HTML

```html
<div class="banner">Centrada</div>
```

CSS

```css
.banner {
    background-image: url("imagens/montanha.jpg");
    background-size: cover;
    background-position: center;
    color: white;
    font-weight: bold;
    padding: 40px 10px;
}
```

RESULTADO ESPERADO



11.3 Dimensões

width — DIMENSÕES

Função	Define a **largura** do elemento.
Sintaxe	`width: 200px | 50% | auto;`
Quando usar	Caixas, imagens, colunas.

EXEMPLO

HTML

```html
<div class="caixa">Metade da largura</div>
```

CSS

```css
.caixa {
    width: 50%;
    background: #dbeafe;
    padding: 6px;
}
```

RESULTADO ESPERADO

Metade da largura

*Os elementos de **bloco** ocupam por omissão toda a largura disponível.*

height

DIMENSÕES

Função	Define a **altura** do elemento.
Sintaxe	`height: 80px;`
Quando usar	Faixas e imagens.

EXEMPLO

HTML

```html
<div class="caixa">Altura fixa</div>
```

CSS

```css
.caixa {
    height: 80px;
    background: #dbeafe;
    padding: 6px;
}
```

RESULTADO ESPERADO

Altura fixa

❌ **ERRO FREQUENTE**

Fixar a altura de caixas com texto: se o texto crescer, **sai da caixa**. Deixa a altura ao cuidado do conteúdo sempre que puderes.

max-width

DIMENSÕES

Função	Define a **largura máxima**: o elemento nunca passa deste valor.
Sintaxe	`max-width: 900px;`
Quando usar	Conteúdo centrado e imagens responsivas.

EXEMPLO

HTML

```html
<div class="caixa">Largura máxima de 200px, mesmo
que haja mais espaço.</div>
```

CSS

```css
.caixa {
    max-width: 200px;
    background: #dbeafe;
    padding: 6px;
}
```

RESULTADO ESPERADO

Largura máxima de 200px, mesmo que haja mais espaço.

✅ **BOA PRÁTICA**

`img { max-width: 100%; }` impede que as imagens saiam do ecrã.

`min-width` — DIMENSÕES

Função	Define a **largura mínima**: o elemento nunca fica mais estreito.
Sintaxe	`min-width: 200px;`
Quando usar	Evitar que botões e colunas fiquem demasiado estreitos.

EXEMPLO

HTML

```
<div class="caixa">Nunca fico com menos de 200px.
</div>
```

CSS

```
.caixa {
    min-width: 200px;
    background: #dbeafe;
    padding: 6px;
}
```

RESULTADO ESPERADO

Nunca fico com menos de 200px.

11.4 Espaçamentos

`margin` — ESPAÇAMENTO

Função	Espaço **exterior**: afasta o elemento dos vizinhos.
Sintaxe	`margin: 20px;`
Quando usar	Separar blocos uns dos outros.

EXEMPLO

HTML

```
<div class="a">Caixa A</div>
<div class="b">Caixa B</div>
```

CSS

```
.a, .b {
    background: #fde9d9;
    border: 1px solid #e06a1b;
}
.b { margin: 20px; }
```

RESULTADO ESPERADO

Caixa A

Caixa B

*A margem é **transparente**: fica fora da caixa.*

padding — ESPAÇAMENTO

Função	Espaço **interior**: afasta o conteúdo da borda.
Sintaxe	`padding: 20px;`
Quando usar	Dar «ar» ao conteúdo de caixas e botões.

EXEMPLO

HTML

```html
<div class="a">Sem padding</div>
<div class="b">Com padding</div>
```

CSS

```css
.a, .b {
    background: #dbeafe;
    border: 1px solid #1d4ed8;
    margin-bottom: 8px;
}
.b { padding: 20px; }
```

RESULTADO ESPERADO

Sem padding

Com padding

*O padding fica **dentro** da caixa, por isso mostra o fundo.*

Escrita	Significado
`margin: 10px;`	10px nos **quatro** lados
`margin: 10px 20px;`	10px em cima e em baixo, 20px à esquerda e à direita
`margin: 5px 10px 15px 20px;`	cima, direita, baixo, esquerda (sentido dos ponteiros do relógio)
`margin: 0 auto;`	sem margem vertical e **centrado** horizontalmente (com largura definida)
`padding-left: 12px;`	só de um lado (`top`, `right`, `bottom`, `left`)

11.5 Contornos e cantos

border — CONTORNO

Função	Desenha um **contorno**: espessura, estilo e cor.
Sintaxe	`border: 2px solid #0b4f8a;`
Quando usar	Cartões, campos de formulário, separadores.

EXEMPLO

HTML

```html
<p class="a">Contorno contínuo</p>
<p class="b">Tracejado</p>
<p class="c">Só em baixo</p>
```

CSS

```css
.a { border: 2px solid #0b4f8a; padding: 6px; }
.b { border: 2px dashed #e06a1b; padding: 6px; }
.c { border-bottom: 3px solid #0f7a48; }
```

RESULTADO ESPERADO

Contorno contínuo

Tracejado

Só em baixo

> 💡 **DICA**
> Ordem: **espessura, estilo, cor**. Sem o estilo (`solid`) o contorno não aparece.

`border-radius` — CONTORNO

Função	Arredonda os **cantos**.
Sintaxe	`border-radius: 10px | 50%;`
Quando usar	Botões, cartões e imagens redondas (`50%` num quadrado dá um círculo).

EXEMPLO

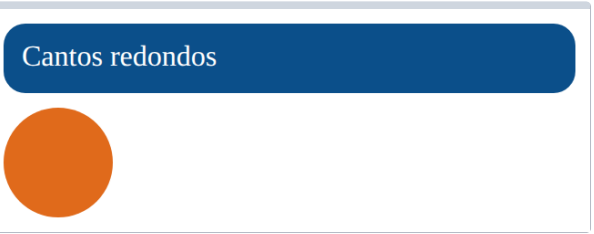
HTML

```html
<div class="a">Cantos redondos</div>
<div class="b"></div>
```

CSS

```css
.a {
    background: #0b4f8a;
    color: white;
    padding: 10px;
    border-radius: 12px;
}
.b {
    width: 60px;
    height: 60px;
    margin-top: 8px;
    background: #e06a1b;
    border-radius: 50%;
}
```

RESULTADO ESPERADO



`display` — DISPOSIÇÃO

Função	Define **como o elemento se comporta** na página.
Sintaxe	`display: block | inline | inline-block | none | flex | grid;`
Quando usar	Menus horizontais (`inline-block`), esconder elementos (`none`) e, nos próximos capítulos, `flex` e `grid`.

EXEMPLO

HTML

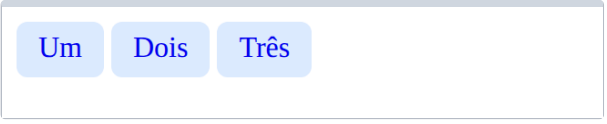
```html
<a href="#">Um</a> <a href="#">Dois</a> <a
href="#">Três</a>
<p class="x">Só ocupo o espaço necessário.</p>
```

CSS

```css
a {
    display: inline-block;
    padding: 6px 12px;
    background: #dbeafe;
    border-radius: 6px;
    text-decoration: none;
}

.x { display: none; }
```

RESULTADO ESPERADO



*O segundo parágrafo com `display: none` **desapareceu** da página.*

> 💡 **DICA**
> Outras propriedades úteis para explorares: `box-shadow` (sombras), `opacity` (transparência), `cursor` (forma do rato), `list-style` (marcadores das listas) e `overflow` (conteúdo que ultrapassa a caixa).

🧪 EXPERIMENTA TU 15 — UMA PÁGINA BONITA

Pega numa página tua com títulos, parágrafos e uma lista. Muda a **cor de fundo** do `body`, o **tipo de letra** e o **tamanho** do texto; dá aos `h2` uma cor e um `border-bottom`; acrescenta `padding` e `border-radius` a uma caixa.

🧩 COMPLETA O CÓDIGO — D29 (5 MIN)

Completa para obter um parágrafo com texto branco, fundo azul e cantos arredondados de 8px.

```
p {
    ___: white;
    ___: blue;
    ___: 8px;
}
```

🎯 QUAL SERÁ O RESULTADO? — D30 (5 MIN)

Quantos pixéis de espaço há entre o texto e o contorno? E entre o contorno e o elemento vizinho?

```
.caixa {
    border: 1px solid black;
    padding: 20px;
    margin: 10px;
}
```

✏️ MODIFICA O EXEMPLO — D31 (10 MIN)

Modifica o cartão azul do exemplo de `border-radius` para que o texto fique **centrado**, com letra de **20px** e um contorno branco de 3px.

🛠️ CORRIGE O CÓDIGO — D32 (5 MIN)

O contorno não aparece. Corrige.

```
p {
    border: 2px red;
}
```

📋 Ficha rápida — Propriedades CSS

Elemento	Função	Exemplo
`color`	Cor do texto	`color: #333;`
`font-family`	Tipo de letra	`font-family: Arial;`
`font-size`	Tamanho da letra	`font-size: 18px;`
`font-weight`	Negrito	`font-weight: bold;`
`text-align`	Alinhamento	`text-align: center;`
`line-height`	Altura da linha	`line-height: 1.6;`
`background-color`	Cor de fundo	`background-color: #fff;`
`background-image`	Imagem de fundo	`url("x.jpg")`
`width / height`	Largura / altura	`width: 50%;`
`max-width`	Largura máxima	`max-width: 900px;`
`margin`	Espaço exterior	`margin: 0 auto;`
`padding`	Espaço interior	`padding: 10px;`
`border`	Contorno	`border: 1px solid #ccc;`
`border-radius`	Cantos	`border-radius: 8px;`

PARTE B · CAPÍTULO 12

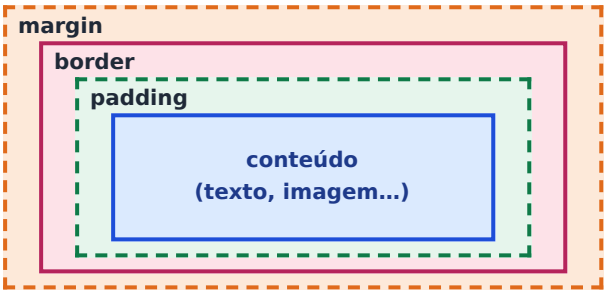
O modelo de caixa (Box Model)

Para o navegador, **tudo é uma caixa**. Perceber as camadas de cada caixa é essencial para controlar espaços e tamanhos.

Neste capítulo vais aprender a:

- identificar as quatro camadas de uma caixa;
- distinguir `margin` de `padding`;
- calcular a largura total de um elemento;
- usar `box-sizing: border-box`.

12.1 As quatro camadas



Camada	Função	Propriedade
Conteúdo	O texto, a imagem ou o que estiver dentro da caixa	`width / height`
Padding	Espaço **interior**, entre o conteúdo e o contorno (tem o fundo do elemento)	`padding`
Border	O contorno da caixa	`border`
Margin	Espaço **exterior**, entre esta caixa e as vizinhas (transparente)	`margin`

> 💡 **DICA**
> No navegador, carrega em `F12` (ferramentas de programador) e passa o rato sobre um elemento: vês estas camadas coloridas, exatamente como no esquema. É a melhor forma de perceber o espaço que falta ou sobra.

Vê as camadas

CAIXA

CÓDIGO

HTML

```html
<div class="caixa">Conteúdo</div>
```

CSS

```css
.caixa {
    width: 160px;
    padding: 20px;
    border: 5px solid #b5245b;
    margin: 15px;
    background: #dbeafe;
}
```

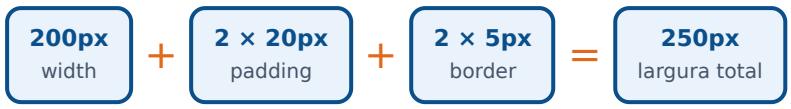
RESULTADO ESPERADO

Conteúdo

Fundo azul = conteúdo + padding; contorno rosa = border; o afastamento à volta = margin.

12.2 Qual é a largura real?

Por omissão (`content-box`), o `width` mede só o **conteúdo**; o padding e o contorno são somados por fora:



Com `box-sizing: border-box`, o `width` passa a incluir o padding e o contorno: a caixa fica com **exatamente** 200px, e o conteúdo encolhe para caber.

box-sizing

MODELO DE CAIXA

Função	Define se `width` e `height` incluem o **padding** e o **contorno**.
Sintaxe	`box-sizing: border-box | content-box;`
Quando usar	No **início de todos os teus ficheiros CSS**.

EXEMPLO

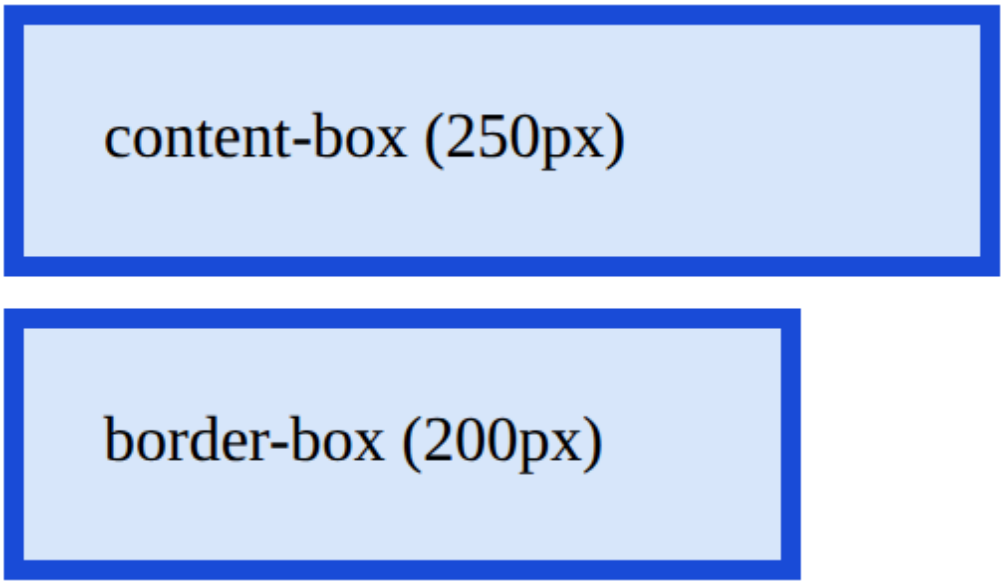
HTML

```html
<div class="a">content-box (250px)</div>
<div class="b">border-box (200px)</div>
```

CSS

```css
.a, .b {
    width: 200px;
    padding: 20px;
    border: 5px solid #1d4ed8;
    background: #dbeafe;
    margin-bottom: 8px;
}
.a { box-sizing: content-box; }
.b { box-sizing: border-box; }
```

RESULTADO ESPERADO



As duas caixas têm `width: 200px`, mas a primeira ocupa 250px. Repara na diferença de larguras.

> ✅ **BOA PRÁTICA**
> Começa sempre o CSS com `* { box-sizing: border-box; }`: os cálculos ficam previsíveis.

💻 CÓDIGO — O PRIMEIRO BLOCO DO ESTILO.CSS

```css
* {
    box-sizing: border-box;
}
```

12.3 Centrar uma caixa

`margin: 0 auto` — MODELO DE CAIXA

Função	Centra **horizontalmente** um bloco com largura definida.
Sintaxe	`width: 300px; margin: 0 auto;`
Quando usar	Centrar o conteúdo principal, como o `<main>` do projeto.

EXEMPLO

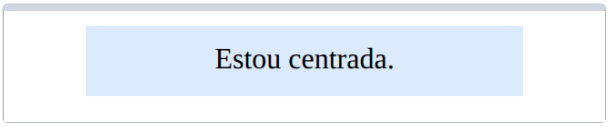
HTML

```html
<div class="caixa">Estou centrada.</div>
```

CSS

```css
.caixa {
    width: 220px;
    margin: 0 auto;
    padding: 10px;
    background: #dbeafe;
    text-align: center;
}
```

RESULTADO ESPERADO



O truque é o `auto`: o navegador reparte o espaço livre igualmente pela esquerda e pela direita.

🧪 EXPERIMENTA TU 16 — F12

Abre o teu projeto no navegador, prime `F12`, seleciona o `<main>` e observa a margem e o padding. Altera valores diretamente no painel e vê o efeito (não se guarda no ficheiro).

🎯 QUAL SERÁ O RESULTADO? — D33 (5 MIN)

Qual é a largura total desta caixa, sem `box-sizing: border-box`?

```css
.caixa {
    width: 100px;
    padding: 10px;
    border: 2px solid black;
}
```

🛠️ CORRIGE O CÓDIGO — D34 (5 MIN)

Queremos uma caixa de **300px** no total, com padding de 20px, mas ela ocupa mais. Corrige.

```css
.caixa {
    width: 300px;
    padding: 20px;
    border: 2px solid black;
}
```

🧩 COMPLETA O CÓDIGO — D35 (5 MIN)

Completa para que a caixa fique centrada na página.

```css
.caixa {
    width: 400px;
    ___: 0 ___;
}
```

📋 Ficha rápida — Box model

Elemento	Função	Exemplo
`content`	Conteúdo	`width / height`
`padding`	Espaço interior	`padding: 10px;`
`border`	Contorno	`border: 1px solid;`
`margin`	Espaço exterior	`margin: 10px;`
`box-sizing`	Inclui padding e border	`box-sizing: border-box;`
`margin: 0 auto`	Centrar bloco	`width: 800px;`

PARTE B · CAPÍTULO 13

Flexbox

O **Flexbox** organiza elementos **numa linha ou numa coluna**: alinhar, distribuir espaço e centrar tornou-se simples.

Neste capítulo vais aprender a:

- ativar o Flexbox com `display: flex`;
- controlar direção, distribuição e alinhamento;
- criar menus, cartões e galerias;
- aplicar Flexbox ao menu e aos cartões do projeto.

13.1 A ideia

Coloca-se `display: flex` no elemento-**pai** (o *contentor*). Os filhos diretos (os *itens*) passam a alinhar-se lado a lado e podem ser organizados com poucas propriedades.



Eixo	Direção por omissão	Propriedade que o controla
Principal	horizontal (da esquerda para a direita)	`justify-content`
Secundário	vertical (de cima para baixo)	`align-items`

> 💡 **DICA**
> Se mudares `flex-direction` para `column`, os eixos **trocam**: o principal passa a ser vertical.

`display: flex` — FLEXBOX

Função	Transforma o elemento num **contentor flex**: os filhos ficam lado a lado.
Sintaxe	`display: flex;`
Quando usar	No pai dos elementos que queres organizar.

EXEMPLO

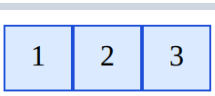
HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

CSS

```css
.caixa {
    display: flex;
}
```

RESULTADO ESPERADO



Sem `display: flex` as três caixas ficariam empilhadas (são blocos).

> 💡 **DICA**
> Só os **filhos diretos** ficam flexíveis, não os netos.

flex-direction

FLEXBOX

Função	Define a **direção** dos itens: em linha ou em coluna.
Sintaxe	`flex-direction: row | column;`
Quando usar	Trocar entre disposição horizontal e vertical (muito usado em ecrãs pequenos).

EXEMPLO

HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

CSS

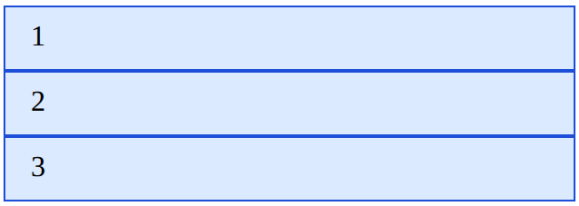
```css
.caixa {
    display: flex;
    flex-direction: row;
}
```

RESULTADO ESPERADO

ROW (POR OMISSÃO)



COLUMN



justify-content

FLEXBOX

Função	Distribui os itens ao longo do **eixo principal**.
Sintaxe	`justify-content: flex-start | center | flex-end | space-between | space-around;`
Quando usar	Centrar, empurrar para um lado ou repartir o espaço livre.

EXEMPLO

HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

CSS

```css
.caixa {
    display: flex;
    justify-content: flex-start;
}
```

RESULTADO ESPERADO

FLEX-START



CENTER



FLEX-END



SPACE-BETWEEN



SPACE-AROUND



> ✅ **BOA PRÁTICA**
> `space-between` é ótimo para menus: primeiro e último item ficam nas pontas.

align-items

FLEXBOX

Função Alinha os itens no **eixo secundário** (na vertical, em linha).

Sintaxe `align-items: stretch | flex-start | center | flex-end;`

Quando usar Centrar verticalmente, alinhar ao topo ou à base.

EXEMPLO

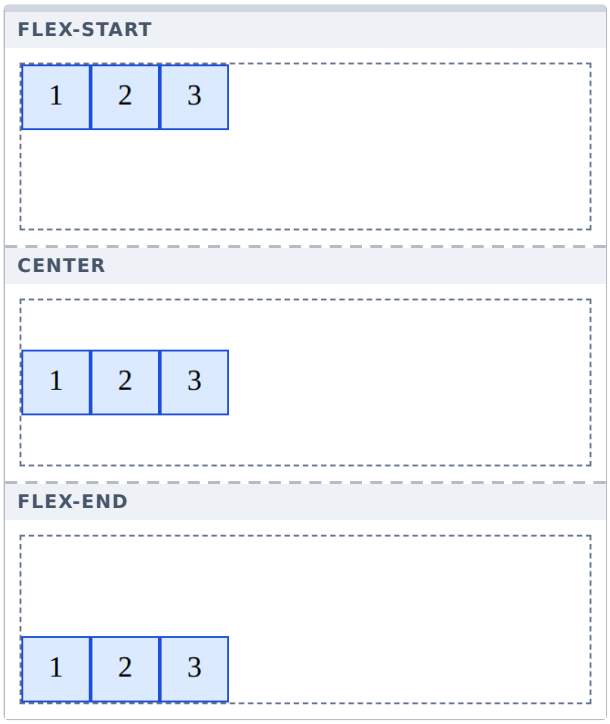
HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

CSS

```css
.caixa {
    display: flex;
    align-items: flex-start;
    height: 90px;
    border: 1px dashed #64748b;
}
```

RESULTADO ESPERADO



*O contorno tracejado é o contentor; precisa de **altura** para se ver o efeito.*

gap

FLEXBOX

Função Define o **espaço entre** os itens.

Sintaxe `gap: 16px;`

Quando usar Sempre que quiseres separar itens (em vez de usar margens).

EXEMPLO

HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

CSS

```css
.caixa {
    display: flex;
    gap: 16px;
}
```

RESULTADO ESPERADO



> ✅ **BOA PRÁTICA**
> Prefere `gap` às margens: não cria espaço a mais nas pontas.

flex-wrap

FLEXBOX

Função	Permite que os itens **passem para a linha seguinte** quando não cabem.
Sintaxe	`flex-wrap: nowrap | wrap;`
Quando usar	Galerias e listas com muitos itens.

EXEMPLO

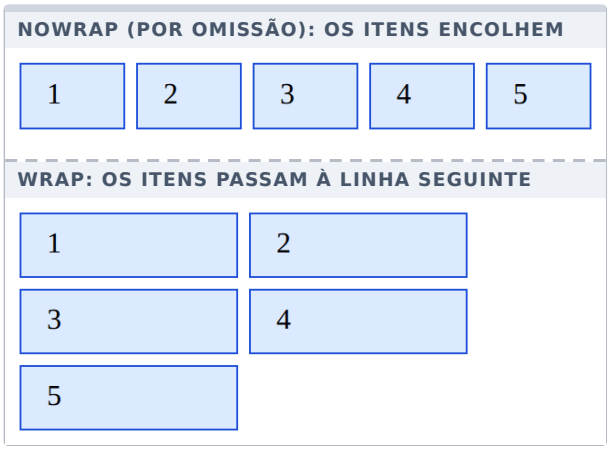
HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
</div>
```

CSS

```css
.caixa {
    display: flex;
    flex-wrap: nowrap;
    gap: 6px;
}
.item { width: 90px; }
```

RESULTADO ESPERADO



Com `nowrap` os itens são espremidos numa só linha.

flex: 1

FLEXBOX (ITENS)

Função	Faz o item **crescer** para ocupar o espaço livre; vários itens com `flex: 1` ficam com a **mesma largura**.
Sintaxe	`flex: 1;`
Quando usar	Colunas e cartões iguais.

EXEMPLO

HTML

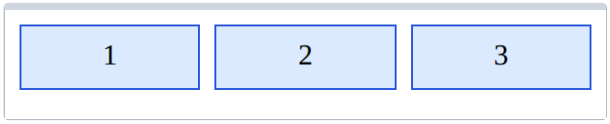
```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

CSS

```css
.caixa {
    display: flex;
    gap: 8px;
}

.item { flex: 1; }
```

RESULTADO ESPERADO



> 💡 **DICA**
> Foi o que usámos nos cartões do projeto.

13.2 Padrões que vais usar sempre

Menu horizontal

PADRÃO 1

CÓDIGO

HTML

```html
<nav>
    <ul>
        <li><a href="#">Início</a></li>
        <li><a href="#">Galeria</a></li>
        <li><a href="#">Contactos</a></li>
    </ul>
</nav>
```

CSS

```css
nav ul {
    display: flex;
    gap: 10px;
    list-style: none;
    margin: 0;
    padding: 0;
    background: #0b4f8a;
}

nav a {
    display: block;
    padding: 8px 12px;
    color: white;
    text-decoration: none;
}
```

RESULTADO ESPERADO



A lista tem `display: flex`; os `<li>` ficam lado a lado. Fica mais simples do que `inline-block`.

Centrar tudo (horizontal e vertical)

PADRÃO 2

CÓDIGO

HTML

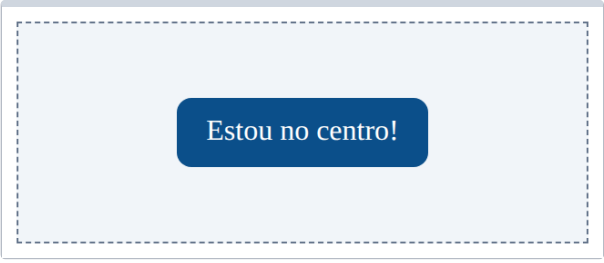
```html
<div class="ecra">
    <div class="centro">Estou no centro!</div>
</div>
```

CSS

```css
.ecra {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 120px;
    background: #f1f5f9;
    border: 1px dashed #64748b;
}

.centro {
    background: #0b4f8a;
    color: white;
    padding: 10px 16px;
    border-radius: 8px;
}
```

RESULTADO ESPERADO



Dois valores `center` resolvem um problema que durante anos foi difícil em CSS.

Cartões lado a lado

PADRÃO 3

CÓDIGO

HTML

```html
<div class="cartoes">
    <article class="cartao">
        <h3>Saídas</h3>
        <p>Fotografamos a cidade e a natureza.</p>
    </article>
    <article class="cartao">
        <h3>Workshops</h3>
        <p>Composição, luz e edição de imagem.</p>
    </article>
    <article class="cartao">
        <h3>Exposições</h3>
        <p>Mostramos os melhores trabalhos.</p>
    </article>
</div>
```

CSS

```css
/* ===== Cartões (Flexbox) ===== */
.cartoes {
    display: flex;
    gap: 16px;
}

.cartao {
    flex: 1;
    background: #fff;
    padding: 14px;
    border-radius: 8px;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.12);
}

.cartao h3 { margin-top: 0; color: #0b4f8a; }
```

RESULTADO ESPERADO



Três cartões com `flex: 1` partilham a largura igualmente.

O QUE ACRESCENTAS

css/estilo.css — substitui o menu anterior

```css
/* ===== Menu de navegação (com Flexbox) ===== */
nav ul {
    list-style: none;
    margin: 0;
    padding: 0;
    display: flex;
    gap: 10px;
}

nav a {
    display: block;
    color: #fff;
    text-decoration: none;
    padding: 8px 14px;
    background: rgba(255, 255, 255, 0.15);
    border-radius: 6px;
}

nav a:hover { background: #e06a1b; }
```

css/estilo.css — acrescenta

```css
/* ===== Cartões (Flexbox) ===== */
.cartoes {
    display: flex;
    gap: 16px;
}

.cartao {
    flex: 1;
    background: #fff;
    padding: 14px;
    border-radius: 8px;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.12);
}

.cartao h3 { margin-top: 0; color: #0b4f8a; }
```

RESULTADO ESPERADO



O menu passa a usar `display: flex`. No `index.html`, troca a lista de atividades pelos três cartões do «Padrão 3» (`.cartoes` com três `article.cartao`).

🧪 EXPERIMENTA TU 17 — FLEX-WRAP

Cria um contentor com 8 caixas de 100px de largura. Experimenta `flex-wrap: wrap`, mude a janela do navegador e vê os itens a mudarem de linha.

🧩 COMPLETA O CÓDIGO — D36 (5 MIN)

Completa para que as três caixas fiquem lado a lado, com 10px de espaço, centradas horizontalmente.

```css
.caixa {
    ___: flex;
    ___: center;
    ___: 10px;
}
```

🎯 QUAL SERÁ O RESULTADO? — D37 (5 MIN)

Como ficam dispostos os itens?

```
.caixa {
    display: flex;
    flex-direction: column;
    gap: 8px;
}
```

🔍 ENCONTRA O ERRO — D38 (5 MIN)

Queremos os itens lado a lado, mas continuam empilhados. Porquê?

```
.caixa {
    justify-content: center;
    gap: 10px;
}
```

✏️ MODIFICA O EXEMPLO — D39 (10 MIN)

Altera o «Menu horizontal» para que os itens fiquem **encostados à direita** e o item por baixo do rato fique com fundo laranja.

📋 Ficha rápida — Flexbox

Elemento	Função	Exemplo
`display: flex`	Ativa o Flexbox (no pai)	`display: flex;`
`flex-direction`	Linha ou coluna	`flex-direction: column;`
`justify-content`	Eixo principal	`justify-content: center;`
`align-items`	Eixo secundário	`align-items: center;`
`gap`	Espaço entre itens	`gap: 16px;`
`flex-wrap`	Quebra de linha	`flex-wrap: wrap;`
`flex: 1`	Item que cresce	`flex: 1;`

PARTE B · CAPÍTULO 14

CSS Grid

Enquanto o Flexbox organiza numa **linha** ou numa **coluna**, o **Grid** organiza em **linhas e colunas**, como uma tabela invisível.

Neste capítulo vais aprender a:

- criar uma grelha com `display: grid`;
- definir colunas com `grid-template-columns` e `fr`;
- usar `gap` e `repeat()`;
- construir uma galeria de fotografias.

	Flexbox	Grid
Dimensões	Uma (linha **ou** coluna)	Duas (linhas **e** colunas)
Ideal para	Menus, alinhamentos, cartões numa linha	Galerias, layouts de página, grelhas de cartões
Ativa-se com	`display: flex`	`display: grid`

`display: grid`

GRID

Função Transforma o elemento numa **grelha**. Sem colunas definidas, os itens ficam empilhados.

Sintaxe `display: grid;`

Quando usar No pai dos itens que queres arrumar em grelha.

EXEMPLO

HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
</div>
```

CSS

```css
.caixa {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
}
```

RESULTADO ESPERADO

1	2	3
4	5	6

grid-template-columns

GRID

Função	Define **quantas colunas** há e a largura de cada uma.
Sintaxe	`grid-template-columns: 200px 1fr 1fr;`
Quando usar	Decidir o número e o tamanho das colunas.

EXEMPLO

HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
</div>
```

CSS

```css
.caixa {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 6px;
}
```

RESULTADO ESPERADO

1FR 1FR

1	2
3	4
5	6

1FR 1FR 1FR

1	2	3
4	5	6

100PX 1FR 2FR (UMA COLUNA FIXA)

1	2	3
4	5	6

A unidade `fr` (fração) reparte o espaço livre: `1fr 2fr` dá à segunda coluna o dobro da primeira.

repeat()

GRID

Função Repete um padrão de colunas, para não escreveres o mesmo várias vezes.

Sintaxe `grid-template-columns: repeat(3, 1fr);`

Quando usar Sempre que as colunas sejam iguais.

EXEMPLO

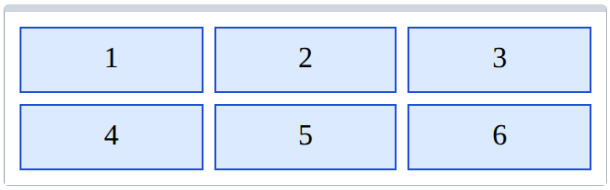
HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
</div>
```

CSS

```css
.caixa {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 6px;
}
```

RESULTADO ESPERADO



> 💡 **DICA**
> `repeat(3, 1fr)` equivale a `1fr 1fr 1fr`: três colunas iguais.

gap

GRID

Função Define o **espaço entre** linhas e colunas.

Sintaxe `gap: 16px; /* linhas colunas: gap: 8px 24px; */`

Quando usar Separar cartões ou fotografias.

EXEMPLO

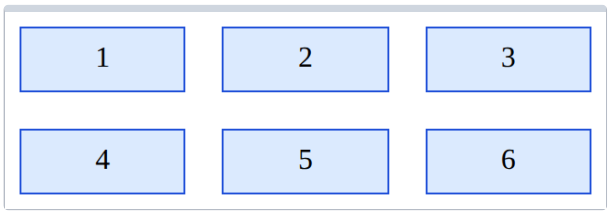
HTML

```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
</div>
```

CSS

```css
.caixa {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}
```

RESULTADO ESPERADO



> ✅ **BOA PRÁTICA**
> Usa `gap` em vez de margens: o espaço só existe **entre** os itens.

grid-column: span

GRID (ITENS)

Função Faz um item ocupar **várias colunas**.

Sintaxe `grid-column: span 2;`

Quando usar Destacar um item maior do que os outros.

EXEMPLO

HTML

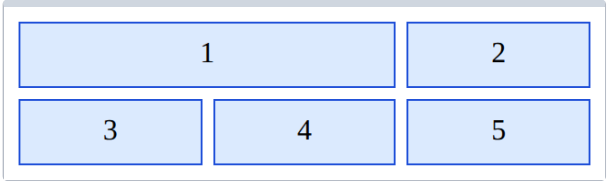
```html
<div class="caixa">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
</div>
```

CSS

```css
.caixa {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 6px;
}

.item:first-child {
    grid-column: span 2;
}
```

RESULTADO ESPERADO



14.1 Galeria de fotografias

Galeria em três colunas GRID

CÓDIGO

HTML

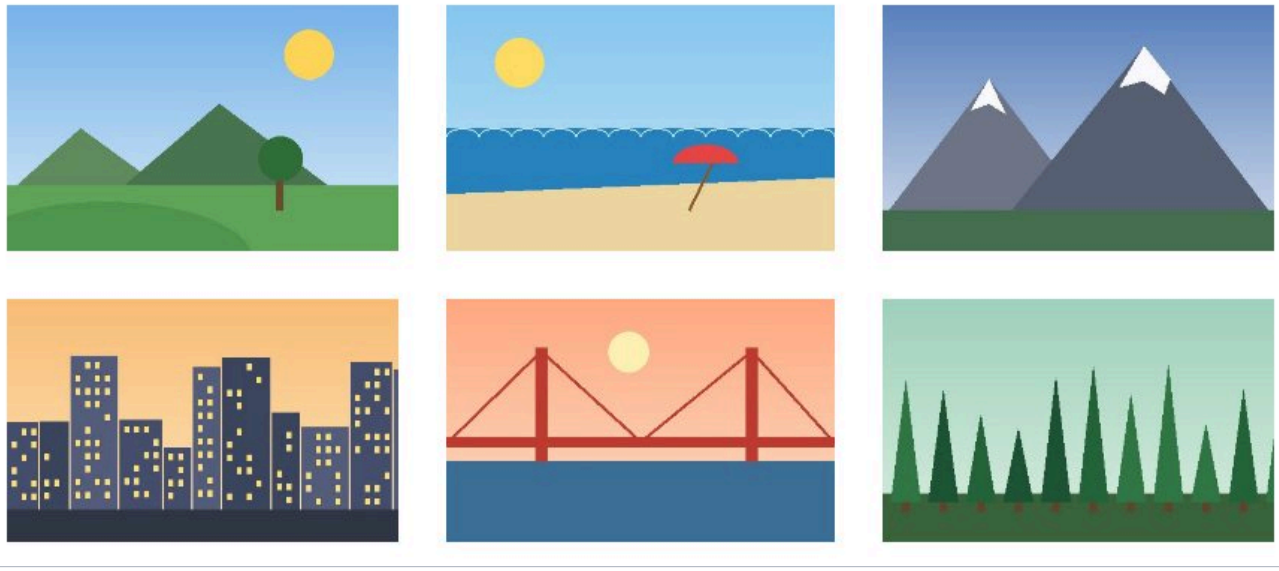
```html
<div class="galeria">
    <img src="imagens/paisagem.jpg" alt="Paisagem com colinas e sol">
    <img src="imagens/praia.jpg" alt="Praia com mar e areia">
    <img src="imagens/montanha.jpg" alt="Montanhas com neve no cume">
    <img src="imagens/cidade.jpg" alt="Cidade ao final da tarde">
    <img src="imagens/ponte.jpg" alt="Ponte ao pôr do sol">
    <img src="imagens/floresta.jpg" alt="Floresta de pinheiros">
</div>
```

CSS

```css
/* ===== Galeria (CSS Grid) ===== */
.galeria {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}

.galeria img { width: 100%; }
```

RESULTADO ESPERADO



Seis fotografias, três colunas, 16px de espaço e `width: 100%` para cada imagem preencher a sua coluna.

> ✅ **BOA PRÁTICA**
> Combina Flexbox e Grid: **Grid** para a estrutura geral e para galerias; **Flexbox** para alinhar o conteúdo **dentro** de cada zona (menus, botões, cartões).

🏗️ Projeto progressivo · Etapa 12 de 14 — A GALERIA (GALERIA.HTML)

O QUE ACRESCENTAS

galeria.html — corpo

```
<main>
    <section>
        <h2>As nossas fotografias</h2>
        <div class="galeria">
        <img src="imagens/paisagem.jpg"
alt="Paisagem com colinas e sol">
        <img src="imagens/praia.jpg" alt="Praia com
mar e areia">
        <img src="imagens/montanha.jpg"
alt="Montanhas com neve no cume">
        <img src="imagens/cidade.jpg" alt="Cidade
ao final da tarde">
        <img src="imagens/ponte.jpg" alt="Ponte ao
pôr do sol">
        <img src="imagens/floresta.jpg"
alt="Floresta de pinheiros">
        </div>
    </section>
</main>
```

css/estilo.css — acrescenta

```
/* ===== Galeria (CSS Grid) ===== */
.galeria {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}

.galeria img { width: 100%; }
```

RESULTADO ESPERADO



Nova página `galeria.html`, com o mesmo cabeçalho, menu e rodapé do `index.html` (só se mostra o essencial).

🧪 EXPERIMENTA TU 18 — GRELHA DE CARTÕES

Cria 6 cartões (com título e texto) numa grelha de **2 colunas**. Depois muda para 3 e para 4 colunas, alterando apenas o número do `repeat()`.

🧩 COMPLETA O CÓDIGO — D40 (5 MIN)

Completa a grelha com **quatro** colunas iguais e 12px de espaço.

```
.galeria {
    ___: grid;
    grid-template-columns: ___(4, ___);
    ___: 12px;
}
```

🎯 QUAL SERÁ O RESULTADO? — D41 (5 MIN)

Quantas colunas tem a grelha e qual é a largura relativa de cada uma?

```
.caixa {
    display: grid;
    grid-template-columns: 1fr 2fr;
}
```

🛠️ CORRIGE O CÓDIGO — D42 (5 MIN)

As imagens não formam uma grelha. Corrige.

```
.galeria {
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}
```

📋 Ficha rápida — CSS Grid

Elemento	Função	Exemplo
`display: grid`	Ativa o Grid (no pai)	`display: grid;`
`grid-template-columns`	Define as colunas	`1fr 1fr 1fr`
`fr`	Fração do espaço livre	`1fr 2fr`
`repeat()`	Repete colunas	`repeat(3, 1fr)`
`gap`	Espaço entre células	`gap: 16px;`
`grid-column`	Item em várias colunas	`grid-column: span 2;`

PARTE B · CAPÍTULO 15

Design responsivo

Um site responsivo **adapta-se** ao tamanho do ecrã: computador, tablet ou telemóvel, com o mesmo código.

Neste capítulo vais aprender a:

- explicar o que é design responsivo;
- usar a meta `viewport`;
- escrever *media queries* com `@media`;
- tornar imagens e layouts flexíveis.

Mais de metade das visitas a sites são feitas em telemóveis. Um site que obrigue a *fazer zoom* e a arrastar para os lados é mau para o utilizador. Para o evitar precisamos de **quatro ingredientes**:

	Ingrediente	Exemplo
1	A meta **viewport** no `<head>`	`<meta name="viewport" content="width=device-width, initial-scale=1.0">`
2	Imagens **flexíveis**	`img { max-width: 100%; height: auto; }`
3	Layouts **flexíveis** (Flexbox e Grid, larguras em `%` e `max-width`)	`display: flex; / display: grid;`
4	**Media queries** para mudar o layout	`@media (max-width: 768px) { … }`

`<meta name="viewport">` — RESPONSIVO

Função — Diz ao **telemóvel** para usar a largura real do ecrã, em vez de fingir que tem um ecrã de computador.

Sintaxe — `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

Quando usar — No `<head>` de **todas** as páginas.

EXEMPLO

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <title>O meu site</title>
</head>
```

RESULTADO ESPERADO

*Sem resultado visível: **a página já incluía esta linha desde o capítulo 2**. Sem ela, os telemóveis mostram o site «encolhido» como se fosse um ecrã de computador.*

❌ **ERRO FREQUENTE**

Esquecer a meta `viewport`: os `@media` deixam de funcionar bem nos telemóveis.

15.1 Media queries

Uma *media query* é uma regra CSS que **só se aplica** quando a condição é verdadeira. A mais usada é `max-width`: «até esta largura de ecrã».

💻 CÓDIGO — ESTRUTURA DE UMA MEDIA QUERY

```
@media (max-width: 768px) {
    /* regras que só valem em ecrãs até 768px */
    .cartoes { flex-direction: column; }
}
```

Cartões que passam a coluna

RESPONSIVO

CÓDIGO

HTML

```
<div class="cartoes">
    <div class="c">Saídas</div>
    <div class="c">Workshops</div>
    <div class="c">Exposições</div>
</div>
```

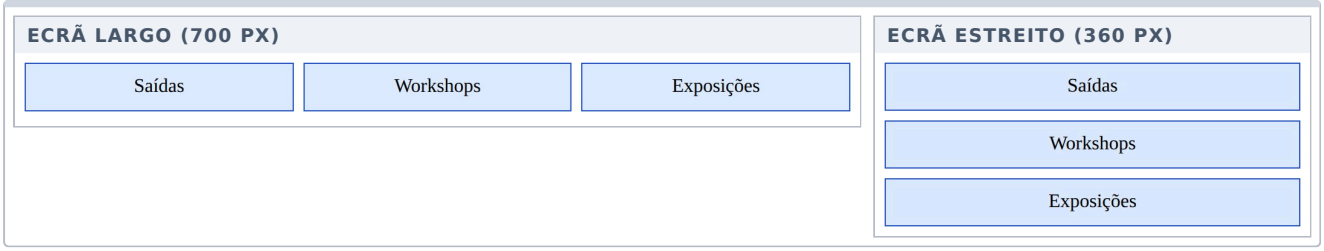
CSS

```
.cartoes {
    display: flex;
    gap: 8px;
}

.c {
    flex: 1;
    background: #dbeafe;
    border: 1px solid #1d4ed8;
    padding: 10px;
    text-align: center;
}

@media (max-width: 500px) {
    .cartoes { flex-direction: column; }
}
```

RESULTADO ESPERADO



O mesmo HTML e o mesmo CSS, dois aspetos diferentes: a regra dentro do `@media` só entrou em ação no ecrã estreito.

Dispositivo	Largura aproximada	Breakpoint habitual
Telemóvel	até 480 px	`@media (max-width: 480px)`
Tablet	481 a 768 px	`@media (max-width: 768px)`
Computador	mais de 768 px	regras normais, sem `@media`

💡 DICA

Os valores 480 px e 768 px são **convenções**. O melhor «ponto de quebra» é aquele em que o **teu layout começa a ficar mal**: redimensiona a janela e vê.

15.2 Imagens flexíveis

`img { max-width: 100% }` RESPONSIVO

Função	Impede que as imagens sejam **mais largas do que o espaço disponível**; `height: auto` mantém a proporção.
Sintaxe	`img { max-width: 100%; height: auto; }`
Quando usar	Em todos os sites, no início do CSS.

EXEMPLO

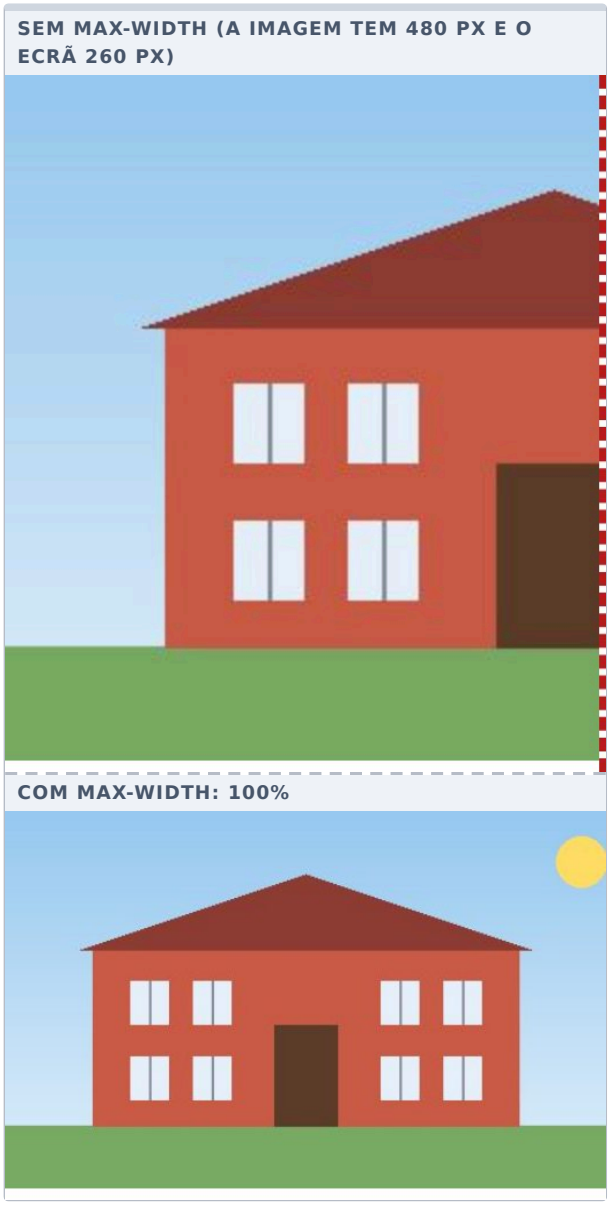
HTML

```html
<img src="imagens/escola.jpg" alt="Escola">
```

CSS

```css
img {
    max-width: 100%;
    height: auto;
}
```

RESULTADO ESPERADO



*Sem a regra, a imagem **sai do ecrã** (a linha vermelha marca o limite do ecrã) e obriga a arrastar para os lados.*

> ✅ **BOA PRÁTICA**
> Coloca esta regra no início do `estilo.css` e esquece o problema.

15.3 Grid responsivo sem media queries (bónus)

auto-fit e minmax

GRID

CÓDIGO

HTML

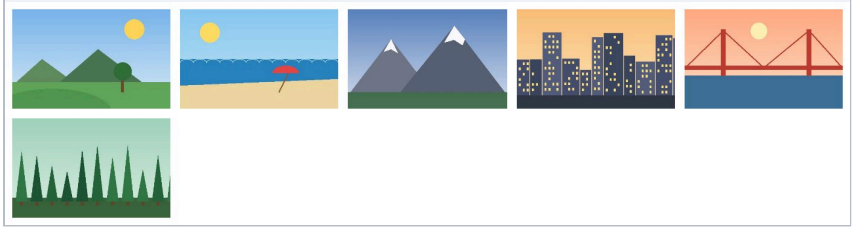
```html
<div class="galeria">
    <img src="imagens/paisagem.jpg" alt="Paisagem">
    <img src="imagens/praia.jpg" alt="Praia">
    <img src="imagens/montanha.jpg" alt="Montanha">
    <img src="imagens/cidade.jpg" alt="Cidade">
    <img src="imagens/ponte.jpg" alt="Ponte">
    <img src="imagens/floresta.jpg" alt="Floresta">
</div>
```

CSS

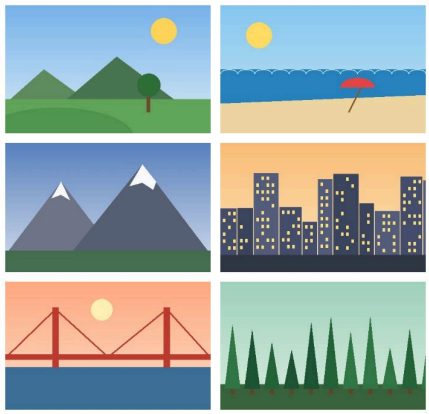
```css
.galeria {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
    gap: 8px;
}
.galeria img { width: 100%; }
```

RESULTADO ESPERADO

ECRÃ LARGO



ECRÃ ESTREITO



O navegador decide sozinho quantas colunas cabem (com pelo menos 120 px cada). É um truque avançado; podes usá-lo já, mas não precisas de o decorar.

15.4 Como testar

1. Abre a página no navegador e carrega em `F12`.
2. Ativa a **barra de dispositivos** (ícone de telemóvel/tablet, ou `Ctrl`+`Shift`+`M`).
3. Escolhe um dispositivo (por exemplo, *iPhone*) ou arrasta a largura.
4. Confirma que **nada sai do ecrã** e que o texto se lê sem fazer zoom.

🏗️ Projeto progressivo · Etapa 13 de 14

O SITE TORNA-SE RESPONSIVO

O QUE ACRESCENTAS

index.html, galeria.html, contactos.html — <head>

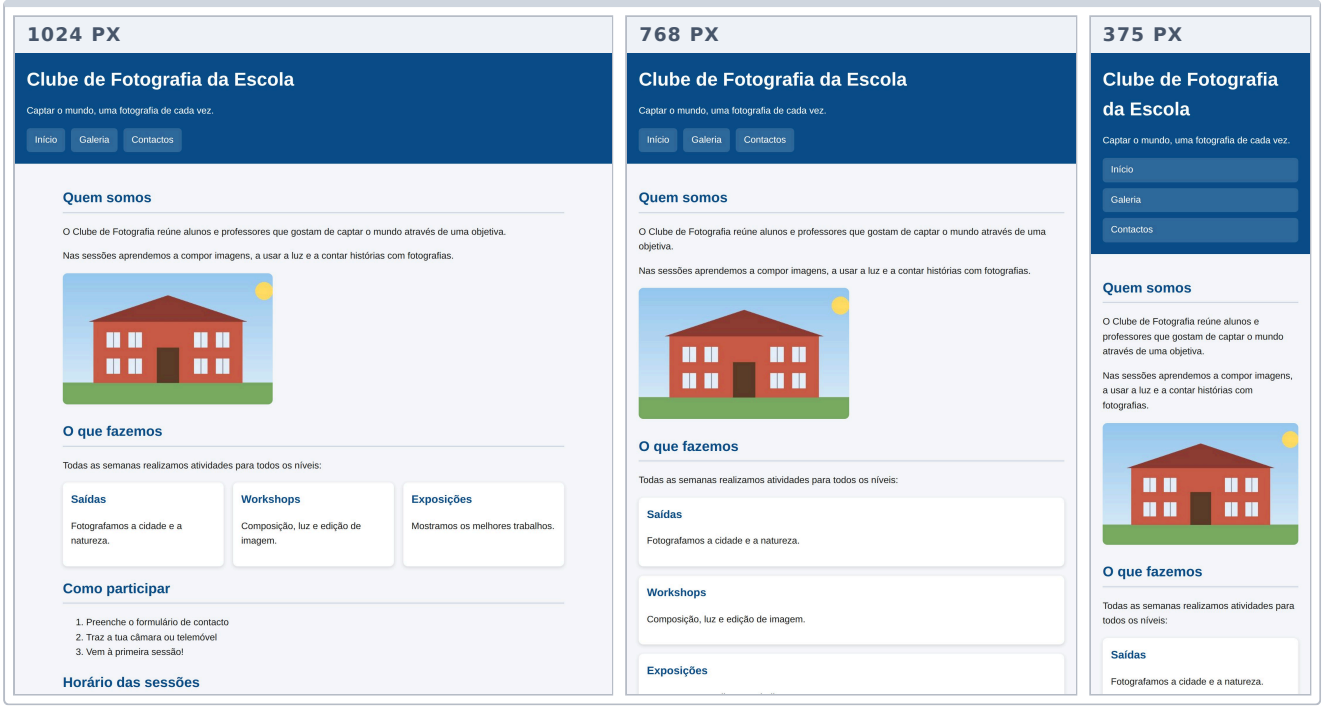
```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

css/estilo.css — acrescenta no fim

```
/* ===== Responsivo ===== */
@media (max-width: 768px) {
    .cartoes { flex-direction: column; }
    .galeria { grid-template-columns: repeat(2, 1fr); }
}

@media (max-width: 480px) {
    nav ul { flex-direction: column; }
    .galeria { grid-template-columns: 1fr; }
}
```

RESULTADO ESPERADO



Em ecrãs até 768 px os cartões passam a coluna e a galeria a duas colunas; até 480 px o menu e a galeria ficam numa só coluna.

🧪 EXPERIMENTA TU 19 — REDIMENSIONA

Abre o teu `index.html`, arrasta o limite da janela do navegador para a esquerda e observa: em que largura muda o layout? Depois altera o `768px` para `1000px` e repete.

🧩 COMPLETA O CÓDIGO — D43 (5 MIN)

Completa para que, em ecrãs até 600 px, o fundo do `body` fique cinzento.

```
@media (___: 600px) {
    body {
        ___: gray;
    }
___
```

🎯 QUAL SERÁ O RESULTADO? — D44 (5 MIN)

Num telemóvel com 400 px de largura, quantas colunas tem a galeria?

```
.galeria { display: grid; grid-template-columns: repeat(3, 1fr); }
@media (max-width: 768px) { .galeria { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 480px) { .galeria { grid-template-columns: 1fr; } }
```

🔍 ENCONTRA O ERRO — D45 (5 MIN)

Nos telemóveis a página aparece muito pequena e o `@media` parece não funcionar. O que falta?

```
<head>
    <meta charset="UTF-8">
    <title>O meu site</title>
    <link rel="stylesheet" href="css/estilo.css">
</head>
```

🛠️ CORRIGE O CÓDIGO — D46 (5 MIN)

A imagem sai do ecrã em telemóveis. Escreve a regra que resolve o problema.

📋 Ficha rápida — Design responsivo

Elemento	Função	Exemplo
`meta viewport`	Largura real do ecrã	`content="width=device-width…"`
`@media`	Regras condicionais	`@media (max-width: 768px) { }`
`max-width`	Até esta largura	`(max-width: 480px)`
`img`	Imagem flexível	`max-width: 100%; height: auto;`
`F12 + Ctrl+Shift+M`	Testar no navegador	`barra de dispositivos`

agrupamento de escolas de rio tinto n.º3

aert3

Linguagem JavaScript

JS

Sebenta de Estudo



PARTE C · CAPÍTULO 16

Introdução ao JavaScript

O JavaScript dá **comportamento** à página: reage a cliques, calcula, decide e altera o que o utilizador vê.

Neste capítulo vais aprender a:

- ligar um ficheiro JavaScript à página;
- guardar informação em variáveis;
- usar operadores, condições e funções;
- reagir a um clique com um evento.

16.1 Onde se escreve

Tal como o CSS, o JavaScript escreve-se num **ficheiro externo** (`js/script.js`) que se liga no **fim do** `<body>`:

`<script>` — JAVASCRIPT

Função	Liga a página a um **ficheiro JavaScript**.
Sintaxe	`<script src="js/script.js"></script>`
Quando usar	No fim do `<body>`, depois de todo o conteúdo.

EXEMPLO

index.html — fim do body

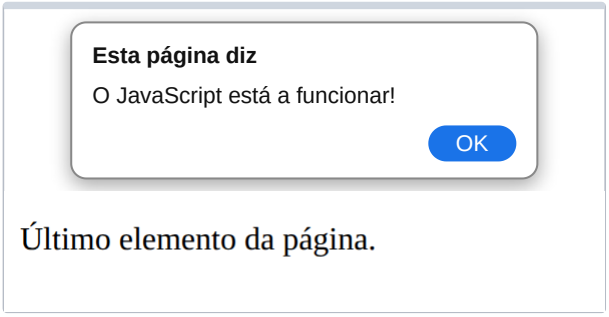
```
    ...
    <p>Último elemento da página.</p>

    <script src="js/script.js"></script>
</body>
</html>
```

js/script.js

```
alert("O JavaScript está a funcionar!");
```

RESULTADO ESPERADO



A janela «Esta página diz» é o `alert()`, simulado aqui tal como o navegador o mostra.

✅ **BOA PRÁTICA**

Coloca o `<script>` **depois** do conteúdo: assim, quando o código corre, os elementos já existem.

❌ **ERRO FREQUENTE**

Caminho errado em `src` (ou esquecer o `</script>`): o código não corre e não aparece nenhum aviso na página.

16.2 A consola: o teu melhor amigo

Carrega em `F12` e abre o separador **Consola**. Aí aparecem as mensagens de `console.log()` e os **erros** do teu código. É a melhor maneira de testar e de descobrir o que está mal.

`console.log()` — CONSOLA

Função	Escreve uma mensagem na **consola** (não aparece na página).
Sintaxe	`console.log(valor);`
Quando usar	Para testar código e ver valores.

EXEMPLO

```
// isto é um comentário
console.log("Olá, Web!");
console.log(2 + 3);
```

RESULTADO ESPERADO

```
CONSOLA (F12)
> Olá, Web!
> 5
```

> 💡 **DICA**
> Usa `//` para comentários de uma linha e `/* … */` para várias linhas.

16.3 Variáveis e tipos de dados

`let / const` — VARIÁVEIS

Função	Uma **variável** é uma «caixa» com nome onde se guarda um valor.
Sintaxe	`let nome = valor; const nome = valor;`
Quando usar	Sempre que precisas de guardar um dado para usar mais tarde.

EXEMPLO

```
let nome = "Ana";
const idade = 17;

console.log("Olá, " + nome);
console.log("Tens " + idade + " anos");

nome = "Rui";
console.log(nome);
```

RESULTADO ESPERADO

```
CONSOLA (F12)
> Olá, Ana
> Tens 17 anos
> Rui
```

> ✅ **BOA PRÁTICA**
> Usa `const` por omissão e `let` só se o valor tiver de mudar. Escolhe nomes claros (`idade`, `total`).

> ❌ **ERRO FREQUENTE**
> Tentar mudar uma `const` (`idade = 18;`) dá erro. Os nomes distinguem maiúsculas de minúsculas: `nome` e `Nome` são diferentes.

Tipo	O que guarda	Exemplo
String (texto)	Texto, entre aspas	`"Olá" ou 'Olá'`
Number (número)	Inteiros e decimais (com ponto)	`42 3.14`
Boolean	Verdadeiro ou falso	`true false`

16.4 Operadores

Tipo	Operadores	Exemplo	Resultado
Aritméticos	`+ - * / %`	`7 % 3`	1 (resto da divisão)
Comparação	`=== !== < > <= >=`	`5 === 5`	true
Lógicos	`&& (e) || (ou) ! (não)`	`5 > 2 && 1 > 3`	false
Juntar texto	`+`	`"Olá " + "Ana"`	Olá Ana

Operadores — OPERADORES

Função Fazem **contas** e **comparações**.

Sintaxe `a + b a === b a > b && b > c`

Quando usar Cálculos, comparações e condições.

EXEMPLO

```
let a = 10;
let b = 3;

console.log(a + b);
console.log(a * b);
console.log(a % b);
console.log(a > b);
console.log(a === 10 && b < 2);
```

RESULTADO ESPERADO

```
CONSOLA (F12)
> 13
> 30
> 1
> true
> false
```

❌ **ERRO FREQUENTE**

Usar `=` (atribuir) em vez de `===` (comparar). `a = 5` guarda 5; `a === 5` pergunta se é 5.

16.5 Condições: if / else

if / else — CONDIÇÕES

Função Executa código **só se** uma condição for verdadeira; caso contrário, executa outro.

Sintaxe `if (condição) { … } else { … }`

Quando usar Sempre que o programa tem de tomar uma decisão.

EXEMPLO

```
let nota = 12;

if (nota >= 10) {
    console.log("Aprovado");
} else {
    console.log("Reprovado");
}
```

RESULTADO ESPERADO

```
CONSOLA (F12)
> Aprovado
```

💡 **DICA**

Experimenta mudar `nota` para 8 e vê a mensagem mudar.

❌ **ERRO FREQUENTE**

Esquecer as chavetas `{ }` ou os parênteses à volta da condição.

16.6 Funções

function — FUNÇÕES

Função	Um **bloco de código com nome** que podes executar (chamar) sempre que quiseres.
Sintaxe	`function nome(parametro) { … }`
Quando usar	Para não repetires código e para reagir a eventos.

EXEMPLO

```
function saudar(nome) {
    return "Olá, " + nome + "!";
}

console.log(saudar("Ana"));
console.log(saudar("Rui"));
```

RESULTADO ESPERADO

```
CONSOLA (F12)
> Olá, Ana!
> Olá, Rui!
```

> ⚠️ **ATENÇÃO**
> Definir a função **não a executa**. Só corre quando a **chamas**: `saudar("Ana");` (com parênteses).

16.7 Eventos: reagir a um clique

Um **evento** é algo que acontece na página: um clique, uma tecla, um rato a passar. Com `onclick` dizemos **qual função** executar quando o utilizador clica.

onclick — EVENTOS

Função	Executa uma função quando o utilizador **clica** no elemento.
Sintaxe	`<button onclick="nomeDaFuncao()">`
Quando usar	Botões e outros elementos clicáveis.

EXEMPLO

HTML

```
<button onclick="mostrarMensagem()">Clica
aqui</button>
```

JavaScript

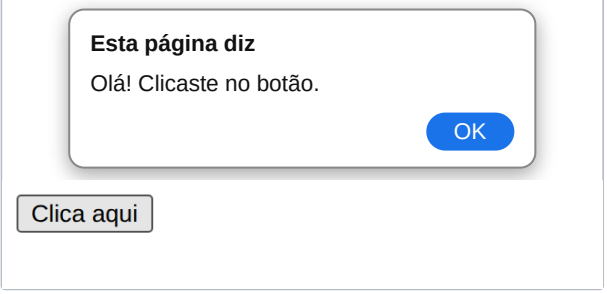
```
function mostrarMensagem() {
    alert("Olá! Clicaste no botão.");
}
```

RESULTADO ESPERADO

ANTES DO CLIQUE



DEPOIS DO CLIQUE



> 💡 **DICA**
> No capítulo seguinte vais aprender uma forma mais moderna: `addEventListener`.

> ❌ **ERRO FREQUENTE**
> Escrever `onclick="mostrarMensagem"` sem os parênteses: a função não é chamada.

🧪 EXPERIMENTA TU 20 — A TUA PRIMEIRA FUNÇÃO

Cria uma função `saudar()` que mostra um `alert` com o teu nome. Liga-a a dois botões diferentes. Depois faz com que o alert mostre a tua idade, guardada numa variável.

🧩 COMPLETA O CÓDIGO — D47 (5 MIN)

Completa para mostrar «Bem-vindo!» ao clicar no botão.

HTML

```
<button ___="saudar()">Entrar</button>
```

JavaScript

```
___ saudar() {
    ___("Bem-vindo!");
}
```

🎯 QUAL SERÁ O RESULTADO? — D48 (5 MIN)

O que aparece na consola?

```
let x = 5;
x = x + 2;
console.log(x * 2);
```

🔍 ENCONTRA O ERRO — D49 (5 MIN)

A mensagem devia dizer «Aprovado», mas diz sempre «Reprovado». Onde está o erro?

```
let nota = 15;
if (nota = 10) {
    console.log("Aprovado");
} else {
    console.log("Reprovado");
}
```

🛠️ CORRIGE O CÓDIGO — D50 (5 MIN)

O botão não faz nada ao clicar. Corrige.

HTML

```
<button onclick="dizerOla">Olá</button>
```

JavaScript

```
function dizerOla() {
    alert("Olá!");
}
```

✏️ MODIFICA O EXEMPLO — D51 (10 MIN)

Modifica `mostrarMensagem()` para receber o **nome** como parâmetro e mostrar «Olá, Ana!» (ou outro nome à tua escolha).

📋 Ficha rápida — JavaScript básico

Conceito	Função	Exemplo
`<script src>`	Ligar o ficheiro	`<script src="js/script.js">`
`console.log()`	Mostrar na consola	`console.log(x);`
`let / const`	Variáveis	`const n = 5;`
`if / else`	Decisões	`if (x > 1) { … }`
`function`	Função	`function f() { }`
`onclick`	Reagir ao clique	`onclick="f()"`
`alert()`	Janela de aviso	`alert("Olá");`

PARTE C · CAPÍTULO 17

JavaScript e DOM

O **DOM** é a «versão viva» da tua página: o JavaScript consegue **encontrar** e **alterar** qualquer elemento.

Neste capítulo vais aprender a:

- selecionar elementos com `getElementById` e `querySelector`;
- alterar texto, imagens e estilos;
- mostrar e esconder elementos com classes;
- reagir a cliques com `addEventListener`.

17.1 O que é o DOM?

Quando o navegador lê o HTML, constrói uma **árvore de elementos** chamada **DOM** (*Document Object Model*). O JavaScript navega nessa árvore e pode alterá-la: a página muda logo, sem recarregar.

```
document
└── html
    ├── head
    │   └── title
    └── body
        ├── h1   id="titulo"
        ├── p    id="texto"
        └── button  id="botao"
```

Repara nos `id`: são o «nome» que damos aos elementos para o JavaScript os encontrar.

17.2 Encontrar elementos

`getElementById()` — SELECIONAR

Função	Encontra o elemento com um determinado **id**.
Sintaxe	`document.getElementById("id")`
Quando usar	Quando o elemento tem `id`.

EXEMPLO

HTML

```html
<h1 id="titulo">Bem-vindo!</h1>
<p class="aviso">Texto de aviso.</p>
<button id="botao">Clica</button>
```

JavaScript

```javascript
const titulo = document.getElementById("titulo");
console.log(titulo.textContent);
```

RESULTADO ESPERADO

```
CONSOLA (F12)
> Bem-vindo!
```

O resultado é o texto do `<h1>`: o JavaScript «encontrou» o elemento.

querySelector() — SELECIONAR

Função	Encontra o **primeiro** elemento que corresponda a um **seletor CSS**.
Sintaxe	`document.querySelector("seletor")`
Quando usar	É a forma mais flexível: usa os mesmos seletores do CSS.

EXEMPLO

HTML

```html
<h1 id="titulo">Bem-vindo!</h1>
<p class="aviso">Texto de aviso.</p>
<button id="botao">Clica</button>
```

JavaScript

```javascript
const titulo = document.querySelector("#titulo");
const aviso  = document.querySelector(".aviso");
const botao  = document.querySelector("#botao");
```

RESULTADO ESPERADO

Sem resultado visível: guarda três elementos em variáveis para usar a seguir.

✅ **BOA PRÁTICA**

Não te esqueças do `#` (id) ou do `.` (classe) dentro das aspas.

❌ **ERRO FREQUENTE**

`querySelector("titulo")` procura uma **tag** chamada «titulo». Faltou o `#`.

17.3 Alterar o conteúdo

textContent — ALTERAR

Função	Lê ou **altera o texto** de um elemento.
Sintaxe	`elemento.textContent = "novo texto";`
Quando usar	Mostrar resultados e mensagens na página.

EXEMPLO

HTML

```html
<p id="msg">Texto original.</p>
<button id="botao">Mudar texto</button>
```

JavaScript

```javascript
const msg = document.querySelector("#msg");
const botao = document.querySelector("#botao");

botao.addEventListener("click", function () {
    msg.textContent = "O texto mudou!";
});
```

RESULTADO ESPERADO

ANTES DO CLIQUE

Texto original.

Mudar texto

DEPOIS DO CLIQUE

O texto mudou!

Mudar texto

💡 **DICA**

`addEventListener("click", função)` diz: «quando clicarem neste elemento, executa esta função».

src ALTERAR

Função	Muda a **imagem** de um `<img>`, alterando o atributo `src`.
Sintaxe	`imagem.src = "imagens/outra.jpg";`
Quando usar	Galerias e trocas de imagem.

EXEMPLO

HTML

```
<img id="foto" src="imagens/paisagem.jpg"
alt="Paisagem" width="160">
<br>
<button id="botao">Mudar imagem</button>
```

JavaScript

```
const foto = document.querySelector("#foto");
const botao = document.querySelector("#botao");

botao.addEventListener("click", function () {
    foto.src = "imagens/praia.jpg";
    foto.alt = "Praia";
});
```

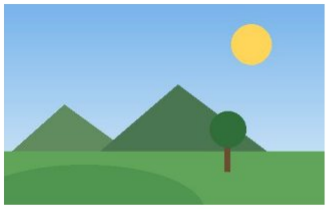
RESULTADO ESPERADO

ANTES DO CLIQUE



Mudar imagem

DEPOIS DO CLIQUE



Mudar imagem

17.4 Alterar o aspeto

style ALTERAR

Função	Altera **diretamente** uma propriedade CSS do elemento.
Sintaxe	`elemento.style.propriedade = "valor";`
Quando usar	Alterações pontuais. Em JavaScript, `background-color` escreve-se `backgroundColor`.

EXEMPLO

HTML

```
<p id="msg">Vou mudar de aspeto.</p>
<button id="botao">Destacar</button>
```

JavaScript

```
const msg = document.querySelector("#msg");
const botao = document.querySelector("#botao");

botao.addEventListener("click", function () {
    msg.style.color = "white";
    msg.style.backgroundColor = "#0b4f8a";
});
```

RESULTADO ESPERADO

ANTES

Vou mudar de aspeto.

Destacar

DEPOIS

Vou mudar de aspeto.

Destacar

> ⚠️ **ATENÇÃO**
> Nomes com hífen em CSS (`font-size`) passam a **camelCase** em JavaScript (`fontSize`).

classList

ALTERAR

Função	Adiciona, remove ou **alterna uma classe** CSS. É a forma mais limpa de mudar o aspeto (o CSS fica no CSS).
Sintaxe	`elemento.classList.toggle("classe");`
Quando usar	Mostrar/esconder, modo escuro, menus que abrem e fecham.

EXEMPLO

HTML

```html
<p id="msg" class="escondido">Agora vês-me!</p>
<button id="botao">Mostrar / esconder</button>
```

JavaScript

```javascript
const msg = document.querySelector("#msg");
const botao = document.querySelector("#botao");

botao.addEventListener("click", function () {
    msg.classList.toggle("escondido");
});
```

RESULTADO ESPERADO

INÍCIO (ESCONDIDO)

Agora vês-me!

Mostrar / esconder

1.º CLIQUE

Agora vês-me!

Mostrar / esconder

2.º CLIQUE

Agora vês-me!

Mostrar / esconder

> ✅ **BOA PRÁTICA**
> Cria a classe no CSS (`.escondido { display: none; }`) e usa o JavaScript só para a **ligar e desligar**.

💻 CÓDIGO — A CLASSE USADA NO EXEMPLO (CSS)

```css
.escondido {
    display: none;
}
```

addEventListener() EVENTOS

Função	Associa uma função a um **evento** de um elemento (clique, teclado...). Substitui o `onclick` no HTML.
Sintaxe	`elemento.addEventListener("click", function () { … });`
Quando usar	Em quase todos os projetos: o HTML fica limpo e o JavaScript fica no ficheiro `.js`.

EXEMPLO

HTML

```
<p>Cliques: <b id="contador">0</b></p>
<button id="botao">Clica</button>
```

JavaScript

```
let cliques = 0;
const botao = document.querySelector("#botao");
const contador =
document.querySelector("#contador");

botao.addEventListener("click", function () {
    cliques = cliques + 1;
    contador.textContent = cliques;
});
```

RESULTADO ESPERADO

INÍCIO

Cliques: **0**

Clica

DEPOIS DE 1 CLIQUE

Cliques: **0**

Clica

DEPOIS DE 2 CLIQUES

Cliques: **0**

Clica

> ✅ **BOA PRÁTICA**
> Mantém o comportamento no ficheiro `.js`: um HTML sem `onclick` é mais limpo.

> ❌ **ERRO FREQUENTE**
> Se a consola mostrar `Cannot read properties of null`, o JavaScript **não encontrou o elemento**: confirma o `id`, o `#`, e que o `<script>` está **no fim do body**.

🏗️ Projeto progressivo · Etapa 14 de 14

O QUE ACRESCENTAS

index.html

```
<button id="btn-tema">Modo escuro</button>   <!-- no header -->

<button id="btn-mais">Ler mais</button>
<p id="mais" class="escondido">Fundado em 2018, …
</p>

<script src="js/script.js"></script>   <!-- antes de </body> -->
```

css/estilo.css — acrescenta

```
/* ===== Interatividade (JavaScript) ===== */
.escondido { display: none; }

body.escuro { background: #14171c; color: #e5e7eb;
}
body.escuro .cartao,
body.escuro table { background: #222831; }
body.escuro h2,
body.escuro .cartao h3 { color: #7db7ee; }
body.escuro td { border-color: #3a4350; }
```

js/script.js

```
// ===== Modo escuro =====
const botaoTema = document.querySelector("#btn-
tema");

botaoTema.addEventListener("click", function () {
    document.body.classList.toggle("escuro");
});

// ===== Ler mais / Ler menos =====
const botaoMais = document.querySelector("#btn-
mais");
const textoMais = document.querySelector("#mais");

botaoMais.addEventListener("click", function () {
    textoMais.classList.toggle("escondido");

    if (textoMais.classList.contains("escondido"))
{
        botaoMais.textContent = "Ler mais";
    } else {
        botaoMais.textContent = "Ler menos";
    }
});
```

RESULTADO ESPERADO



*Resultado depois de clicar nos dois botões. Dois botões: um alterna o **modo escuro** (classe `escuro` no `body`) e outro mostra ou esconde o texto **«Ler mais»** (classe `escondido`).*

🧪 EXPERIMENTA TU 21 — MODO ESCURO

Acrescenta ao teu projeto o botão «Modo escuro». Depois cria a classe `.escuro` com outras cores (fundo, texto, cartões) e experimenta acrescentar mais uma regra, por exemplo, para os `h2`.

🧩 COMPLETA O CÓDIGO — D52 (5 MIN)

Completa para trocar o texto do parágrafo `#msg` por «Olá!».

```
const p = document.___("#msg");
p.___ = "Olá!";
```

🔍 ENCONTRA O ERRO — D53 (5 MIN)

O JavaScript dá o erro `Cannot read properties of null`. Porquê?

```
<body>
    <script src="js/script.js"></script>
    <button id="botao">Clica</button>
</body>
```

🎯 QUAL SERÁ O RESULTADO? — D54 (5 MIN)

Se o utilizador clicar **duas vezes** no botão, o parágrafo fica visível ou escondido?

HTML

```
<p id="p" class="escondido">Olá</p>
<button id="b">Alternar</button>
```

JavaScript

```
const p = document.querySelector("#p");
document.querySelector("#b").addEventListener("click", function () {
    p.classList.toggle("escondido");
});
```

🛠️ CORRIGE O CÓDIGO — D55 (5 MIN)

O JavaScript não encontra o título. Corrige.

HTML

```
<h1 id="titulo">Olá</h1>
```

JavaScript

```
const t = document.querySelector("titulo");
t.textContent = "Adeus";
```

✏️ MODIFICA O EXEMPLO — D56 (15 MIN)

Modifica o exemplo da imagem para que o botão **alterne** entre a paisagem e a praia (dica: usa um `if` que verifica qual é o `src` atual, ou uma variável).

📋 Ficha rápida — JavaScript e DOM

Elemento	Função	Exemplo
`getElementById()`	Elemento por id	`document.getElementById("x")`
`querySelector()`	Elemento por seletor	`document.querySelector(".x")`
`textContent`	Alterar texto	`el.textContent = "…";`
`src`	Alterar imagem	`img.src = "a.jpg";`
`style`	Alterar CSS	`el.style.color = "red";`
`classList.toggle()`	Ligar/desligar classe	`el.classList.toggle("x");`
`addEventListener()`	Reagir a eventos	`el.addEventListener("click", f);`

PARTE D

Prática

Projeto progressivo, erros, boas práticas e projetos finais

PARTE D · CAPÍTULO 18

O projeto progressivo: visão geral

Ao longo da sebenta construíste, passo a passo, o **site do Clube de Fotografia da Escola**. Aqui tens o percurso completo, para rever e para o usares como modelo.

Neste capítulo vais aprender a:

- rever as 14 etapas do projeto;
- perceber como cada capítulo contribuiu para o site final;
- conhecer a estrutura final de ficheiros.

18.1 O percurso em 14 etapas

Etapa	Cap.	O que acrescentámos	Tecnologias	Ficheiros
1	2	Página base	`h1` e `p`	`index.html`
2	3	Títulos, parágrafos e destaques	`h2`, `p`, `strong`	`index.html`
3	3	Listas	`ul` e `ol`	`index.html`
4	3	Hiperligações	`a`: internas, externas e e-mail	`index.html`
5	3	Imagens	`img`, caminhos relativos	`index.html`, `imagens/`
6	4	Tabela de horários	`table`, `thead`, `tbody`	`index.html`
7	5	Formulário de contacto	`form`, `input`, `select`…	`contactos.html`
8	6	Estrutura semântica	`header`, `nav`, `main`, `section`, `footer`	`index.html`
9	9	CSS base	ficheiro externo, cores, letra, espaços	`css/estilo.css`
10	10-11	Menu de navegação	seletores descendentes, `:hover`	`css/estilo.css`
11	13	Flexbox	menu e cartões	`css/estilo.css`
12	14	Galeria com Grid	`display: grid`	`galeria.html`, `css/estilo.css`
13	15	Responsivo	`viewport`, `@media`	`todas as páginas`, `css/estilo.css`
14	16-17	JavaScript	modo escuro, «Ler mais»	`js/script.js`

18.2 Como o site foi evoluindo

Etapa 2

Clube de Fotografia da Escola

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.

O que fazemos

Todas as semanas realizamos atividades para todos os níveis:

Etapa 5

Clube de Fotografia da Escola

Início | Galeria | Contactos

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



Etapa 6

Clube de Fotografia da Escola

Início | Galeria | Contactos

Olá Mundo! Bem-vindo ao **Clube de Fotografia**.

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



Etapa 9

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

- Início
- Galeria
- Contactos

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a

Etapa 11

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

Início Galeria Contactos

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.

Etapa 14

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

Modo escuro

Início

Galeria

Contactos

Quem somos

O Clube de Fotografia reúne alunos e professores

*Seis momentos do projeto: só HTML (etapas 2, 5 e 6), depois CSS (9), Flexbox (11) e, por fim, JavaScript (14). O **conteúdo** quase não mudou; o que mudou foi a **estrutura** e o **aspeto**.*

18.3 Estrutura final de ficheiros

```
clube_fotografia/
│
├── index.html          (etapas 1-8, 11, 13, 14)
├── galeria.html        (etapa 12)
├── contactos.html      (etapa 7)
│
├── css/
│   └── estilo.css      (etapas 9-14)
│
├── js/
│   └── script.js       (etapa 14)
│
└── imagens/
    ├── escola.jpg
    ├── paisagem.jpg    praia.jpg    montanha.jpg
    └── cidade.jpg      ponte.jpg    floresta.jpg
```

> 💡 **DICA**
> O **código completo** de todas as páginas está no **Anexo B**, com imagens do resultado em computador e telemóvel. Se ficares preso numa etapa, compara o teu código com o do anexo.

18.4 O que aprendeste, em resumo

	HTML	CSS	JavaScript
Função	Estrutura e conteúdo	Aparência e disposição	Comportamento
Usaste	Títulos, listas, ligações, imagens, tabela, formulário, elementos semânticos	Seletores, cor, texto, box model, Flexbox, Grid, media queries	Variáveis, funções, eventos, DOM
No projeto	As três páginas	`estilo.css`	`script.js`

PARTE D · CAPÍTULO 19

Erros frequentes e como os resolver

Todos os programadores erram: a diferença está em **encontrar o erro depressa**. Estes são os erros que mais aparecem nas aulas.

Neste capítulo vais aprender a:

- reconhecer os erros mais comuns em HTML, CSS e JavaScript;
- seguir um método simples para descobrir o que está mal.

> 🔎 **MÉTODO EM 5 PASSOS**
>
> 1. **Guardaste** o ficheiro? (`Ctrl`+`S`)
> 2. Abre a **consola** (`F12`): há mensagens de erro?
> 3. Confere **tags, aspas, chavetas e ponto e vírgula** (a indentação ajuda muito).
> 4. **Comenta** partes do código até o erro desaparecer.
> 5. Faz um **exemplo mínimo** só com o problema; é mais fácil de perceber (e de pedir ajuda).

HTML

1. Esquecer de fechar uma tag

Sintoma: o resto da página fica todo em negrito (ou com um aspeto estranho).

❌ **ERRADO**

```
<p>Isto é <strong>importante.</p>
<p>E isto?</p>
```

✅ **CERTO**

```
<p>Isto é <strong>importante</strong>.</p>
<p>E isto?</p>
```

> 💡 **DICA**
>
> Escreve sempre a tag de fecho logo a seguir à de abertura e só depois preenche o conteúdo.

2. Caminho da imagem errado

Sintoma: aparece o ícone de imagem partida e o texto alternativo.

❌ **ERRADO**

```
<!-- a imagem está em imagens/escola.jpg -->
<img src="escola.jpg" alt="Escola">
```

✅ **CERTO**

```
<img src="imagens/escola.jpg" alt="Escola">
```

> 💡 **DICA**
>
> Numa página dentro de `paginas/`, o caminho passa a ser `../imagens/escola.jpg`.

3. Aspas em falta ou desiguais

Sintoma: a ligação não funciona e o resto da linha desaparece.

❌ **ERRADO**

```
<a href="galeria.html>Galeria</a>
```

✅ **CERTO**

```
<a href="galeria.html">Galeria</a>
```

4. Ids repetidos

Sintoma: o JavaScript só encontra o primeiro elemento e o CSS por `#id` só afeta um.

❌ **ERRADO**

```
<p id="aviso">Um</p>
<p id="aviso">Dois</p>
```

✅ **CERTO**

```
<p class="aviso">Um</p>
<p class="aviso">Dois</p>
```

> 💡 **DICA**
> O `id` é **único**. Para vários elementos, usa `class`.

5. Caminhos absolutos do teu computador

Sintoma: a imagem aparece no teu computador, mas **desaparece** noutro computador ou quando publicas o site.

❌ **ERRADO**

```
<img src="C:\Users\Ana\Desktop\escola.jpg"
alt="Escola">
```

✅ **CERTO**

```
<img src="imagens/escola.jpg" alt="Escola">
```

6. Maiúsculas e minúsculas nos nomes de ficheiros

Sintoma: funciona no Windows, mas falha quando o site é publicado num servidor.

❌ **ERRADO**

```
<!-- o ficheiro chama-se escola.jpg -->
<img src="imagens/Escola.JPG" alt="Escola">
```

✅ **CERTO**

```
<img src="imagens/escola.jpg" alt="Escola">
```

> 💡 **DICA**
> Usa **sempre minúsculas**, sem espaços nem acentos: `galeria-verao.jpg`.

CSS

7. Esquecer o ponto e vírgula

Sintoma: só uma das duas propriedades é aplicada.

❌ **ERRADO**

```
p {
    color: red
    font-size: 20px;
}
```

✅ **CERTO**

```
p {
    color: red;
    font-size: 20px;
}
```

8. CSS fora das chavetas

Sintoma: a regra é ignorada e não aparece nenhum erro.

❌ **ERRADO**

```
p {
    color: red;
}
font-size: 20px;
```

✅ **CERTO**

```
p {
    color: red;
    font-size: 20px;
}
```

9. Confundir classe com id

Sintoma: o estilo não é aplicado.

❌ **ERRADO**

```
<div class="cartao">…</div>

/* CSS */
#cartao { background: yellow; }
```

✅ **CERTO**

```
<div class="cartao">…</div>

/* CSS */
.cartao { background: yellow; }
```

> 💡 **DICA**
> Classe → ponto (`.cartao`). Id → cardinal (`#cartao`).

10. Ficheiro CSS mal ligado

Sintoma: a página aparece sem qualquer estilo.

❌ **ERRADO**

```
<link rel="stylesheet" href="estilo.css">
<!-- o ficheiro está em css/estilo.css -->
```

✅ **CERTO**

```
<link rel="stylesheet" href="css/estilo.css">
```

> 💡 **DICA**
> Abre o **separador Rede** (F12 → Network): um ficheiro a vermelho é um caminho errado.

JavaScript

11. Script antes do elemento

Sintoma: consola: `Cannot read properties of null`.

❌ **ERRADO (SCRIPT ANTES)**

```
<script src="js/script.js"></script>
<button id="botao">Clica</button>
```

✅ **CERTO (SCRIPT NO FIM)**

```
<button id="botao">Clica</button>
<script src="js/script.js"></script>
```

12. Usar = em vez de ===

Sintoma: a condição é sempre verdadeira.

❌ **ERRADO**

```
if (nota = 10) {
    …
}
```

✅ **CERTO**

```
if (nota === 10) {
    …
}
```

> 💡 **DICA**
> `=` guarda um valor; `===` compara dois valores.

13. Não guardar antes de atualizar

Sintoma: fizeste a alteração, mas a página «não muda».

❌ **SINTOMA**

```
// alteraste o código, mas o ficheiro está com um ponto (●) no separador
```

✅ **SOLUÇÃO**

```
Guarda com Ctrl + S (ou ativa «Guardar automaticamente»)
```

💡 **DICA**

Com o **Live Server**, a página atualiza sozinha assim que guardas.

🔍 **ENCONTRA O ERRO — D57 (10 MIN)**

A página aparece sem estilo e sem mensagens de erro. Quantos problemas encontras?

HTML

```html
<link rel="stylesheet" href="estilo.css">
<h1>Olá</h1>
```

CSS

```css
h1 {
    color: blue
    font-size: 30px;
}
```

🔍 **ENCONTRA O ERRO — D58 (5 MIN)**

A imagem não aparece. A pasta é `meu_site/` com `imagens/foto.jpg`. Qual o problema?

```html
<img src="C:\Users\Rui\foto.jpg" alt="Foto">
```

PARTE D · CAPÍTULO 20

Boas práticas

Um código que funciona é o primeiro passo; um código **organizado e legível** é o que faz de ti um bom profissional.

Neste capítulo vais aprender a:

- organizar ficheiros e pastas;
- escrever HTML, CSS e JavaScript legíveis;
- criar páginas mais acessíveis.

20.1 Ficheiros e pastas

	Faz	Evita
Nomes	minúsculas, sem espaços nem acentos: `galeria-verao.jpg`	`Galeria Verão (2).JPG`
Pastas	`css/`, `js/`, `imagens/`	tudo misturado na mesma pasta
Página inicial	`index.html`	`pagina1_final_v2.html`
Cópias	guarda cópias de segurança do projeto	trabalhar só numa pen ou numa pasta temporária

20.2 HTML

- Começa sempre com `<!DOCTYPE html>`, `lang="pt"`, `charset` e `viewport`.
- Usa **um só** `<h1>` por página e não saltes níveis de títulos.
- Prefere **elementos semânticos** (`header`, `nav`, `main`, `footer`) a `div` em tudo.
- Indenta com 4 espaços e deixa linhas em branco entre blocos.
- Escreve texto **alternativo** descritivo em todas as imagens, e uma `label` ligada a cada campo de formulário.
- Usa tabelas para **dados**, nunca para desenhar a página.

❌ **LIGAÇÃO VAGA**

```
<a href="horario.html">Clica aqui</a>
```

✅ **LIGAÇÃO DESCRITIVA**

```
<a href="horario.html">Consulta o horário das sessões</a>
```

20.3 CSS

- Usa **um ficheiro externo** ligado a todas as páginas.
- Começa com `* { box-sizing: border-box; }` e `img { max-width: 100%; height: auto; }`.
- Prefere **classes** a ids para estilos; dá **nomes que descrevem a função** (`.cartao`, `.menu`), não o aspeto (`.azul-grande`).
- Organiza o ficheiro por secções, com comentários (`/* ===== Menu ===== */`).
- Escreve **uma declaração por linha** e não te esqueças do `;`.
- Não repitas regras: agrupa seletores com vírgulas.
- Pensa primeiro no **telemóvel** e acrescenta `@media` para ecrãs maiores (ou vice-versa, mas testa nos dois).

❌ **NOME QUE DESCREVE O ASPETO**

```
.azul-grande {
    color: blue;
    font-size: 30px;
}
```

✅ **NOME QUE DESCREVE A FUNÇÃO**

```
.titulo-destaque {
    color: var(--cor-principal, #0b4f8a);
    font-size: 30px;
}
```

20.4 JavaScript

- Liga o `<script>` **no fim do body**.
- Usa `const` por omissão e `let` quando o valor muda; dá nomes claros às variáveis.
- Mantém o comportamento em `script.js`: prefere `addEventListener` a `onclick` no HTML.
- Muda o aspeto **com classes CSS** (`classList.toggle`) em vez de escreveres estilos em JavaScript.
- Usa a **consola** (`console.log`) para investigar e apaga os testes no fim.

20.5 Acessibilidade em poucas linhas

Pergunta	Como verificar
Consigo ler o texto?	Contraste forte entre letra e fundo; tamanho mínimo de 16 px no texto corrido.
Consigo navegar sem rato?	Usa `Tab`: consegues chegar a todas as ligações e botões?
As imagens fazem sentido sem serem vistas?	Todas têm `alt` descritivo (ou `alt=""` se forem só decoração).
A página funciona no telemóvel?	Testa com a barra de dispositivos (F12 → Ctrl+Shift+M).

20.6 Lista de verificação antes de entregar

	Verifica
☐	Todas as ligações abrem a página certa (testa uma a uma).
☐	Todas as imagens aparecem e têm `alt`.
☐	Não há caminhos absolutos do computador.
☐	A consola (F12) não mostra erros.
☐	A página funciona em ecrã de computador e de telemóvel.
☐	O código está indentado e comentado onde é útil.
☐	Os ficheiros e pastas têm nomes em minúsculas, sem espaços.

PARTE D · CAPÍTULO 21

Mini-projetos

Seis projetos curtos, de dificuldade crescente, para consolidar o que aprendeste. Cada um usa o que estudaste até ao momento.

Neste capítulo vais aprender a:

- aplicar HTML, CSS e JavaScript em projetos pequenos;
- construir páginas do zero, sem seguir passo a passo.

O **resultado esperado** é apenas uma sugestão: usa o teu tema, as tuas cores e as tuas imagens. Se ficares preso, volta ao capítulo indicado.

Mini-projeto 1 · Página pessoal

SÓ HTML · CAPS. 2 E 3

Objetivo Criar uma página sobre ti, só com HTML.

Ficheiros `pessoal/index.html`, `pessoal/imagens/`

REQUISITOS

- Título principal e pelo menos dois subtítulos.
- Uma imagem (com `alt`).
- Uma lista dos teus interesses.
- Uma ligação externa (novo separador) e uma ligação de e-mail.

PASSOS SUGERIDOS

1. Cria a pasta e o ficheiro `index.html` com a estrutura base.
2. Escreve os títulos e os parágrafos.
3. Acrescenta a imagem, a lista e as ligações.
4. Abre no navegador e confirma que tudo aparece.

RESULTADO ESPERADO



Ana Silva

Sobre mim

Tenho 17 anos e estudo Informática na Escola Profissional.

Os meus interesses

- Fotografia
- Música
- Futebol

Contacto: ana@exemplo.pt

Gosto de ler a Wikipédia.

> 🚀 **EXTENSÃO**
> Acrescenta uma tabela com o teu horário semanal.

Mini-projeto 2 · Website de três páginas

HTML · CAPS. 3, 5 E 6

Objetivo	Ligar três páginas com o mesmo menu.
Ficheiros	`escola/index.html`, `cursos.html`, `contactos.html`

REQUISITOS

- Três ficheiros HTML com **o mesmo menu**.
- Estrutura semântica (`header`, `nav`, `main`, `footer`).
- Uma página com um formulário de contacto.
- Ligações internas **sempre a funcionar** nos dois sentidos.

PASSOS SUGERIDOS

1. Cria `index.html` completo e testa-o.
2. Copia-o para `cursos.html` e `contactos.html` e muda o conteúdo.
3. Confirma cada ligação do menu, em cada página.
4. Acrescenta o formulário em `contactos.html`.

RESULTADO ESPERADO

Escola de Música

Início | Cursos | Contactos

Bem-vindos!

Aprende a tocar o teu instrumento preferido com professores experientes.

- Guitarra
- Piano
- Bateria

© 2026 Escola de Música

> 🚀 **EXTENSÃO**
> Acrescenta uma página com uma tabela de horários e uma quarta ligação no menu.

Mini-projeto 3 · Página com CSS

HTML + CSS · CAPS. 9 A 12

Objetivo Dar aspeto profissional à página pessoal do mini-projeto 1.

Ficheiros `pessoal/index.html`, `pessoal/css/estilo.css`

REQUISITOS

- Ficheiro CSS **externo**, ligado com `<link>`.
- Cores, tipo de letra e espaçamento coerentes.
- Pelo menos uma classe e um seletor descendente.
- Uso do box model: `padding`, `margin` e `border-radius`.

PASSOS SUGERIDOS

1. Cria a pasta `css/` e o ficheiro `estilo.css` e liga-o.
2. Define `body`, títulos e imagem.
3. Estiliza a lista e as ligações.
4. Centra o conteúdo com `max-width` e `margin: 0 auto`.

RESULTADO ESPERADO



🚀 EXTENSÃO

Acrescenta um efeito `:hover` às ligações e uma sombra (`box-shadow`) à lista.

Mini-projeto 4 · Galeria responsiva

GRID + RESPONSIVO · CAPS. 14 E 15

Objetivo Mostrar fotografias numa grelha que se adapta ao ecrã.

Ficheiros `galeria/index.html`, `galeria/css/estilo.css`, `galeria/imagens/`

REQUISITOS

- Seis ou mais imagens com `alt`.
- Grelha de **3 colunas** em computador, **2** em tablet e **1** em telemóvel.
- Imagens flexíveis (`width: 100%`).
- Meta `viewport` no `<head>`.

PASSOS SUGERIDOS

1. Escolhe as imagens e coloca-as na pasta `imagens/`.
2. Cria o `div.galeria` e as imagens.
3. Aplica `display: grid` com `repeat(3, 1fr)`.
4. Acrescenta as duas *media queries* e testa com o F12.

RESULTADO ESPERADO



🚀 EXTENSÃO

Acrescenta uma legenda a cada foto usando `<figure>` e `<figcaption>`.

Mini-projeto 5 · Website responsivo

FLEXBOX + RESPONSIVO · CAPS. 6, 13 E 15

Objetivo Construir a página de um negócio local (ou de um clube) que funcione em qualquer ecrã.

Ficheiros `negocio/index.html`, `negocio/css/estilo.css`

REQUISITOS

- Estrutura semântica completa.
- Menu horizontal com Flexbox.
- Três cartões com `flex: 1`.
- Em ecrãs pequenos, cartões e menu passam a coluna (`@media`).

PASSOS SUGERIDOS

1. Escreve o HTML e testa-o sem CSS.
2. Estiliza o cabeçalho, o menu e o rodapé.
3. Cria os cartões com Flexbox.
4. Acrescenta o `@media` e testa com a barra de dispositivos.

RESULTADO ESPERADO



🚀 EXTENSÃO

Acrescenta uma segunda página (Contactos) com o formulário, mantendo o mesmo cabeçalho.

Mini-projeto 6 · Página interativa

HTML + CSS + JAVASCRIPT · CAPS. 16 E 17

Objetivo Criar uma página que reage às ações do utilizador.

Ficheiros `interativa/index.html`, `css/estilo.css`, `js/script.js`

REQUISITOS

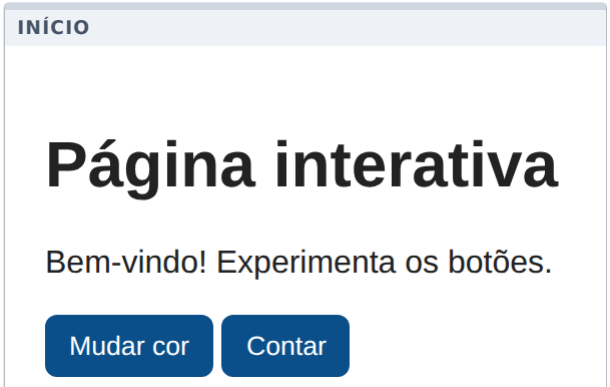
- Um botão que **alterna uma classe** (por exemplo, mudar as cores da página).
- Um botão que **conta** cliques e mostra o valor na página.
- JavaScript num ficheiro externo, ligado no fim do `body`.
- O aspeto alterado **definido em CSS**, não no JavaScript.

PASSOS SUGERIDOS

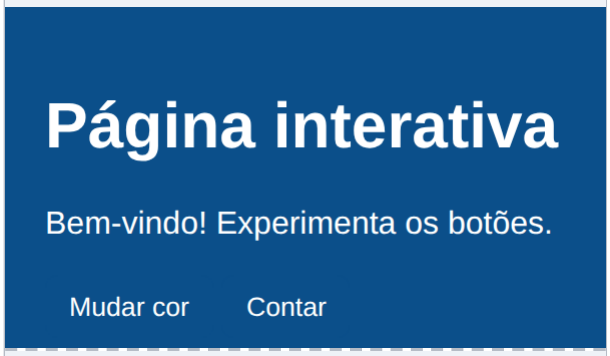
1. Cria o HTML com os botões e o texto.
2. Escreve a classe alternativa no CSS.
3. Seleciona os elementos com `querySelector`.
4. Liga cada botão a uma função com `addEventListener`.

RESULTADO ESPERADO

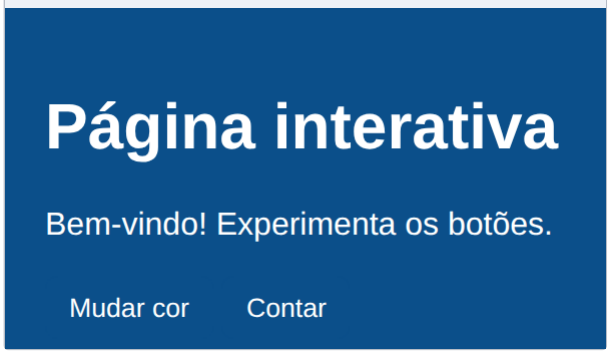
INÍCIO



DEPOIS DE «MUDAR COR»



DEPOIS DE «CONTAR» DUAS VEZES



> 🚀 **EXTENSÃO**
> Acrescenta um botão «Ler mais» que mostra e esconde um parágrafo, como no projeto do clube.

PARTE D · CAPÍTULO 22

Projeto final: o teu website

Chegou a hora de juntar tudo: constrói um **website completo e responsivo**, com o tema que quiseres.

Neste capítulo vais aprender a:

- planear e construir um pequeno website;
- aplicar HTML, CSS e JavaScript no mesmo projeto;
- avaliar o teu trabalho com uma grelha de critérios.

22.1 Enunciado

Cria um website com **três páginas**: uma página inicial (`index.html`) e mais **duas** à tua escolha (por exemplo, `galeria.html` e `contactos.html`). Escolhe um tema que te interesse: um clube, uma banda, uma receita, um desporto, uma viagem, um negócio fictício…

> 💡 **DICA**
> Podes usar o Clube de Fotografia (Anexo B) como **modelo de estrutura**, mas o tema, os textos, as imagens e o design devem ser **teus**.

22.2 Requisitos

	Requisito	Capítulos
☐	**3 páginas** ligadas entre si por um menu comum.	3, 6
☐	Ficheiros **CSS e JavaScript externos**, ligados corretamente.	9, 16
☐	Estrutura **semântica**: `header`, `nav`, `main`, `footer` (e `section` quando fizer sentido).	6
☐	Pelo menos **uma lista** e **imagens** com `alt` e caminhos relativos.	3
☐	Ligações **internas** e pelo menos uma **externa** (novo separador).	3
☐	Pelo menos **uma tabela** com `thead` e `tbody`.	4
☐	Um **formulário** com `label`, vários tipos de campo e botão de envio.	5
☐	Layout com **Flexbox** e **Grid** (por exemplo, menu e cartões com Flex; galeria com Grid).	13, 14
☐	Design **responsivo**: meta `viewport` e pelo menos uma `@media`.	15
☐	Pelo menos **uma funcionalidade em JavaScript** (modo escuro, mostrar/esconder, contador…).	16, 17
☐	Código **indentado**, comentado e organizado em pastas.	20

22.3 Plano de trabalho

Fase	O que fazer	Capítulos
1. Planear	Escolhe o tema, desenha à mão o esquema das três páginas e lista os conteúdos.	1, 18
2. Estrutura	Cria a pasta do projeto (com `css/`, `js/` e `imagens/`, como no Anexo B) e escreve o HTML das três páginas **sem CSS**. Testa as ligações.	2 a 8
3. Aparência	Liga o `estilo.css` e define cores, letra, espaços e menu.	9 a 12
4. Layout	Aplica Flexbox e Grid e torna o site responsivo.	13 a 15
5. Interatividade	Acrescenta pelo menos uma funcionalidade em JavaScript.	16, 17
6. Revisão	Percorre a lista de verificação do capítulo 20 e corrige os erros.	19, 20

22.4 Grelha de avaliação (20 valores)

Critério	Descrição	Valores
Estrutura HTML	Página base correta, elementos semânticos, títulos, listas, imagens e ligações que funcionam.	4
Tabela e formulário	Tabela bem construída; formulário completo, com `label` e vários tipos de campo.	2
CSS e aparência	Ficheiro externo; cores, tipo de letra, espaçamento e box model coerentes.	4
Layout	Flexbox e Grid usados de forma adequada.	3
Responsivo	Funciona em computador e telemóvel; `viewport` e `@media`.	2
JavaScript	Pelo menos uma funcionalidade que funciona sem erros na consola.	3
Boas práticas	Organização de ficheiros, indentação, comentários, nomes claros, sem caminhos absolutos.	2
Total		**20**

22.5 Entrega

1. Confirma que o site funciona a partir da pasta (abre o `index.html`).
2. Comprime a pasta **inteira** num ficheiro `.zip`.
3. Nomeia o ficheiro `nome_apelido_turma.zip` e entrega-o como indicado pelo professor.

> 🚀 **DESAFIO**
> Quando terminares, pede a um colega que abra o teu site no **telemóvel** e tente encontrar erros. Um olhar novo descobre sempre coisas que tu já não vês!

ANEXO A

Soluções dos desafios de código

Tenta **sempre resolver primeiro**! Só depois confirma a solução. Há muitas formas de escrever código correto: se a tua funciona e faz o que era pedido, está certa.

1. Introdução ao desenvolvimento Web

D1 · Qual será o resultado?

Os nomes têm **espaços** e **letras maiúsculas/acentos** (“Página Inicial”, “Meu Site”). O ideal seria `index.html` e `meu_site`.

2. HTML5: a estrutura da página

D2 · Qual será o resultado?

Aparece o título **“Bom dia”** e o parágrafo **“Segunda linha”**. O comentário e o parágrafo comentado **não aparecem**.

D3 · Encontra o erro

1) Falta a tag de fecho `</p>`; 2) as tags `<strong>` e `<p>` estão cruzadas — o `</strong>` deveria fechar antes de `</p>`.

D4 · Completa o código

`<html lang="pt">`, `<head>`, `<title>`, `<body>` e `</html>`.

3. Principais tags HTML

D5 · Qual será o resultado?

Uma linha com “Linha um” e “linha dois” em duas linhas, porque o `<br>` parte a linha; “dois” fica em **negrito**. Depois, uma linha horizontal e “H₂O” com o 2 em subscrito.

D6 · Encontra o erro

Em `<h1>Bem-vindo<h1>` a segunda tag devia ser de **fecho**: `</h1>`. Sem a barra, o navegador não fecha o título.

D7 · Completa o código

`<ol>`, `<li>Espanha</li>` e `</ol>`.

D8 · Corrige o código

O texto solto tem de ir dentro de `<li>`: `<li>Banana</li>`. Falta também o `</ul>`.

D9 · Corrige o código

`<a href="https://www.wikipedia.org" target="_blank">Wikipédia</a>`. Faltavam as aspas e o atributo `target="_blank"`.

D10 · Encontra o erro

Falta subir uma pasta: o caminho correto é `../imagens/escola.jpg`. Falta também o atributo `alt`.

D11 · Modifica o exemplo

```
<img src="imagens/escola.jpg" alt="Fotografia da escola" width="250">
```

D12 · Qual será o resultado?

Aparece a primeira (o caminho `imagens/praia.jpg` existe). A segunda **falha**: aparece o texto alternativo “Montanha”, porque `../imagens` sobe uma pasta acima da raiz do site.

4. Tabelas

D13 · Qual será o resultado?

Tem **2 linhas** e **2 colunas**. Na primeira linha aparecem os cabeçalhos **Nome** e **Idade** (a negrito e centrados); na segunda, **Rui** e **17**.

D14 · Completa o código

`<th colspan="2">Total</th>` e `<td>9 €</td>`.

D15 · Encontra o erro

A primeira linha tem **3 células** e a segunda só **2**. Falta uma célula (por exemplo, `<td>2 €</td>`) ou um `colspan`.

5. Formulários

D16 · Completa o código

```
<input type="email" id="email" name="email" placeholder="nome@exemplo.pt" required>
```

D17 · Encontra o erro

O `for="nome"` da label não coincide com o `id="name"` do campo. Têm de ser iguais (e o `name` deve existir para enviar os dados).

D18 · Qual será o resultado?

No grupo **turno** só **uma** (radio com o mesmo `name`). No grupo **hobby** pode escolher-se **várias** (checkbox).

6. HTML semântico

D19 · Completa o código

`header`, `nav`, `main` e `footer`, por esta ordem.

D20 · Qual será o resultado?

(a) `nav`; (b) `article`; (c) `aside`; (d) `footer`.

7. Multimédia

D21 · Encontra o erro

Falta o atributo `controls` na tag `<video>`.

8. iframe: conteúdo incorporado

D22 · Corrige o código

```
<iframe src="mapa.html" width="400" height="250" title="Mapa da escola"></iframe>
```

9. Introdução ao CSS

D23 · Completa o código

```
h2 { color: red; }
```

D24 · Encontra o erro

Dois: falta `;` a seguir a `green` (não é grave aqui, mas é boa prática) e a declaração `font-size` está **fora das chavetas**.

D25 · Corrige o código

`<link rel="stylesheet" href="css/estilo.css">`: faltava a pasta `css/` no caminho.

10. Seletores CSS

D26 · Completa o código

```
.nota { font-style: italic; }
```

D27 · Qual será o resultado?

Verde: o `id` é mais específico do que a classe.

D28 · Encontra o erro

O HTML usa `id="cartao"` e o CSS procura uma **classe**. Usa `class="cartao"` (ou `#cartao` no CSS).

11. Propriedades CSS mais usadas

D29 · Completa o código

```
color: white; background-color: blue; border-radius: 8px;
```

D30 · Qual será o resultado?

20px entre o texto e o contorno (padding) e **10px** entre o contorno e o vizinho (margin).

D31 · Modifica o exemplo

Acrescentar: `text-align: center; font-size: 20px; border: 3px solid white;`.

D32 · Corrige o código

Faltava o **estilo** do contorno: `border: 2px solid red;`.

12. O modelo de caixa (Box Model)

D33 · Qual será o resultado?

100 + 2×10 + 2×2 = **124px**.

D34 · Corrige o código

Acrescentar `box-sizing: border-box;`.

D35 · Completa o código

`margin: 0 auto;`

13. Flexbox

D36 · Completa o código

`display: flex; justify-content: center; gap: 10px;`

D37 · Qual será o resultado?

Numa coluna, empilhados verticalmente (o eixo principal passa a vertical) e com 8px entre eles.

D38 · Encontra o erro

Falta `display: flex` no **contentor**. `justify-content` só funciona em contentores flex.

D39 · Modifica o exemplo

`justify-content: flex-end;` na lista e `nav a:hover { background: #e06a1b; }`.

14. CSS Grid

D40 · Completa o código

`display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px;`

D41 · Qual será o resultado?

Duas colunas: a primeira ocupa **1/3** e a segunda **2/3** do espaço.

D42 · Corrige o código

Faltava `display: grid` (sem ele, `grid-template-columns` não faz nada).

15. Design responsivo

D43 · Completa o código

`@media (max-width: 600px) { body { background: gray; } }`

D44 · Qual será o resultado?

Uma coluna: 400 px é menor do que 480 px e a regra mais específica (a última) manda.

D45 · Encontra o erro

Falta a meta `viewport` no `<head>`.

D46 · Corrige o código

`img { max-width: 100%; height: auto; }`

16. Introdução ao JavaScript

D47 · Completa o código

`function saudar()` e `alert("Bem-vindo!")`; o botão usa `onclick="saudar()"`.

D48 · Qual será o resultado?

14: x começa em 5, passa a 7 e depois calcula 7 × 2.

D49 · Encontra o erro

Usa `=` (atribuição) dentro do `if`. Deve ser `nota === 15` (ou `>=`).

D50 · Corrige o código

Faltavam os **parênteses** no `onclick`: `onclick="dizerOla()"`.

D51 · Modifica o exemplo

`function mostrarMensagem(nome) { alert("Olá, " + nome + "!"); }` e `onclick="mostrarMensagem('Ana')"`.

17. JavaScript e DOM

D52 · Completa o código

`document.querySelector("#msg")` e `.textContent`.

D53 · Encontra o erro

O `<script>` está **antes** do botão: quando o código corre, o botão ainda não existe. Move o `<script>` para o fim do `<body>`.

D54 · Qual será o resultado?

Escondido: o primeiro clique remove a classe (fica visível) e o segundo volta a pô-la.

D55 · Corrige o código

Faltava o `#`: `document.querySelector("#titulo")`.

D56 · Modifica o exemplo

Exemplo: guarda uma variável `let praia = false;` e, a cada clique, troca o valor e o `src` conforme o valor.

19. Erros frequentes e como os resolver

D57 · Encontra o erro

Dois: o `href` devia ser `css/estilo.css` e a regra do `h1` não tem `;` a seguir a `blue`.

D58 · Encontra o erro

O caminho é absoluto do computador. Deve ser `imagens/foto.jpg`.

ANEXO B

Website de referência: código completo

Código completo do **Clube de Fotografia da Escola**, tal como fica no fim das 14 etapas. Cada bloco corresponde a um ficheiro do projeto (`clube_fotografia/`).

```
clube_fotografia/
├── index.html
├── galeria.html
├── contactos.html
├── css/estilo.css
├── js/script.js
└── imagens/ (escola, paisagem, praia, montanha, cidade, ponte, floresta)
```

B.1 index.html

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Clube de Fotografia da Escola</title>
    <link rel="stylesheet" href="css/estilo.css">
</head>
<body>

    <header>
        <h1>Clube de Fotografia da Escola</h1>
        <p>Captar o mundo, uma fotografia de cada vez.</p>
        <button id="btn-tema">Modo escuro</button>
        <nav>
            <ul>
                <li><a href="index.html">Início</a></li>
                <li><a href="galeria.html">Galeria</a></li>
                <li><a href="contactos.html">Contactos</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <section id="quem-somos">
            <h2>Quem somos</h2>
            <p>O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de
uma objetiva.</p>
            <p>Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com
fotografias.</p>
            <img src="imagens/escola.jpg" alt="Edifício da escola" width="360">
            <button id="btn-mais">Ler mais</button>
            <p id="mais" class="escondido">Fundado em 2018, o clube já organizou mais de 20 exposições
e participa todos os anos no concurso nacional de fotografia escolar.</p>
        </section>

        <section id="atividades">
            <h2>O que fazemos</h2>
            <p>Todas as semanas realizamos atividades para todos os níveis:</p>
            <div class="cartoes">
                <article class="cartao">
                    <h3>Saídas</h3>
                    <p>Fotografamos a cidade e a natureza.</p>
                </article>
                <article class="cartao">
```

```
                    <h3>Workshops</h3>
                    <p>Composição, luz e edição de imagem.</p>
                </article>
                <article class="cartao">
                    <h3>Exposições</h3>
                    <p>Mostramos os melhores trabalhos.</p>
                </article>
            </div>
        </section>

        <section id="participar">
            <h2>Como participar</h2>
            <ol>
                <li>Preenche o formulário de contacto</li>
                <li>Traz a tua câmara ou telemóvel</li>
                <li>Vem à primeira sessão!</li>
            </ol>
        </section>

        <section id="horario">
            <h2>Horário das sessões</h2>
            <table>
                <caption>Horário das sessões</caption>
                <thead>
                    <tr>
                        <th>Dia</th>
                        <th>Hora</th>
                        <th>Sala</th>
                    </tr>
                </thead>
                <tbody>
                    <tr>
                        <td>Terça-feira</td>
                        <td>15:30</td>
                        <td>Sala 12</td>
                    </tr>
                    <tr>
                        <td>Quinta-feira</td>
                        <td>16:00</td>
                        <td>Laboratório de Imagem</td>
                    </tr>
                </tbody>
            </table>
        </section>
    </main>

    <footer>
        <p>Saber mais: <a href="https://pt.wikipedia.org/wiki/Fotografia" target="_blank" rel="noopener
noreferrer">fotografia na Wikipédia</a></p>
        <p>&copy; 2026 Clube de Fotografia da Escola &middot; <a
href="mailto:clube@escola.pt">clube@escola.pt</a></p>
    </footer>

    <script src="js/script.js"></script>
</body>
</html>
```

B.2 galeria.html

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Galeria — Clube de Fotografia</title>
    <link rel="stylesheet" href="css/estilo.css">
</head>
<body>

    <header>
        <h1>Clube de Fotografia da Escola</h1>
        <p>Captar o mundo, uma fotografia de cada vez.</p>
        <nav>
            <ul>
                <li><a href="index.html">Início</a></li>
                <li><a href="galeria.html">Galeria</a></li>
                <li><a href="contactos.html">Contactos</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <section>
            <h2>As nossas fotografias</h2>
            <div class="galeria">
            <img src="imagens/paisagem.jpg" alt="Paisagem com colinas e sol">
            <img src="imagens/praia.jpg" alt="Praia com mar e areia">
            <img src="imagens/montanha.jpg" alt="Montanhas com neve no cume">
            <img src="imagens/cidade.jpg" alt="Cidade ao final da tarde">
            <img src="imagens/ponte.jpg" alt="Ponte ao pôr do sol">
            <img src="imagens/floresta.jpg" alt="Floresta de pinheiros">
            </div>
        </section>
    </main>

    <footer>
        <p>Saber mais: <a href="https://pt.wikipedia.org/wiki/Fotografia" target="_blank" rel="noopener
noreferrer">fotografia na Wikipédia</a></p>
        <p>&copy; 2026 Clube de Fotografia da Escola &middot; <a
href="mailto:clube@escola.pt">clube@escola.pt</a></p>
    </footer>

</body>
</html>
```

B.3 contactos.html

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contactos — Clube de Fotografia</title>
    <link rel="stylesheet" href="css/estilo.css">
</head>
<body>

    <header>
        <h1>Clube de Fotografia da Escola</h1>
        <p>Captar o mundo, uma fotografia de cada vez.</p>
        <nav>
            <ul>
                <li><a href="index.html">Início</a></li>
                <li><a href="galeria.html">Galeria</a></li>
                <li><a href="contactos.html">Contactos</a></li>
            </ul>
        </nav>
    </header>
```

```
    <main>
        <section>
            <h2>Fala connosco</h2>
            <form action="#" method="post">
                <label for="nome">Nome</label>
                <input type="text" id="nome" name="nome" placeholder="O teu nome" required>

                <label for="email">Email</label>
                <input type="email" id="email" name="email" placeholder="nome@exemplo.pt" required>

                <label for="assunto">Assunto</label>
                <select id="assunto" name="assunto">
                    <option value="inscricao">Quero inscrever-me</option>
                    <option value="info">Pedido de informação</option>
                    <option value="outro">Outro assunto</option>
                </select>

                <label for="mensagem">Mensagem</label>
                <textarea id="mensagem" name="mensagem" rows="4"></textarea>

                <button type="submit">Enviar mensagem</button>
            </form>
        </section>
    </main>

    <footer>
        <p>Saber mais: <a href="https://pt.wikipedia.org/wiki/Fotografia" target="_blank" rel="noopener
noreferrer">fotografia na Wikipédia</a></p>
        <p>&copy; 2026 Clube de Fotografia da Escola &middot; <a
href="mailto:clube@escola.pt">clube@escola.pt</a></p>
    </footer>

</body>
</html>
```

B.4 css/estilo.css

estilo.css · parte 1 de 6

```
/* ===== Base ===== */
* { box-sizing: border-box; }

body {
    font-family: Arial, Helvetica, sans-serif;
    line-height: 1.6;
    color: #222;
    margin: 0;
    background: #f4f6f9;
}

header {
    background: #0b4f8a;
    color: #fff;
    padding: 20px;
}

header a { color: #fff; }

header h1 { margin: 0 0 8px; }

main {
    max-width: 900px;
    margin: 0 auto;
    padding: 20px;
```

```
}

h2 {
    color: #0b4f8a;
    border-bottom: 2px solid #d0d9e4;
    padding-bottom: 4px;
}

img {
    max-width: 100%;
    height: auto;
    border-radius: 8px;
}

table {
    width: 100%;
    border-collapse: collapse;
    background: #fff;
}

th, td {
    border: 1px solid #ccd3dc;
    padding: 8px;
    text-align: left;
}

th { background: #0b4f8a; color: #fff; }

footer {
    background: #222;
    color: #ddd;
    text-align: center;
    padding: 16px;
    margin-top: 20px;
}

footer a { color: #ffb37a; }

/* ===== Formulário ===== */
label { display: block; margin-top: 12px; font-weight: bold; }

input, select, textarea {
    width: 100%;
    padding: 8px;
    border: 1px solid #b5bdc9;
    border-radius: 6px;
    font: inherit;
}

button {
    background: #e06a1b;
    color: #fff;
    border: 0;
    padding: 10px 18px;
    border-radius: 6px;
    cursor: pointer;
    font: inherit;
    margin-top: 12px;
}
```

estilo.css · parte 2 de 6

```
/* ===== Menu de navegação (com Flexbox) ===== */
nav ul {
    list-style: none;
    margin: 0;
```

```
    padding: 0;
    display: flex;
    gap: 10px;
}

nav a {
    display: block;
    color: #fff;
    text-decoration: none;
    padding: 8px 14px;
    background: rgba(255, 255, 255, 0.15);
    border-radius: 6px;
}

nav a:hover { background: #e06a1b; }
```

estilo.css · parte 3 de 6

```
/* ===== Cartões (Flexbox) ===== */
.cartoes {
    display: flex;
    gap: 16px;
}

.cartao {
    flex: 1;
    background: #fff;
    padding: 14px;
    border-radius: 8px;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.12);
}

.cartao h3 { margin-top: 0; color: #0b4f8a; }
```

estilo.css · parte 4 de 6

```
/* ===== Galeria (CSS Grid) ===== */
.galeria {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}

.galeria img { width: 100%; }
```

estilo.css · parte 5 de 6

```
/* ===== Responsivo ===== */
@media (max-width: 768px) {
    .cartoes { flex-direction: column; }
    .galeria { grid-template-columns: repeat(2, 1fr); }
}

@media (max-width: 480px) {
    nav ul { flex-direction: column; }
    .galeria { grid-template-columns: 1fr; }
}
```

estilo.css · parte 6 de 6

```
/* ===== Interatividade (JavaScript) ===== */
.escondido { display: none; }

body.escuro { background: #14171c; color: #e5e7eb; }
body.escuro .cartao,
body.escuro table { background: #222831; }
body.escuro h2,
```

```
body.escuro .cartao h3 { color: #7db7ee; }
body.escuro td { border-color: #3a4350; }
```

B.5 js/script.js

```
// ===== Modo escuro =====
const botaoTema = document.querySelector("#btn-tema");

botaoTema.addEventListener("click", function () {
    document.body.classList.toggle("escuro");
});

// ===== Ler mais / Ler menos =====
const botaoMais = document.querySelector("#btn-mais");
const textoMais = document.querySelector("#mais");

botaoMais.addEventListener("click", function () {
    textoMais.classList.toggle("escondido");

    if (textoMais.classList.contains("escondido")) {
        botaoMais.textContent = "Ler mais";
    } else {
        botaoMais.textContent = "Ler menos";
    }
});
```

B.6 Imagens do resultado

Cada página em **computador** (esquerda) e em **telemóvel** (direita). As imagens dos telemóveis aparecem recortadas.

index.html

COMPUTADOR (1024 PX)

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

Modo escuro

Início Galeria Contactos

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



Ler mais

O que fazemos

Todas as semanas realizamos atividades para todos os níveis:

Saídas

Fotografamos a cidade e a natureza.

Workshops

Composição, luz e edição de imagem.

Exposições

Mostramos os melhores trabalhos.

Como participar

1. Preenche o formulário de contacto
2. Traz a tua câmara ou telemóvel
3. Vem à primeira sessão!

Horário das sessões

Horário das sessões

Dia	Hora	Sala
Terça-feira	15:30	Sala 12
Quinta-feira	16:00	Laboratório de Imagem

TELEMÓVEL (375 PX)

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

Modo escuro

Início

Galeria

Contactos

Quem somos

O Clube de Fotografia reúne alunos e professores que gostam de captar o mundo através de uma objetiva.

Nas sessões aprendemos a compor imagens, a usar a luz e a contar histórias com fotografias.



Ler mais

O que fazemos

Todas as semanas realizamos atividades para todos os níveis:

Saídas

Fotografamos a cidade e a natureza.

Workshops

Composição, luz e edição de imagem.

galeria.html

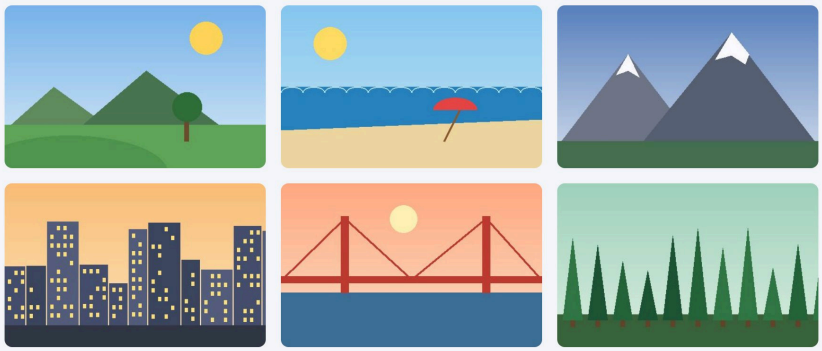
COMPUTADOR (1024 PX)

Clube de Fotografia da Escola

Captar o mundo, uma fotografia de cada vez.

Início | Galeria | Contactos

As nossas fotografias



Saber mais: fotografia na Wikipédia

© 2026 Clube de Fotografia da Escola · clube@escola.pt

TELEMÓVEL (375 PX)

Clube de Fotografia da Escola

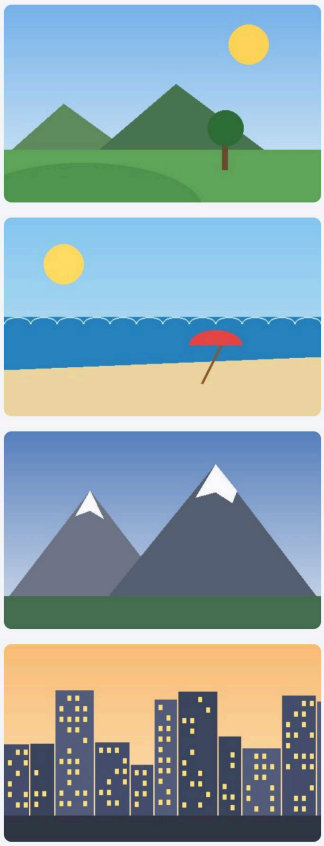
Captar o mundo, uma fotografia de cada vez.

Início

Galeria

Contactos

As nossas fotografias



contactos.html

COMPUTADOR (1024 PX)



TELEMÓVEL (375 PX)

